SCTR's Pune Institute of Computer Technology, Pune - 411043

Department of Electronics & Computer Engineering | BE | Assessment Year 2026-2027

Laboratory Practice I - Data Science & Visualization [410346]

MASTER JOURNAL: Experiments 1 to 13

Each experiment: Theory > Algorithm / Test cases > Program, Output & Practice problems > Important Q&A > Conclusion

Expt	Title	Pages	Q&A on page
1	Data Manipulation in Python using Pandas	2 - 17	13
2	Mean, Median, Variance, Correlation and Normal Distribution	19 - 38	35
3	CSV Import / Export, Data Analysis, Confusion Matrix	40 - 56	53
4	Data Visualization using Matplotlib	58 - 75	72
5	Stock Market Analysis using Pandas (Tesla data)	77 - 94	91
6	Titanic Dataset - EDA using Seaborn	96 - 114	111
7	Titanic Survival Prediction - Training and Testing	116 - 131	127
8	Simple and Multiple Linear Regression (scikit-learn)	133 - 146	143
9	K-Means Clustering (Mall Customers)	148 - 160	157
10	Decision Tree Classifier (Iris)	162 - 174	172
11	AND / NAND Gate using Feed-Forward Neural Network	176 - 184	182
12	OR / NOR Gate using Feed-Forward Neural Network	186 - 194	192
13	EX-OR Gate using Feed-Forward Neural Network	196 - 205	203

One blank page follows every experiment. Page numbers refer to the small "Master p." number at the bottom centre.
Header fields (Roll No, dates, LAB Ref), Conclusion and Signature are left blank on every experiment for you to fill by hand.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:1	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	**Submission date:**
PROBLEM STATEMENT	Data Manipulation in Python using Pandas.	
PREREQUISITES	Basic knowledge of Python syntax (variables, loops, functions) Understanding of data structures (lists, dictionaries) Basic concept of tabular data (rows and columns)	
OBJECTIVES:	• To understand the creation and structure of Pandas Series and DataFrames. • To perform data manipulation operations such as filtering, sorting, grouping, and merging using Pandas. • To apply indexing, slicing, and aggregation functions for real-time data analysis.	
Tools Used	• Python Version: Python 3.x • IDE/Editor: Jupyter Notebook / Anaconda and Google Colab • Libraries: pandas, numpy	
THEORY:		
(Pandas library and its various functions used for data manipulation and analysis) (Numpy library and its various functions used for data manipulation and analysis		
	Data manipulation refers to the process of transforming, cleaning, and reorganizing data to make it suitable for analysis, visualization, and further processing. In data science, data manipulation is a critical step in the data preprocessing phase, where raw data is refined and structured to extract meaningful insights efficiently. This process often involves various operations, such as filtering, merging, reshaping, and aggregating data. Some typical data manipulation tasks are: • **Data Cleaning:** Identifying and handling missing values, outliers, and inconsistencies in the data to ensure accuracy and reliability. • **Data Transformation:** Converting data from one format to another, changing data types, and scaling numerical values to bring them into a range. • **Filtering and Sub-setting:** Selecting specific rows or columns based on certain conditions to focus on relevant data for analysis. • **Data Aggregation:** Combining data into groups and calculating summary statistics (e.g., mean, sum, count) for each group. • **Data Joining and Merging:** Combining data from multiple sources based on	

common attributes to construct a unified dataset.

- **Pivoting and Reshaping:** Reorganizing data to change its structure, such as moving rows to columns or columns to rows (or "pivoting") to see different summaries of the source data.
- **Data Encoding:** Converting categorical variables into numerical representations for analysis.
- **Feature Engineering:** Creating new features or variables from existing data that may improve the performance of machine learning models.
- **Data Imputation:** Fill in missing values using various techniques to maintain the integrity of the dataset.
- **Data Normalization:** Scaling numerical data to a standard range, often between 0 and 1, to prevent the dominance of certain features.

The quality of data manipulation directly impacts the accuracy and reliability of any data analysis or machine learning models built on the processed data. Therefore, data scientists spend considerable period of time and effort on data manipulation to ensure that the data is in the most suitable form for meaningful insights and predictions.

Python offers several powerful data manipulation libraries that are widely used in the data science community. These libraries provide various tools and functions for data cleaning, transformation, analysis, and visualization. Here are some of the most popular data manipulation libraries in Python:

1. **Pandas:**
   Pandas is a fundamental data manipulation library in Python. It provides data structures like DataFrame and Series, enabling the smooth handling of structured data. Pandas provides a set of tools to facilitate data cleaning, filtering, merging, grouping, and aggregation.
2. **NumPy:**
   While NumPy is primarily known for its numerical computing capabilities, it also plays a significant role in data manipulation. It provides support for arrays and matrices, enabling efficient manipulation of large datasets.
3. **SciPy:**
   SciPy builds on NumPy and provides additional scientific computing functionalities, including statistical functions, optimization, integration, interpolation, and more.

	4. **Dask:** Dask is a parallel computing library that extends the capabilities of Pandas and NumPy. It allows you to work with larger-than-memory datasets by performing operations in parallel and leveraging distributed computing resources.
REFERENCES: Refer lab manual for the same	
	Jake VanderPlas – Python Data Science Handbook, Ch. 3 – Data Manipulation with Pandas **Joel Grus – Data Science from Scratch, Ch. 10 – Working with Data**

Practice Problem Statements:

1. Create a DataFrame of 6 products with columns Name, Category, Price, and Stock. Filter products with Stock < 10.
2. Using the employee DataFrame, find the employee with the maximum salary using idxmax().
3. Create two DataFrames one with student names/roll numbers and another with their marks — and merge them using an inner join.
4. Add a new column 'Experience_Level' to the employee DataFrame using a conditional (np.where) based on Age.
5. Group a sales DataFrame by 'Region' and compute both the sum and count of 'Sales' using .agg().

Important Questions: -

1. What is the difference between a Pandas Series and a DataFrame?
2. Explain the difference between loc[] and iloc[] with examples.
3. How does groupby() work internally in Pandas? Explain the split-apply-combine strategy.
4. What are the different types of joins supported by pd.merge()? Explain each with an example.
5. How do you handle duplicate rows in a DataFrame?
6. Differentiate between df.sort_values() and df.sort_index().
7. What is the difference between concat() and merge() in Pandas?
8. How can you add, rename, and delete columns in a DataFrame?

SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE - 411043

Department of Electronics & Computer Engineering

ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**

SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]

EXPT No:1

PROBLEM STATEMENT

Data Manipulation in Python using Pandas.

ALGORITHM

Steps of data manipulation with Pandas:

1. Import **pandas** and **numpy**. Create a Series / DataFrame from lists and dictionaries (or read the data from a file).
2. Inspect the structure of the data: `df.shape`, `df.head()`, `df.info()`, `df.dtypes`.
3. Select data: label-based `df.loc[row, col]`, position-based `df.iloc[r, c]` and Boolean filtering `df[df['Marks'] > 75]`.
4. Modify columns: add a column, create a conditional column with `np.where(cond, a, b)`, rename and drop columns.
5. Clean the data: find duplicates with `df.duplicated()` and remove them with `drop_duplicates()`.
6. Sort the data with `sort_values(by=...)` or `sort_index()`.
7. Group and aggregate (split-apply-combine): `df.groupby('Region')['Sales'].agg(['sum', 'count'])`.
8. Combine tables with `pd.merge(df1, df2, on='key', how='inner')` (left / right / outer) or `pd.concat()`.
9. Display the result, verify its shape and note the observations.

ERRORS and TEST CASES, USE CASES

Common errors and their remedies

Error / symptom	Cause	Remedy
KeyError: 'Marks'	Column name is misspelt, has a different case or a trailing space.	Check `df.columns`; use the exact name or `df.rename()`.
ValueError: Length of values does not match length of index	A new column is given fewer / more values than the number of rows.	Supply exactly `len(df)` values or an index-aligned Series.
IndexError: single positional indexer is out-of-bounds	`iloc` position is beyond the number of rows / columns.	Check `df.shape` before using `iloc`.
ValueError: You are trying to merge on int64 and object columns	The join keys have different data types.	Convert both keys with `astype()` before merging.
SettingWithCopyWarning	Assignment is made on a slice of another DataFrame.	Use `.copy()` or `.loc[row, col] = value`.
Unexpected NaN values after a merge	Keys present in only one table (left / outer join).	Choose the proper `how=`; compare keys with `isin()`.

SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE - 411043

Department of Electronics & Computer Engineering

ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**

SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]

EXPT No:1

PROBLEM STATEMENT Data Manipulation in Python using Pandas.

Test cases

ID	Input / condition	Expected result
T1	Series [10, 20, 30, 40] with index a, b, c, d; `s['b']`	20
T2	`df.loc[1, 'Name']` on the 4-student table (Amit, Priya, Rahul, Sneha)	'Priya'
T3	`df.shape` of 4 students x 3 columns	(4, 3)
T4	Filter Marks > 75 on marks 85, 78, 92, 70	3 rows: Amit, Priya, Rahul
T5	`idxmax()` on the Salary column	Index label of the row with the highest salary
T6	Inner merge when one roll number exists in only one table	That roll number is dropped (a left merge keeps it with NaN)
T7	`groupby('Region').agg(['sum','count'])`	One row per region; sum of the counts = total number of rows
T8	A DataFrame with one repeated row, then `drop_duplicates()`	Row count reduces by 1
T9	(Negative test) `df['Marks ']` with a trailing space	KeyError is raised

Use cases

- Cleaning and preparing sales, customer or student records before analysis.
- Merging data from different departments, e.g. students with their marks, orders with customers.
- Region-wise or category-wise summary reports using `groupby`.
- Filtering low-stock products, top performers or defaulters from large tables.
- Creating new features (experience level, age group) for machine-learning models.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:1	
PROBLEM STATEMENT	Data Manipulation in Python using Pandas.

Programs are written in Python using the Pandas and NumPy libraries and executed in Jupyter Notebook. Sample DataFrames required by the practice problems are created inside the programs.

PROGRAM AND OUTPUT

Import libraries

Code:

```
import pandas as pd
import numpy as np
pd.set_option("display.width", 85)
print("Pandas version:", pd.__version__)
print("NumPy version :", np.__version__)
```

Output:

```
Pandas version: 3.0.5
NumPy version : 2.5.3
```

Pandas Series and DataFrame

Code:

```
s = pd.Series([10, 20, 30, 40], index=["a", "b", "c", "d"])
print("Series:")
print(s)

df = pd.DataFrame({"Name": ["Amit", "Priya", "Rahul", "Sneha"],
                   "Age": [20, 21, 20, 22],
                   "Marks": [85, 78, 92, 70]})
print("\nDataFrame:")
print(df)
print("\nShape of DataFrame:", df.shape)

print("\ndf.loc[1, 'Name']  ->", df.loc[1, "Name"])
print("df.iloc[0:2, 0:2] ->")
print(df.iloc[0:2, 0:2])
print("\nStudents with Marks > 75:")
print(df[df["Marks"] > 75])
```

Output:

```
Series:
a    10
b    20
c    30
d    40
dtype: int64

DataFrame:
    Name  Age  Marks
0   Amit   20     85
1  Priya   21     78
```

```
2  Rahul   20     92
3  Sneha   22     70

Shape of DataFrame: (4, 3)

df.loc[1, 'Name']  -> Priya
df.iloc[0:2, 0:2] ->
    Name  Age
0   Amit   20
1  Priya   21

Students with Marks > 75:
    Name  Age  Marks
0   Amit   20     85
1  Priya   21     78
2  Rahul   20     92
```

Problem 1: products, Stock < 10

Code:

```
products = pd.DataFrame({
    "Name":     ["Laptop", "Mouse", "Keyboard", "Monitor", "Printer", "Webcam"],
    "Category": ["Computers", "Accessories", "Accessories", "Computers", "Office",
"Accessories"],
    "Price":    [55000, 500, 1200, 12000, 8500, 2500],
    "Stock":    [5, 40, 25, 8, 12, 3]})
print("All products:")
print(products)

low_stock = products[products["Stock"] < 10]
print("\nProducts with Stock < 10:")
print(low_stock)
```

Output:

```
All products:
       Name     Category  Price  Stock
0    Laptop    Computers  55000      5
1     Mouse  Accessories    500     40
2  Keyboard  Accessories   1200     25
3   Monitor    Computers  12000      8
4   Printer       Office   8500     12
5    Webcam  Accessories   2500      3

Products with Stock < 10:
      Name     Category  Price  Stock
0   Laptop    Computers  55000      5
3  Monitor    Computers  12000      8
5   Webcam  Accessories   2500      3
```

EXPT No:1	
PROBLEM STATEMENT	Data Manipulation in Python using Pandas.

Problem 2: idxmax

Code:

```
emp = pd.DataFrame({
    "EmpID":  [101, 102, 103, 104, 105, 106],
    "Name":   ["Ravi", "Anita", "Karan", "Meera", "Suresh", "Neha"],
    "Age":    [24, 31, 45, 29, 38, 52],
    "Dept":   ["IT", "HR", "IT", "Finance", "Sales", "Finance"],
    "Salary": [40000, 52000, 95000, 61000, 58000, 120000]})
print(emp)

idx = emp["Salary"].idxmax()
print("\nIndex of maximum salary:", idx)
print("Employee with maximum salary:")
print(emp.loc[idx])
```

Output:

```
   EmpID    Name  Age     Dept  Salary
0    101    Ravi   24       IT   40000
1    102   Anita   31       HR   52000
2    103   Karan   45       IT   95000
3    104   Meera   29  Finance   61000
4    105  Suresh   38    Sales   58000
5    106    Neha   52  Finance  120000

Index of maximum salary: 5
Employee with maximum salary:
EmpID         106
Name         Neha
Age            52
Dept      Finance
Salary     120000
Name: 5, dtype: object
```

Problem 3: inner merge

Code:

```
students = pd.DataFrame({"RollNo": [1, 2, 3, 4, 5],
                         "Name": ["Amit", "Priya", "Rahul", "Sneha", "Vikas"]})
marks = pd.DataFrame({"RollNo": [2, 3, 4, 5, 6],
                      "Marks": [78, 92, 70, 66, 88]})
print("Students:")
print(students)
print("\nMarks:")
print(marks)

result = pd.merge(students, marks, on="RollNo", how="inner")
print("\nInner join result:")
print(result)
```

Output:

```
Students:
   RollNo   Name
0       1   Amit
1       2  Priya
2       3  Rahul
3       4  Sneha
4       5  Vikas

Marks:
   RollNo  Marks
0       2     78
1       3     92
2       4     70
3       5     66
4       6     88

Inner join result:
   RollNo   Name  Marks
0       2  Priya     78
1       3  Rahul     92
2       4  Sneha     70
3       5  Vikas     66
```

Problem 4: Experience_Level using np.where

Code:

```
emp["Experience_Level"] = np.where(emp["Age"] > 35, "Senior", "Junior")
print(emp[["Name", "Age", "Experience_Level"]])
```

Output:

```
     Name  Age Experience_Level
0    Ravi   24           Junior
1   Anita   31           Junior
2   Karan   45           Senior
3   Meera   29           Junior
4  Suresh   38           Senior
5    Neha   52           Senior
```

Problem 5: groupby with agg

Code:

```
sales = pd.DataFrame({
    "Region": ["East", "West", "East", "North", "West", "North", "East", "South"],
    "Sales":  [250, 400, 150, 300, 200, 500, 350, 275]})
print(sales)

summary = sales.groupby("Region")["Sales"].agg(["sum", "count"])
print("\nRegion-wise sum and count of Sales:")
print(summary)
```

Output:

```
  Region  Sales
0   East    250
1   West    400
2   East    150
3  North    300
4   West    200
5  North    500
6   East    350
7  South    275

Region-wise sum and count of Sales:
        sum  count
Region
East    750      3
North   800      2
South   275      1
West    600      2
```

Additional operations

Code:

```
print("Sorted by Salary (descending):")
print(emp.sort_values(by="Salary", ascending=False)[["Name", "Salary"]])

emp["Bonus"] = emp["Salary"] * 0.10
emp = emp.rename(columns={"Salary": "Pay"})
print("\nAfter adding Bonus and renaming Salary -> Pay:")
print(emp[["Name", "Pay", "Bonus"]].head(3))
emp = emp.drop(columns=["Bonus"])
print("\nColumns after dropping Bonus:", list(emp.columns))

d = pd.DataFrame({"A": [1, 2, 2, 3], "B": ["x", "y", "y", "z"]})
print("\nDuplicated rows:", d.duplicated().sum())
print(d.drop_duplicates())

print("\nconcat of two DataFrames:")
print(pd.concat([d.head(2), d.tail(2)], ignore_index=True))
```

Output:

```
Sorted by Salary (descending):
     Name  Salary
5    Neha  120000
2   Karan   95000
3   Meera   61000
4  Suresh   58000
1   Anita   52000
0    Ravi   40000

After adding Bonus and renaming Salary -> Pay:
```

EXPT No:1	
PROBLEM STATEMENT	Data Manipulation in Python using Pandas.

```
    Name    Pay   Bonus
0   Ravi  40000  4000.0
1  Anita  52000  5200.0
2  Karan  95000  9500.0

Columns after dropping Bonus: ['EmpID', 'Name', 'Age', 'Dept', 'Pay', 'Experience_Level']

Duplicated rows: 1
   A  B
0  1  x
1  2  y
3  3  z

concat of two DataFrames:
   A  B
0  1  x
1  2  y
2  2  y
3  3  z
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:1	
PROBLEM STATEMENT	Data Manipulation in Python using Pandas.

IMPORTANT QUESTIONS AND ANSWERS

Q1. What is the difference between a Pandas Series and a DataFrame?

A **Series** is a one-dimensional labelled array that holds data of a single type. It is like one column of a table. A **DataFrame** is a two-dimensional labelled table made up of rows and columns, where every column is a Series and different columns can hold different data types.

Series	DataFrame
One-dimensional	Two-dimensional (rows and columns)
Holds one data type	Each column can have a different data type
Has only an index	Has a row index and column labels
Created with pd.Series()	Created with pd.DataFrame()
Size is the number of elements	Shape is (rows, columns)

```
import pandas as pd
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
df = pd.DataFrame({'Name': ['Amit', 'Priya'], 'Marks': [85, 78]})
print(df['Marks'])      # one column of a DataFrame is a Series
```

Q2. Explain the difference between loc[] and iloc[] with examples.

loc[] selects data using row and column **labels**, and the end label of a slice is **included**. **iloc[]** selects data using integer **positions** (0, 1, 2, ...), and the end position of a slice is **excluded**.

```
df = pd.DataFrame({'Name': ['A', 'B', 'C', 'D'], 'Marks': [70, 80, 90, 60]},
                  index=['r1', 'r2', 'r3', 'r4'])
df.loc['r2']                  # row with label r2
df.loc['r1':'r3', 'Marks']    # r1, r2 and r3 (r3 included)
df.iloc[1]                    # second row (position 1)
df.iloc[0:3, 1]               # rows 0, 1, 2 (3 not included), column 1
df.loc[df['Marks'] > 65]      # loc also accepts a boolean condition
```

Q3. How does groupby() work internally in Pandas? Explain the split-apply-combine strategy.

groupby() follows the **split-apply-combine** strategy:

1. **Split** - the DataFrame is divided into groups based on the unique values of one or more key columns.
2. **Apply** - a function (sum, mean, count, or a custom function) is applied to each group independently.

3. **Combine** - the results of all the groups are combined into a new Series or DataFrame.

```
sales = pd.DataFrame({'Region': ['East', 'West', 'East', 'West', 'North'],
                      'Sales':  [100, 200, 150, 50, 300]})
sales.groupby('Region')['Sales'].agg(['sum', 'count'])
#          sum  count
# East     250      2
# North    300      1
# West     250      2
```

The groupby() call itself does not compute anything; it only creates a GroupBy object. The actual computation happens when an aggregation such as sum() or agg() is called on it.

Q4. What are the different types of joins supported by pd.merge()? Explain each with an example.

```
a = pd.DataFrame({'Roll': [1, 2, 3], 'Name': ['Amit', 'Priya', 'Rahul']})
b = pd.DataFrame({'Roll': [2, 3, 4], 'Marks': [78, 92, 65]})
```

Join (how=)	Meaning	Result for a and b on Roll
inner (default)	Only keys present in both tables	Roll 2, 3
left	All keys of left table, matching rows from right (NaN if none)	Roll 1, 2, 3 (Marks of 1 is NaN)
right	All keys of right table, matching rows from left	Roll 2, 3, 4 (Name of 4 is NaN)
outer	All keys from both tables, NaN where there is no match	Roll 1, 2, 3, 4

```
pd.merge(a, b, on='Roll', how='inner')
pd.merge(a, b, on='Roll', how='left')
pd.merge(a, b, on='Roll', how='right')
pd.merge(a, b, on='Roll', how='outer')
```

Pandas also supports how='cross', which gives the Cartesian product of the two tables.

Q5. How do you handle duplicate rows in a DataFrame?

Duplicates are found with **duplicated()** and removed with **drop_duplicates()**.

EXPT No:1	
PROBLEM STATEMENT	Data Manipulation in Python using Pandas.

```
df.duplicated()                          # True for every repeated row
df.duplicated().sum()                    # number of duplicate rows
df.drop_duplicates()                     # keeps the first occurrence
df.drop_duplicates(subset=['Name'])      # check only selected columns
df.drop_duplicates(keep='last')          # keep last (or keep=False to drop
all)
df.drop_duplicates(inplace=True)         # change the original DataFrame
```

Q6. Differentiate between df.sort_values() and df.sort_index().

sort_values()	sort_index()
Sorts by the values of one or more columns	Sorts by the row index labels (or column labels with axis=1)
Needs the 'by' argument: df.sort_values(by='Marks')	No column needed: df.sort_index()
Can sort by several columns: by=['A', 'B']	Sorts only by index
Used to rank or order records	Used to restore original order after filtering or shuffling

```
df.sort_values(by='Marks', ascending=False)
df.sort_index()
```

Q7. What is the difference between concat() and merge() in Pandas?

concat()	merge()
Stacks DataFrames one below the other (axis=0) or side by side (axis=1)	Combines DataFrames by matching values of key column(s), like a SQL join
Works on the index / position; no key is needed	Needs a common column (on=) or index
Can take a list of many DataFrames	Joins two DataFrames at a time
Used to append rows or add columns	Used to combine related data from two tables

```
pd.concat([df1, df2], axis=0, ignore_index=True)   # add rows
pd.concat([df1, df2], axis=1)                      # add columns
pd.merge(df1, df2, on='Roll', how='inner')
```

Q8. How can you add, rename, and delete columns in a DataFrame?

```
# Add
df['Bonus'] = df['Salary'] * 0.10
df['Level'] = np.where(df['Age'] > 30, 'Senior', 'Junior')
df.insert(1, 'City', 'Pune')            # insert at position 1

# Rename
df = df.rename(columns={'Salary': 'Pay', 'Age': 'Years'})
df.columns = ['A', 'B', 'C']             # rename all columns at once

# Delete
df = df.drop(columns=['Bonus'])          # or df.drop('Bonus', axis=1)
del df['City']
df.pop('Level')                          # removes and returns the column
```

CONCLUSION:
Signature:

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 **Department of Electronics & Computer Engineering** **ASSESSMENT YEAR: 2026-2027** | **CLASS: BE** **SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]**	
EXPT No:2	**LAB Ref: BE/2026-27/**	**Date of Performance:**
	Roll No:	
PROBLEM STATEMENT	Calculating Mean, Median, variance and plotting Correlation and Normal Distribution of a data using Python.	
PREREQUISITES	Basic statistics concepts (mean, median, variance, standard deviation) Understanding of correlation and normal distribution theory Basic Pandas DataFrame operations (from Practical 1)	
OBJECTIVES:	To compute statistical measures (mean, median, variance, standard deviation) for a given dataset. To visualize the correlation between variables using a heatmap. To plot and interpret the normal distribution curve of a dataset.	
Tools Used	• Python 3.x • Jupyter Notebook • Libraries:numpy,pandas, matplotlib,seaborn,scipy	
THEORY:		
	• **Introduction to Statistics:** Statistics is the branch of mathematics concerned with collecting, organizing, analyzing, interpreting and presenting data. In Data Science, statistical measures help us understand the central tendency, variability and relationships present in a dataset. The important statistical measures used in this experiment are: • Mean • Median • Variance • Standard deviation • Correlation • Normal distribution **Mean:** The **mean** is the arithmetic average of a set of observations. For a dataset: $X=\{x_1,x_2,x_3,\ldots,x_n\}$ the mean is calculated as:	

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

where:

- x_i = individual observation
- n = number of observations
- $\bar{x}$ = arithmetic mean

Mean gives a measure of the central value of the dataset.

Median: The **median** is the middle value of a dataset after arranging the observations in ascending or descending order.

Case 1: Odd number of observations

For:

10,20,30,40,50

the middle value is:

Median=30

Case 2: Even number of observations

For:

10,20,30,40

the median is the average of the two middle values:

Median=220+30=25

Importance of Median

Median is less affected by extreme values than mean.

Therefore, median is particularly useful for datasets containing outliers or skewed data

Variance: Variance measures how far the observations in a dataset are spread from the mean.

For a population:

$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2$$

For a sample:

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

where:

- xi = individual observation

- μ = population mean
- $\bar{x}$ = sample mean
- N = population size
- n = sample size
- Low variance → observations are close to the mean.
- High variance → observations are widely spread around the mean.

Variance is expressed in squared units of the original data.

Standard Deviation: Standard deviation is the **square root of variance**.

$$\sigma = \sqrt{\sigma^2}$$

Standard deviation is easier to interpret than variance because it is expressed in the same units as the original data. If the average examination score is 70 and the standard deviation is 5, most observations are relatively close to 70. If the standard deviation is 20, the scores have much greater variation.

Correlation: Correlation measures the **strength and direction of the relationship between two variables**. The commonly used measure is the **Pearson correlation coefficient**, represented by r.

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

The value of r lies between:

$-1 \leq r \leq +1$

Positive correlation: If one variable tends to increase when another variable increases, the variables have a **positive correlation**.

Example: Study hours and examination marks

Negative correlation: If one variable tends to decrease when another variable increases, the variables have a **negative correlation**.

Example: Price and quantity demanded

Correlation describes an association between variables. It does not by itself establish a cause-and-effect relationship.

Correlation Matrix: When a dataset contains several numerical attributes, correlation can be calculated between every pair of attributes. The result is called a **correlation matrix**.

Visualization of Correlation Using Heatmap: A **heatmap** represents numerical values

using different levels of visual intensity.

A correlation matrix can be represented using a heatmap to make relationships between variables easier to identify.

A heatmap helps identify:

- Strong positive relationships
- Strong negative relationships
- Weak relationships

This experiment therefore combines statistical analysis with data visualization.

Example: **sns.heatmap(data.corr(), annot=True)**

data.corr() calculates the correlation matrix.

sns.heatmap() creates the graphical representation.

annot=True displays the correlation values.

Probability and Normal Distribution: Probability provides a mathematical framework for describing uncertainty. In data science, probability distributions are used to describe how values of a random variable are distributed.

Normal Distribution :

A normal distribution is characterized by its:

- Mean (μ)
- Standard deviation (σ)

Its probability density function is:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

The normal distribution is:

1. Symmetric about its mean.
2. Bell-shaped.
3. Determined by its mean and standard deviation.
4. Continuous.
5. Characterized by the fact that mean, median and mode coincide.

Mean=Median=Mode

Standard Normal Distribution

A normal distribution with:

μ=0 and σ=1 : called the standard normal distribution.

A value can be converted to the standard normal scale using the z-score:

$$z = \frac{x - \mu}{\sigma}$$

The z-score indicates how many standard deviations an observation lies from the mean.

Normal Distribution and Standard Deviation:

For a normally distributed dataset, a commonly used empirical rule is:

- Approximately 68% of observations lie within 1σ of the mean.
- Approximately 95% lie within 2σ.
- Approximately 99.7% lie within 3σ.

This is known as the 68–95–99.7 rule.

It provides a useful way to understand the spread of normally distributed data.

Visualizing the Distribution Important Python Functions are:

Operation	Python function
Mean	data.mean()
Median	data.median()
Variance	data.var()
Standard deviation	data.std()
Correlation matrix	data.corr()
Heatmap	sns.heatmap()
Histogram	sns.histplot()
KDE curve	sns.kdeplot()
Normal distribution	stats.norm

REFERENCES: Refer lab manual for the same

Joel Grus – Data Science from Scratch, Ch. 5 – Statistics; Ch. 6 – Probability
Jiawei Han et al. – Data Mining, Ch. 2 – Getting to Know Your Data

Practice Problem Statements:

1. Prepare the academic performance sheet for college which takes the performance of 30 students based on their study hours, attendance, and examination marks. Using Python, calculate the mean, median, variance, and standard deviation of marks, and determine the correlation between study hours and marks. Plot appropriate graphs, including a scatter plot and normal distribution curve, and interpret the results to understand student performance and data distribution.
2. A company wants to analyze the monthly salaries and years of experience of its employees. Using Python,

calculate the mean, median, variance, and standard deviation of salaries, determine the correlation between experience and salary, and plot a scatter plot and histogram with a normal distribution curve. Interpret the statistical results and identify whether experience has a strong relationship with salary.

3. A real-estate company wants to analyze house prices, carpet area, and number of bedrooms for 50 houses. Using Python, calculate the mean, median, variance, and standard deviation of house prices, study the correlation between carpet area and price, and visualize the data using scatter plots and a histogram with a normal distribution curve. Interpret the results and determine whether house prices are approximately normally distributed and whether larger houses tend to have higher prices. [Use housing dataset available on Kaggle: https://www.kaggle.com/datasets/yasserh/housing-prices-dataset]

Important Questions: -

1. What is the difference between population variance and sample variance?
2. Why is median preferred over mean for skewed data?
3. Explain Pearson correlation coefficient and its range of values.
4. What are the properties of a normal distribution curve?
5. What is the empirical rule (68-95-99.7 rule) in normal distribution?
6. How does a correlation heatmap help in feature selection for machine learning?
7. What is the difference between correlation and causation?

SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE - 411043

Department of Electronics & Computer Engineering

ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**

SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]

EXPT No:2

PROBLEM STATEMENT

Calculating Mean, Median, variance and plotting Correlation and Normal Distribution of a data using Python.

ALGORITHM

Steps of statistical analysis and plotting:

1. Import **numpy, pandas, matplotlib, seaborn** and `scipy.stats`.
2. Load or create the dataset (students: study hours and marks; employees: experience and salary; Housing.csv).
3. Compute descriptive statistics: mean = $\Sigma x / n$; median = middle value of the sorted data; sample variance $s^2 = \Sigma(x - \text{mean})^2 / (n - 1)$; standard deviation $s = \sqrt{s^2}$.
4. Compute the Pearson correlation coefficient $r = \text{cov}(x, y) / (s_x \cdot s_y)$ using `x.corr(y)`.
5. Draw a scatter plot of x against y together with a best-fit line.
6. Draw a histogram with `density=True` and overlay the normal curve $N(\mu, \sigma)$ using `stats.norm.pdf`.
7. Build the correlation matrix `df.corr()` and display it as a seaborn heatmap.
8. Check the 68-95-99.7 rule and compare mean with median to judge symmetry or skewness.
9. Interpret the sign and strength of r and the shape of the distribution.

ERRORS and TEST CASES, USE CASES

Common errors and their remedies

Error / symptom	Cause	Remedy
ValueError: could not convert string to float	`df.corr()` applied on text columns.	Use `df.corr(numeric_only=True)` or select numeric columns.
Result is nan	Missing values or a constant column (standard deviation = 0).	`dropna()` / `fillna()`; remove constant columns.
Variance differs from the NumPy value	pandas `var()` uses ddof = 1, `np.var()` uses ddof = 0.	Pass `ddof` explicitly.
Normal curve is flat / not visible	Histogram drawn as counts while the pdf is a density.	Use `density=True` in `hist()`.
ValueError: x and y must be the same size	Unequal lengths in the scatter plot.	Check `len()`; align the data after `dropna()`.
Wrong conclusion (logical error)	Correlation treated as causation.	Report association only; do not claim cause and effect.

Test cases

ID	Input / condition	Expected result
T1	Data [10, 20, 30, 40, 50]	Mean 30, median 30, sample variance 250, standard deviation 15.81
T2	Data [2, 4, 4, 4, 5, 5, 7, 9]	Mean 5; population variance 4 (np.var); sample variance 4.57 (pandas)
T3	x = [1, 2, 3, 4, 5], y = [2, 4, 6, 8, 10]	r = +1.0 (perfect positive correlation)
T4	x = [1, 2, 3, 4, 5], y = [10, 8, 6, 4, 2]	r = −1.0 (perfect negative correlation)
T5	Data [10, 12, 11, 13, 200]	Mean 49.2 is far above median 12: outlier / right skew
T6	A column in which every value is equal	Correlation with any other column = NaN
T7	Normally distributed data	About 68% of values lie within one standard deviation of the mean
T8	Housing.csv: area vs price	r = 0.536, a moderate positive correlation

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:2	
PROBLEM STATEMENT	Calculating Mean, Median, variance and plotting Correlation and Normal Distribution of a data using Python.

Use cases

- Academic performance analysis: relation between study hours, attendance and marks.
- Salary and experience analysis in human-resource planning.
- Real-estate price studies: effect of area and bedrooms on price.
- Quality control: checking whether a process stays within ±3σ limits.
- Feature selection in machine learning using the correlation matrix.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 **Department of Electronics & Computer Engineering** **ASSESSMENT YEAR: 2026-2027** | **CLASS: BE** **SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]**
EXPT No:2	
PROBLEM STATEMENT	Calculating Mean, Median, variance and plotting Correlation and Normal Distribution of a data using Python.

Libraries: numpy, pandas, matplotlib, seaborn, scipy. Dataset: Housing.csv (Problem 3). The data of 30 students (Problem 1) and 30 employees (Problem 2) is prepared inside the program.

PROGRAM AND OUTPUT

Import libraries and helper function

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
pd.set_option("display.width", 85)

def analyse(data, xcol, ycol, xlabel, ylabel, title):
    y = data[ycol]
    print("Statistics of", ylabel)
    print("Mean               :", round(y.mean(), 2))
    print("Median             :", round(y.median(), 2))
    print("Variance (sample)  :", round(y.var(), 2))
    print("Standard deviation :", round(y.std(), 2))
    r = data[xcol].corr(y)
    print("Correlation (%s vs %s): %.4f" % (xlabel, ylabel, r))

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    ax[0].scatter(data[xcol], y, color="steelblue", edgecolor="black")
    m, c = np.polyfit(data[xcol], y, 1)
    ax[0].plot(data[xcol], m * data[xcol] + c, color="red")
    ax[0].set_xlabel(xlabel); ax[0].set_ylabel(ylabel)
    ax[0].set_title("Scatter plot: %s vs %s" % (xlabel, ylabel))
    ax[1].hist(y, bins=8, density=True, color="lightgray", edgecolor="black")
    xs = np.linspace(y.min(), y.max(), 200)
    ax[1].plot(xs, stats.norm.pdf(xs, y.mean(), y.std()), color="red", linewidth=2)
    ax[1].set_xlabel(ylabel); ax[1].set_ylabel("Density")
    ax[1].set_title("Histogram with normal curve")
    fig.suptitle(title)
    plt.tight_layout()
    plt.show()
    return r

def interpret(r, y, xname):
    if abs(r) >= 0.7: s = "strong"
    elif abs(r) >= 0.4: s = "moderate"
    else: s = "weak"
    d = "positive" if r > 0 else "negative"
    print("Interpretation:")
    print(" - Correlation r = %.2f, a %s %s relationship with %s." % (r, s, d, xname))
    print(" - Skewness of data = %.2f (%s)." % (stats.skew(y),
          "approximately symmetric" if abs(stats.skew(y)) < 0.5 else "skewed"))
```

```
    print(" - Mean %.2f vs median %.2f." % (y.mean(), y.median()))
```

Problem 1: Academic performance of 30 students

Code:

```
np.random.seed(7)
hours = np.round(np.random.uniform(1, 9, 30), 1)
attendance = np.clip(np.round(55 + 4.5 * hours + np.random.normal(0, 6, 30)), 40, 100)
marks = np.clip(np.round(28 + 5.5 * hours + 0.25 * (attendance - 75) + np.random.normal(0,
6, 30)), 0, 100)
students = pd.DataFrame({"StudyHours": hours, "Attendance": attendance, "Marks": marks})
print(students.head(10))
print("\nShape:", students.shape)
print()
r1 = analyse(students, "StudyHours", "Marks", "Study hours", "Marks", "Academic performance
of 30 students")
print()
interpret(r1, students["Marks"], "study hours")
print("\nCorrelation matrix:")
print(students.corr().round(3))
sns.heatmap(students.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation heatmap - student data")
plt.show()
```

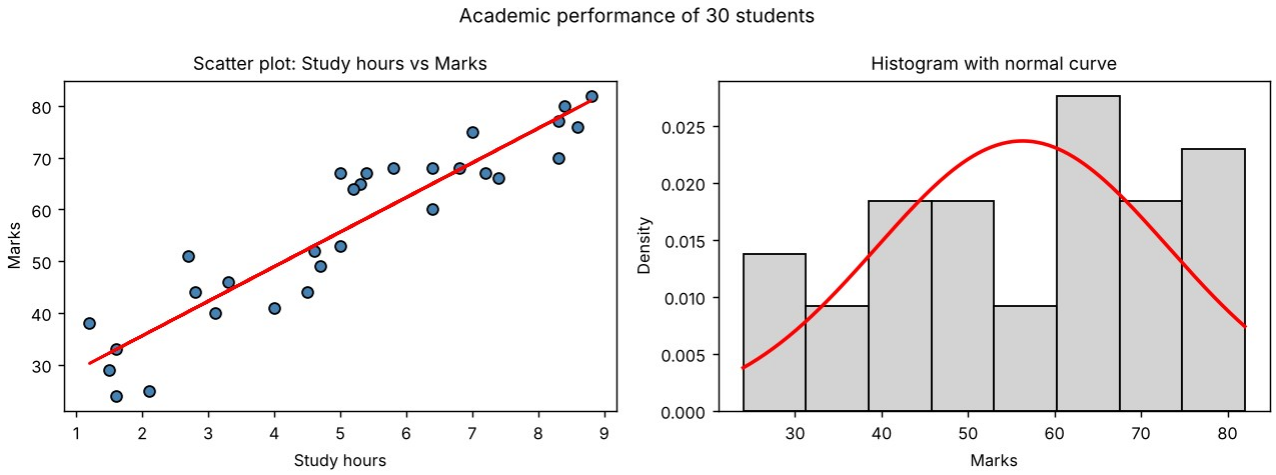
Output:

```
   StudyHours  Attendance  Marks
0         1.6        62.0   33.0
1         7.2        79.0   67.0
2         4.5        73.0   44.0
3         6.8        72.0   68.0
4         8.8       100.0   82.0
5         5.3        76.0   65.0
6         5.0        73.0   53.0
7         1.6        69.0   24.0
8         3.1        59.0   40.0
9         5.0        81.0   67.0

Shape: (30, 3)

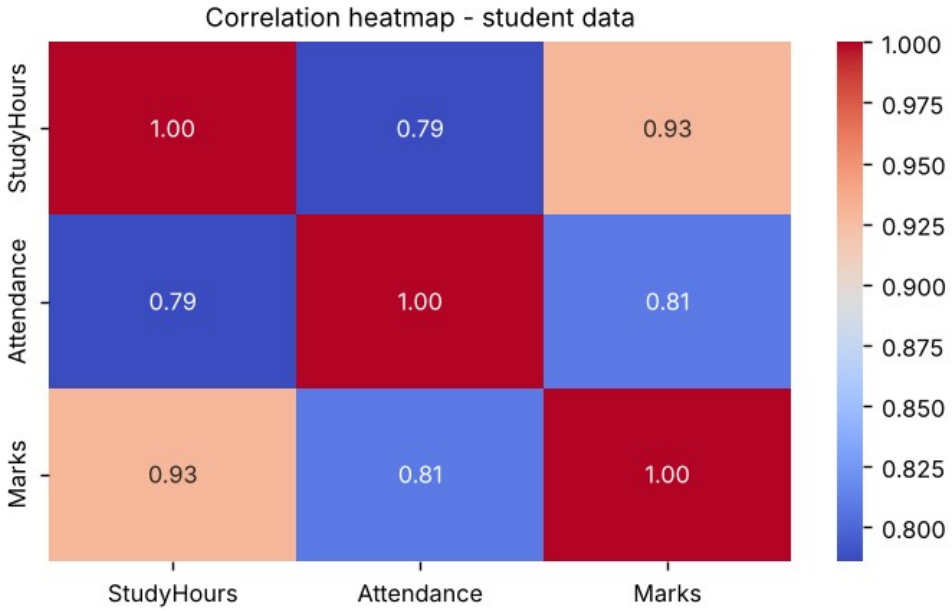
Statistics of Marks
Mean               : 56.3
Median             : 62.0
Variance (sample)  : 284.08
Standard deviation : 16.85
Correlation (Study hours vs Marks): 0.9294
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 **Department of Electronics & Computer Engineering** **ASSESSMENT YEAR: 2026-2027** | **CLASS: BE** **SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]**
EXPT No:2	
PROBLEM STATEMENT	Calculating Mean, Median, variance and plotting Correlation and Normal Distribution of a data using Python.




```
Interpretation:
 - Correlation r = 0.93, a strong positive relationship with study hours.
 - Skewness of data = -0.37 (approximately symmetric).
 - Mean 56.30 vs median 62.00.

Correlation matrix:
             StudyHours  Attendance  Marks
StudyHours        1.000       0.786  0.929
Attendance        0.786       1.000  0.809
Marks             0.929       0.809  1.000
```



Problem 2: Salary and experience

Code:

```
np.random.seed(11)
exp = np.round(np.random.uniform(0.5, 20, 30), 1)
salary = np.round(22000 + 3300 * exp + np.random.normal(0, 9000, 30), -2)
emp = pd.DataFrame({"Experience": exp, "Salary": salary})
print(emp.head(10))
print("\nShape:", emp.shape)
print()
r2 = analyse(emp, "Experience", "Salary", "Experience (years)", "Monthly salary (Rs.)",
"Salary and experience of employees")
print()
interpret(r2, emp["Salary"], "experience")
print("Experience has a strong relationship with salary:", abs(r2) >= 0.7)
```

Output:

```
   Experience   Salary
0         4.0  45100.0
1         0.9  22200.0
2         9.5  59900.0
3        14.6  84100.0
4         8.7  56400.0
5        10.0  55700.0
6         0.7  30900.0
```

```
7        10.0  49200.0
8        18.9  82800.0
9        17.1  73300.0

Shape: (30, 2)

Statistics of Monthly salary (Rs.)
Mean               : 51063.33
Median             : 47350.0
Variance (sample)  : 405116885.06
Standard deviation : 20127.52
Correlation (Experience (years) vs Monthly salary (Rs.)): 0.9276
```




```
Interpretation:
 - Correlation r = 0.93, a strong positive relationship with experience.
 - Skewness of data = -0.09 (approximately symmetric).
 - Mean 51063.33 vs median 47350.00.
Experience has a strong relationship with salary: True
```

Problem 3: Housing prices (Housing.csv)

Code:

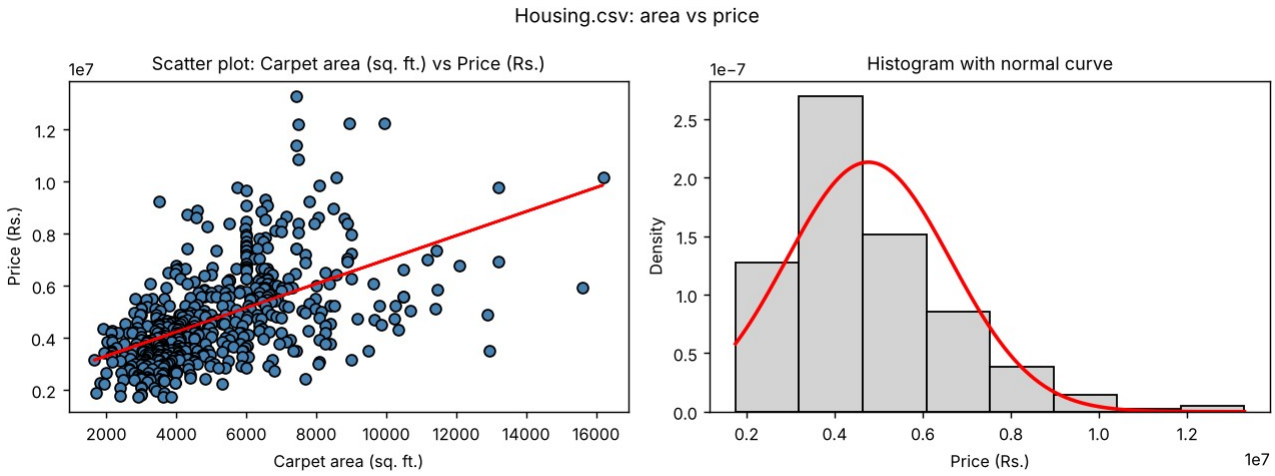
```
house = pd.read_csv("Housing.csv")
print("Shape:", house.shape)
print(house[["price", "area", "bedrooms"]].head())
print()
r3 = analyse(house, "area", "price", "Carpet area (sq. ft.)", "Price (Rs.)", "Housing.csv:
area vs price")
print()
interpret(r3, house["price"], "carpet area")
print(" - Larger houses tend to have higher prices:", r3 > 0.4)
```

```
print(" - Prices approximately normal:", abs(stats.skew(house["price"])) < 0.5)
```

Output:

```
Shape: (545, 13)
      price  area  bedrooms
0  13300000  7420         4
1  12250000  8960         4
2  12250000  9960         3
3  12215000  7500         4
4  11410000  7420         4

Statistics of Price (Rs.)
Mean               : 4766729.25
Median             : 4340000.0
Variance (sample)  : 3498544355820.57
Standard deviation : 1870439.62
Correlation (Carpet area (sq. ft.) vs Price (Rs.)): 0.5360
```




```
Interpretation:
 - Correlation r = 0.54, a moderate positive relationship with carpet area.
 - Skewness of data = 1.21 (skewed).
 - Mean 4766729.25 vs median 4340000.00.
 - Larger houses tend to have higher prices: True
 - Prices approximately normal: False
```

Correlation heatmap of housing data

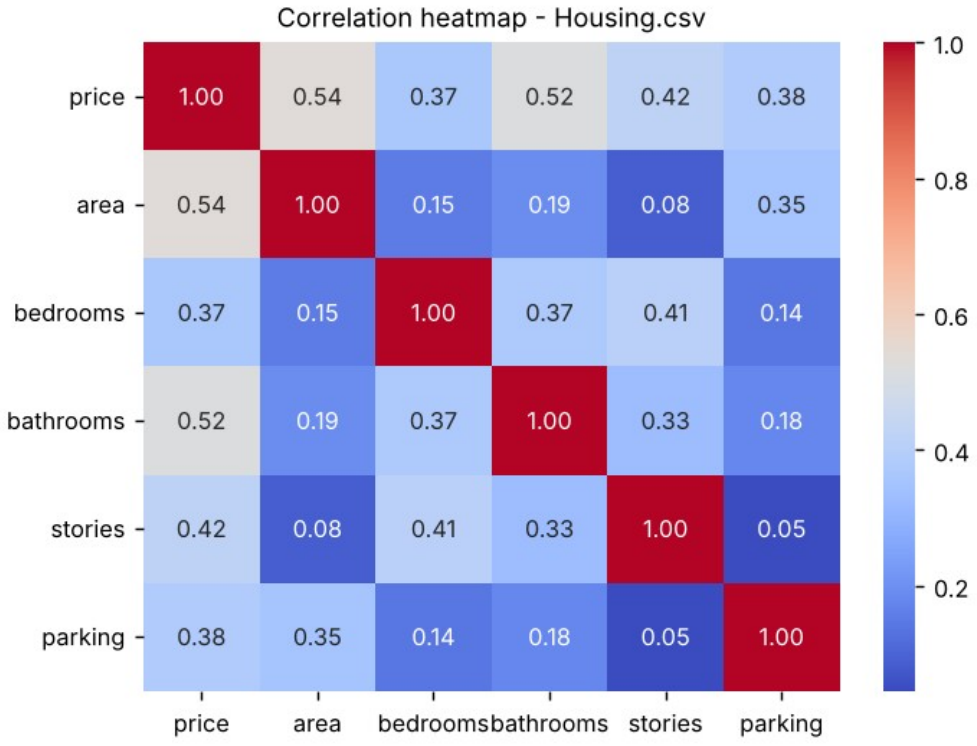
Code:

```
num = house[["price", "area", "bedrooms", "bathrooms", "stories", "parking"]]
print(num.corr().round(3))
plt.figure(figsize=(6, 4.5))
```

```
sns.heatmap(num.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation heatmap - Housing.csv")
plt.show()
```

Output:

```
            price   area  bedrooms  bathrooms  stories  parking
price       1.000  0.536     0.366      0.518    0.421    0.384
area        0.536  1.000     0.152      0.194    0.084    0.353
bedrooms    0.366  0.152     1.000      0.374    0.409    0.139
bathrooms   0.518  0.194     0.374      1.000    0.326    0.177
stories     0.421  0.084     0.409      0.326    1.000    0.046
parking     0.384  0.353     0.139      0.177    0.046    1.000
```



Normal distribution and empirical rule

Code:

```
p = house["price"]
mu, sd = p.mean(), p.std()
for k in (1, 2, 3):
    pct = ((p > mu - k * sd) & (p < mu + k * sd)).mean() * 100
    print("Within %d std. deviation(s): %.1f %%" % (k, pct))
```

```
print("(For an exact normal curve these would be 68%, 95% and 99.7%)")
```

Output:

```
Within 1 std. deviation(s): 74.1 %
Within 2 std. deviation(s): 95.2 %
Within 3 std. deviation(s): 98.9 %
(For an exact normal curve these would be 68%, 95% and 99.7%)
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:2		
PROBLEM STATEMENT	Calculating Mean, Median, variance and plotting Correlation and Normal Distribution of a data using Python.	

IMPORTANT QUESTIONS AND ANSWERS

Q1. What is the difference between population variance and sample variance?

Variance measures how far the observations are spread from the mean.

- **Population variance** is used when the data contains every member of the population: **σ^2 = sum of $(x_i - \mu)^2 / N$**
- **Sample variance** is used when the data is only a sample of a bigger population: **s^2 = sum of $(x_i - \bar{x})^2 / (n - 1)$**

In the sample formula we divide by (n - 1) instead of n. This is called **Bessel's correction**. A sample is closer to its own mean than to the true population mean, so dividing by n would under-estimate the variance. Dividing by (n - 1) gives an unbiased estimate.

In Python, pandas var() uses n - 1 by default (ddof=1), while numpy var() uses n by default (ddof=0).

```
import numpy as np
x = [10, 20, 30, 40, 50]
np.var(x)            # 200.0  (population)
np.var(x, ddof=1)    # 250.0  (sample)
```

Q2. Why is median preferred over mean for skewed data?

The mean uses every value, so a few extreme values (outliers) pull it towards the tail. The median is just the middle value and is not affected by how large or small the extreme values are, so it describes the typical value better.

Example: salaries (in thousands) 20, 25, 30, 35, 500. Mean = 610 / 5 = 122, but median = 30. The mean is misleading because of one very high salary, whereas the median shows what a typical employee earns. This is why median is used for income, house prices, etc. In right-skewed data mean > median, and in left-skewed data mean < median.

Q3. Explain Pearson correlation coefficient and its range of values.

The Pearson correlation coefficient (r) measures the strength and direction of the **linear** relationship between two numerical variables.

r = sum of $(x_i - \bar{x})(y_i - \bar{y}) /$ sqrt(sum of $(x_i - \bar{x})^2$ x sum of $(y_i - \bar{y})^2$)

Its value always lies between **-1 and +1**.

Value of r	Meaning
+1	Perfect positive linear relationship
0.7 to 0.99	Strong positive correlation (e.g. study hours and marks)
0	No linear relationship
-0.7 to -0.99	Strong negative correlation (e.g. price and quantity demanded)
-1	Perfect negative linear relationship

```
df['hours'].corr(df['marks'])
df.corr()        # correlation matrix of all numeric columns
```

Q4. What are the properties of a normal distribution curve?

- It is bell-shaped and perfectly **symmetric** about the mean.
- **Mean = Median = Mode**, all at the centre of the curve.
- It is completely described by two parameters: mean (μ) and standard deviation (σ).
- The curve is continuous and extends to infinity on both sides but never touches the x-axis (asymptotic).
- The total area under the curve is 1.
- A larger σ gives a wider, flatter curve; a smaller σ gives a narrower, taller curve.
- It has points of inflection at μ - σ and μ + σ.
- Skewness is 0 and the empirical rule (68-95-99.7) applies.

```
from scipy.stats import norm
x = np.linspace(mu - 4*sd, mu + 4*sd, 200)
plt.plot(x, norm.pdf(x, mu, sd))
```

Q5. What is the empirical rule (68-95-99.7 rule) in normal distribution?

For normally distributed data:

- About **68%** of the observations lie within 1 standard deviation of the mean (μ ± 1 σ).
- About **95%** lie within 2 standard deviations (μ ± 2 σ).
- About **99.7%** lie within 3 standard deviations (μ ± 3 σ).

Example: if marks have mean 70 and sd 5, then about 68% of students score between 65 and 75, 95% between 60 and 80, and 99.7% between 55 and 85. A value outside $\mu \pm 3\sigma$ is usually treated as an outlier. The z-score $z = (x - \mu) / \sigma$ tells how many standard deviations a value is from the mean.

Q6. How does a correlation heatmap help in feature selection for machine learning?

A correlation heatmap shows the correlation matrix as coloured cells, so relationships between all variables can be seen at a glance.

- **Relevant features:** features with a high correlation (positive or negative) with the target variable are good predictors and are selected.
- **Redundant features:** if two input features are highly correlated with each other (for example above 0.8 or 0.9), they carry the same information (multicollinearity), so one of them can be dropped.
- **Irrelevant features:** features with correlation close to 0 with the target can be removed.

```
import seaborn as sns
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```

Limitation: Pearson correlation only captures linear relationships.

Q7. What is the difference between correlation and causation?

Correlation means two variables move together. **Causation** means a change in one variable directly produces a change in the other. Correlation does not prove causation, because the relationship may be due to a third (confounding) variable or just a coincidence.

Example: ice-cream sales and drowning cases are positively correlated, but ice cream does not cause drowning. Both increase in summer because of hot weather (the hidden third variable). On the other hand, study hours and marks are correlated and there is a plausible cause. Causation can only be established through controlled experiments, not from a correlation coefficient alone.

CONCLUSION:
Signature:

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:3	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	
PROBLEM STATEMENT	Importing and exporting CSV files using Pandas in Python and analyzing data (like shape, display of data in CSV file, checking missing value, and correlation) in CSV files. Download heart dataset from following link: https://www.kaggle.com/zhaoyingzhu/heartcsv Perform following operation on given dataset. a)Find Shape of Data, b)Find Missing Values, c)Find data type of each column, d)Finding out Zero's, e)Find Mean age of patients f) Now extract only Age, Sex, ChestPain, RestBP, Chol. Randomly divide dataset in training (75%) and testing (25%). Through the diagnosis test I predicted 100 report as COVID positive, but only 45 of those were actually positive. Total 50 people in my sample were actually COVID positive. I have total 500 samples. Create confusion matrix based on above data and find I. Accuracy, II. Precision, III. Recall, IV. F-1 score [random_state=0,test_size=0.25]	
PREREQUISITES	• Understanding of CSV file format and structure • Basic Pandas DataFrame knowledge • Concept of missing values in datasets	
OBJECTIVES:	1. To import and export datasets using CSV files in Pandas. 2. To analyze dataset structure (shape, head, info) and identify missing values. 3. To compute correlation between numerical features in the dataset.	
Tools Used	• Python 3.x • Jupyter Notebook • Libraries: pandas, numpy	
THEORY:		
	• CSV File: CSV (Comma-Separated Values) is a commonly used file format for storing tabular data.	

For example, a file named student.csv may contain:

Name	Age	Marks	Attendance
Amit	20	85	90
Priya	21	78	85
Rahul	20	92	95
Sneha	21	70	80

Each row represents a record, while each column represents an attribute or variable. CSV files are widely used in Data Science and Machine Learning because they are simple to create, store, share, and process.

- Pandas: Pandas is a Python library used for:
  - Loading datasets
  - Cleaning data
  - Manipulating data
  - Analyzing data
  - Handling missing values
  - Performing statistical operations

The two important Pandas data structures are:

- Series – one-dimensional data
- DataFrame – two-dimensional tabular data

A CSV file is generally imported into a DataFrame.

Operations on CSV file are:

1. **Importing a CSV File**

The read_csv() function is used to load a CSV file into a Pandas DataFrame.

```
import pandas as pd
data = pd.read_csv("student.csv")
print(data)
```

2. Displaying Data from a CSV File

To display the first three rows: print(data.head(3))

Display the last five records: print(data.tail())

Display column names: print(data.columns)

Display basic information about the dataset: print(data.info())

3. Finding the Shape of the Dataset

Finding the Shape of the Dataset: print(data.shape)

	4. Checking Missing Values Real-world datasets often contain missing or empty values. Pandas represents missing numerical values as NaN (Not a Number): print(data.isnull())- This produces True for missing values and False for available values. 5. Statistical Summary of Data print(data.describe()):The describe() function provides statistical information about numerical columns. It generally displays Count, Mean, Standard deviation, Minimum, 25th percentile, Median (50%), 75th percentile and Maximum. This is useful for obtaining an initial understanding of the dataset. 6. Correlation Analysis Correlation measures the strength and direction of the relationship between two numerical variables. For example, we may want to investigate: Is there a relationship between students' attendance and their marks? Pandas provides the corr() function. e.g. print(data.corr(numeric_only=True))
REFERENCES: Refer lab manual for the same	
	1. Jiawei Han, MichelineKamber, Jian Pei. Data Mining: Concepts and Techniques, 3rd Edition **2.Joel Grus, Data Science from Scratch, O'Reilly Media Inc., ISBN: 9781491901427**

Problem Statements to be Performed:

1. Download temperature data from below link. https://www.kaggle.com/venky73/temperatures-of-india?select=temperatures.csv. This data consists of temperatures of INDIA averaging the temperatures of all places month wise. Temperatures values are recorded in CELSIUS

   I. Apply Linear Regression using a suitable library function and predict the Month-wise temperature.

   II. Assess the performance of regression models using MSE, MAE and R-Square metrics

   III. Visualize a simple regression model.

Important Questions: -

1. What is the difference between read_csv() and read_excel() in Pandas?
2. How do you handle missing values in a dataset? Name at least three methods.
3. What is the purpose of the index=False parameter in to_csv()?
4. Explain the difference between NaN and None in Pandas.
5. How do you check and convert data types of columns in a DataFrame?

6. What does df.describe() return, and how is it useful in initial data analysis?

SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE - 411043

Department of Electronics & Computer Engineering

ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**

SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]

EXPT No:3

PROBLEM STATEMENT

Importing and exporting CSV files using Pandas in Python and analyzing data (like shape, display of data in CSV file, checking missing value, and correlation) in CSV files.

ALGORITHM

Steps of CSV analysis, confusion matrix and regression:

1. Import **pandas, numpy** and the required **scikit-learn** modules.
2. Import the CSV file with `pd.read_csv('Heart.csv')`; display `head()` and `tail()`.
3. Find the shape (`df.shape`), data types (`df.dtypes`), missing values (`df.isnull().sum()`), zeros (`(df == 0).sum()`) and the mean age (`df['Age'].mean()`).
4. Compute the correlation matrix of the numerical columns with `df.corr(numeric_only=True)`.
5. Extract the columns Age, Sex, ChestPain, RestBP and Chol into a new DataFrame.
6. Divide it randomly into training and testing data: `train_test_split(subset, test_size=0.25, random_state=0)`.
7. Export a DataFrame to CSV with `df.to_csv('output.csv', index=False)`.
8. Confusion matrix: TP = 45, FP = 100 − 45 = 55, FN = 50 − 45 = 5, TN = 500 − (45 + 55 + 5) = 395.
9. Metrics: Accuracy = (TP + TN)/N; Precision = TP/(TP + FP); Recall = TP/(TP + FN); F1 = 2PR/(P + R).
10. Linear regression (Age → MaxHR): fit on the training data, predict the test data, compute MSE, MAE and R^2 and plot the line.

ERRORS and TEST CASES, USE CASES

Common errors and their remedies

Error / symptom	Cause	Remedy
FileNotFoundError	Wrong path or the CSV is not in the working folder.	Give the full path or keep the file beside the notebook; check `os.getcwd()`.
UnicodeDecodeError / ParserError	Wrong file encoding or delimiter.	`read_csv(..., encoding='latin1', sep=';')`.
Extra column 'Unnamed: 0'	The CSV was saved together with its index.	Use `index_col=0`, or save with `index=False`.
ValueError: Input contains NaN	Missing values passed to the model.	Impute with `fillna()` or drop with `dropna()`.
ValueError: could not convert string to float: 'typical'	Categorical column (ChestPain) given to a model.	Encode it with `pd.get_dummies()` or `map()`.
UndefinedMetricWarning	Precision computed when TP + FP = 0.	`precision_score(..., zero_division=0)`.

SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE - 411043

Department of Electronics & Computer Engineering

ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**

SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]

EXPT No:3

PROBLEM STATEMENT

Importing and exporting CSV files using Pandas in Python and analyzing data (like shape, display of data in CSV file, checking missing value, and correlation) in CSV files.

Test cases

ID	Input / condition	Expected result
T1	Heart.csv imported with `read_csv`	303 records
T2	`df.isnull().sum().sum()`	6 missing values (as in the practical output)
T3	`train_test_split` of 303 rows, test_size = 0.25, random_state = 0	227 training and 76 testing rows; the same split on every run
T4	100 predicted positive, 45 correct, 50 actually positive, 500 samples	TP = 45, FP = 55, FN = 5, TN = 395
T5	Metrics for T4	Accuracy 0.88, Precision 0.45, Recall 0.90, F1-score 0.60
T6	`to_csv(index=False)` followed by `read_csv`	Same shape; no 'Unnamed: 0' column
T7	Linear regression Age vs MaxHR	Negative coefficient (−0.98); R^2 between 0 and 1 (0.108)
T8	(Negative test) `read_csv('heart.csvv')`	FileNotFoundError is raised

Use cases

- Healthcare analytics: studying patient records and risk factors.
- Evaluating a diagnostic (e.g. COVID) test using precision, recall and F1-score.
- Exchanging data between spreadsheets, databases and Python programs.
- Preparing training and testing sets for machine-learning models.
- Predicting one health indicator from another using simple regression.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 **Department of Electronics & Computer Engineering** **ASSESSMENT YEAR: 2026-2027** | **CLASS: BE** **SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]**
EXPT No:3	
PROBLEM STATEMENT	Importing and exporting CSV files using Pandas in Python and analyzing data (shape, display of data in CSV file, checking missing values and correlation) in CSV files.

Libraries: pandas, numpy, scikit-learn, matplotlib. Dataset: Heart.csv (303 records). The confusion matrix problem is solved with the given values, and linear regression (MSE, MAE, R-square and visualisation) is demonstrated on Heart.csv (Age vs MaxHR).

PROGRAM AND OUTPUT

Import and display CSV data

Code:

```
import pandas as pd
import numpy as np
pd.set_option("display.width", 85)
pd.set_option("display.max_columns", 20)

df = pd.read_csv("Heart.csv")
print("First 5 records:")
print(df.head())
print("\nLast 3 records:")
print(df.tail(3))
print("\nColumn names:")
print(list(df.columns))
```

Output:

```
First 5 records:
   Unnamed: 0  Age  Sex     ChestPain  RestBP  Chol  Fbs  RestECG  MaxHR  ExAng  \
0           1   63    1       typical     145   233    1        2    150      0
1           2   67    1  asymptomatic     160   286    0        2    108      1
2           3   67    1  asymptomatic     120   229    0        2    129      1
3           4   37    1    nonanginal     130   250    0        0    187      0
4           5   41    0    nontypical     130   204    0        2    172      0

   Oldpeak  Slope   Ca        Thal  AHD
0      2.3      3  0.0       fixed   No
1      1.5      2  3.0      normal  Yes
2      2.6      2  2.0  reversable  Yes
3      3.5      3  0.0      normal   No
4      1.4      1  0.0      normal   No

Last 3 records:
     Unnamed: 0  Age  Sex     ChestPain  RestBP  Chol  Fbs  RestECG  MaxHR  ExAng  \
300         301   57    1  asymptomatic     130   131    0        0    115      1
301         302   57    0    nontypical     130   236    0        2    174      0
302         303   38    1    nonanginal     138   175    0        0    173      0

     Oldpeak  Slope   Ca        Thal  AHD
300      1.2      2  1.0  reversable  Yes
301      0.0      2  1.0      normal  Yes
302      0.0      1  NaN      normal   No
```

```
Column names:
['Unnamed: 0', 'Age', 'Sex', 'ChestPain', 'RestBP', 'Chol', 'Fbs', 'RestECG', 'MaxHR',
'ExAng', 'Oldpeak', 'Slope', 'Ca', 'Thal', 'AHD']
```

a) Shape of data

Code:

```
print("Shape of data (rows, columns):", df.shape)
print("Number of rows   :", df.shape[0])
print("Number of columns:", df.shape[1])
```

Output:

```
Shape of data (rows, columns): (303, 15)
Number of rows   : 303
Number of columns: 15
```

b) Missing values

Code:

```
print(df.isnull().sum())
print("\nTotal missing values:", df.isnull().sum().sum())
```

Output:

```
Unnamed: 0    0
Age           0
Sex           0
ChestPain     0
RestBP        0
Chol          0
Fbs           0
RestECG       0
MaxHR         0
ExAng         0
Oldpeak       0
Slope         0
Ca            4
Thal          2
AHD           0
dtype: int64

Total missing values: 6
```

c) Data types

Code:

```
print(df.dtypes)
print()
df.info()
```

Output:

```
Unnamed: 0      int64
```

```
Age             int64
Sex             int64
ChestPain         str
RestBP          int64
Chol            int64
Fbs             int64
RestECG         int64
MaxHR           int64
ExAng           int64
Oldpeak       float64
Slope           int64
Ca            float64
Thal              str
AHD               str
dtype: object

<class 'pandas.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 15 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Unnamed: 0  303 non-null    int64
 1   Age         303 non-null    int64
 2   Sex         303 non-null    int64
 3   ChestPain   303 non-null    str
 4   RestBP      303 non-null    int64
 5   Chol        303 non-null    int64
 6   Fbs         303 non-null    int64
 7   RestECG     303 non-null    int64
 8   MaxHR       303 non-null    int64
 9   ExAng       303 non-null    int64
 10  Oldpeak     303 non-null    float64
 11  Slope       303 non-null    int64
 12  Ca          299 non-null    float64
 13  Thal        301 non-null    str
 14  AHD         303 non-null    str
dtypes: float64(2), int64(10), str(3)
memory usage: 35.6 KB
```

d) Zeros

Code:

```
print((df == 0).sum())
```

Output:

```
Unnamed: 0      0
Age             0
Sex            97
ChestPain       0
RestBP          0
Chol            0
```

```
Fbs           258
RestECG       151
MaxHR           0
ExAng         204
Oldpeak        99
Slope           0
Ca            176
Thal            0
AHD             0
dtype: int64
```

e) Mean age

Code:

```
print("Mean age of patients:", round(df["Age"].mean(), 2))
print("Minimum age:", df["Age"].min(), "  Maximum age:", df["Age"].max())
```

Output:

```
Mean age of patients: 54.44
Minimum age: 29   Maximum age: 77
```

f) Extract columns and split data 75/25

Code:

```
from sklearn.model_selection import train_test_split
subset = df[["Age", "Sex", "ChestPain", "RestBP", "Chol"]]
print(subset.head())
print("Shape of subset:", subset.shape)

train, test = train_test_split(subset, test_size=0.25, random_state=0)
print("\nTraining set shape:", train.shape)
print("Testing set shape :", test.shape)
print("\nFirst 5 rows of training data:")
print(train.head())
```

Output:

```
   Age  Sex     ChestPain  RestBP  Chol
0   63    1       typical     145   233
1   67    1  asymptomatic     160   286
2   67    1  asymptomatic     120   229
3   37    1    nonanginal     130   250
4   41    0    nontypical     130   204
Shape of subset: (303, 5)

Training set shape: (227, 5)
Testing set shape : (76, 5)

First 5 rows of training data:
     Age  Sex     ChestPain  RestBP  Chol
173   62    0  asymptomatic     140   394
261   58    0    nontypical     136   319
```

```
37    57    1  asymptomatic     150   276
101   34    1       typical     118   182
166   52    1    nonanginal     138   223
```

Export CSV, describe and correlation

Code:

```
train.to_csv("heart_train.csv", index=False)
test.to_csv("heart_test.csv", index=False)
print("Files heart_train.csv and heart_test.csv created")
print(pd.read_csv("heart_train.csv").shape)

print("\nStatistical summary:")
print(df[["Age", "RestBP", "Chol", "MaxHR", "Oldpeak"]].describe().round(2))

print("\nCorrelation matrix:")
print(df[["Age", "RestBP", "Chol", "MaxHR", "Oldpeak"]].corr().round(3))
```

Output:

```
Files heart_train.csv and heart_test.csv created
(227, 5)

Statistical summary:
           Age  RestBP    Chol   MaxHR  Oldpeak
count  303.00  303.00  303.00  303.00   303.00
mean    54.44  131.69  246.69  149.61     1.04
std      9.04   17.60   51.78   22.88     1.16
min     29.00   94.00  126.00   71.00     0.00
25%     48.00  120.00  211.00  133.50     0.00
50%     56.00  130.00  241.00  153.00     0.80
75%     61.00  140.00  275.00  166.00     1.60
max     77.00  200.00  564.00  202.00     6.20

Correlation matrix:
            Age  RestBP   Chol  MaxHR  Oldpeak
Age       1.000   0.285  0.209 -0.394    0.204
RestBP    0.285   1.000  0.130 -0.045    0.189
Chol      0.209   0.130  1.000 -0.003    0.047
MaxHR    -0.394  -0.045 -0.003  1.000   -0.343
Oldpeak   0.204   0.189  0.047 -0.343    1.000
```

Confusion matrix: COVID test

Code:

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score,
f1_score
TP = 45
FP = 100 - 45
FN = 50 - 45
TN = 500 - TP - FP - FN
```

```
y_true = [1] * TP + [0] * FP + [1] * FN + [0] * TN
y_pred = [1] * TP + [1] * FP + [0] * FN + [0] * TN

print("TP =", TP, " FP =", FP, " FN =", FN, " TN =", TN)
cm = confusion_matrix(y_true, y_pred, labels=[1, 0])
print("\nConfusion matrix (rows = actual, columns = predicted)")
print(pd.DataFrame(cm, index=["Actual +ve", "Actual -ve"], columns=["Pred +ve", "Pred -
ve"]))
print("\nAccuracy  :", accuracy_score(y_true, y_pred))
print("Precision :", precision_score(y_true, y_pred))
print("Recall    :", recall_score(y_true, y_pred))
print("F1-score  :", round(f1_score(y_true, y_pred), 4))
```

Output:

```
TP = 45  FP = 55  FN = 5  TN = 395

Confusion matrix (rows = actual, columns = predicted)
            Pred +ve  Pred -ve
Actual +ve        45         5
Actual -ve        55       395

Accuracy  : 0.88
Precision : 0.45
Recall    : 0.9
F1-score  : 0.6
```

Linear regression with MSE, MAE, R2

Code:

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

X = df[["Age"]]
y = df["MaxHR"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

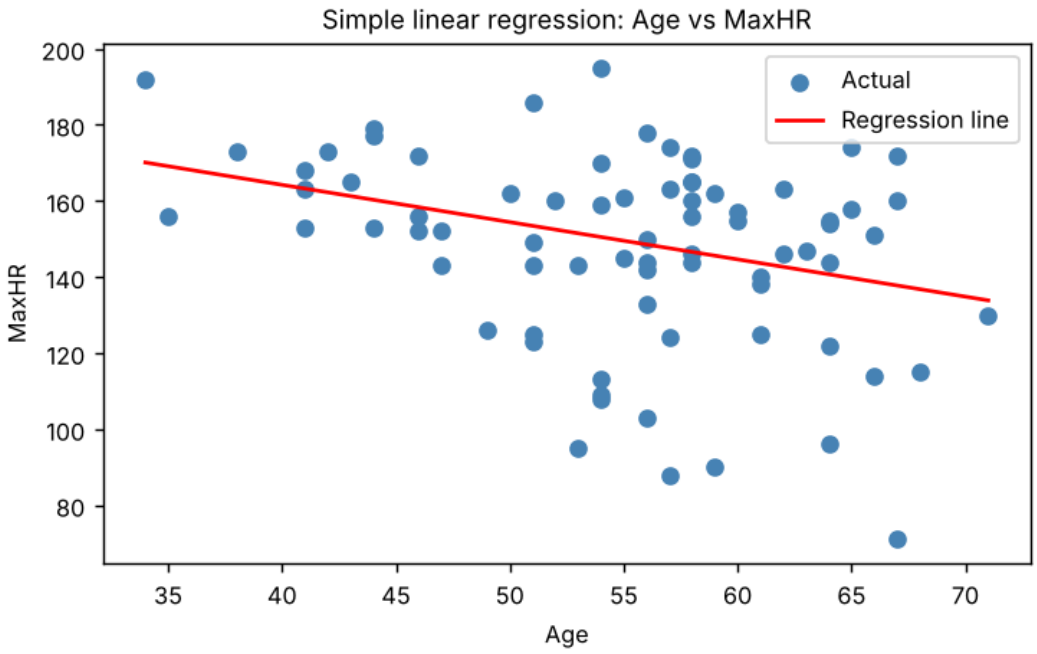
print("Intercept   :", round(model.intercept_, 3))
print("Coefficient :", round(model.coef_[0], 3))
print("MSE :", round(mean_squared_error(y_test, y_pred), 3))
print("MAE :", round(mean_absolute_error(y_test, y_pred), 3))
print("R2  :", round(r2_score(y_test, y_pred), 3))

plt.scatter(X_test, y_test, color="steelblue", label="Actual")
plt.plot(X_test.sort_values("Age"), model.predict(X_test.sort_values("Age")), color="red",
label="Regression line")
plt.xlabel("Age"); plt.ylabel("MaxHR"); plt.title("Simple linear regression: Age vs MaxHR")
```

```
plt.legend()
plt.show()
```

Output:

```
Intercept   : 203.427
Coefficient : -0.98
MSE : 570.683
MAE : 18.516
R2  : 0.108
```



	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 **Department of Electronics & Computer Engineering** **ASSESSMENT YEAR: 2026-2027** | **CLASS: BE** **SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]**
EXPT No:3	
PROBLEM STATEMENT	Importing and exporting CSV files using Pandas in Python and analyzing data (shape, display of data in CSV file, checking missing values and correlation) in CSV files.

IMPORTANT QUESTIONS AND ANSWERS

Q1. What is the difference between read_csv() and read_excel() in Pandas?

read_csv()	**read_excel()**
Reads comma-separated text files (.csv, or .txt with sep=)	Reads Excel workbooks (.xls, .xlsx)
Plain text, so it is fast and light	Binary/zipped format, so it is slower
Important parameters: sep, header, names, index_col, usecols, nrows	Same, plus sheet_name to choose the sheet
No extra library needed	Needs an engine such as openpyxl (xlrd for .xls)
Cannot hold formatting, formulas or multiple sheets	Can read any sheet, or all sheets (sheet_name=None)

```
df = pd.read_csv('heart.csv')
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
```

Q2. How do you handle missing values in a dataset? Name at least three methods.

First detect them with df.isnull().sum(). Then they can be handled in the following ways:

1. **Deletion:** df.dropna() removes rows with missing values (dropna(axis=1) removes columns). It is used when very few values are missing.
2. **Imputation with a statistic:** df['Age'].fillna(df['Age'].mean()) - use mean for normal data, median for skewed data, and mode for categorical data.
3. **Forward / backward fill:** df.ffill() or df.bfill() copies the previous or next value (useful in time series).
4. **Interpolation:** df.interpolate() estimates the missing value from neighbouring values.
5. **Model-based imputation:** SimpleImputer or KNNImputer from scikit-learn.

```
df.isnull().sum()
df['Ca'] = df['Ca'].fillna(df['Ca'].median())
df = df.dropna()
```

Q3. What is the purpose of the index=False parameter in to_csv()?

By default to_csv() writes the DataFrame index (0, 1, 2, ...) as an extra first column in the file. Using **index=False** stops this, so the exported file contains only the real data columns. If it is not used, reading the file back creates an extra column named 'Unnamed: 0'.

```
df.to_csv('output.csv', index=False)
```

Q4. Explain the difference between NaN and None in Pandas.

NaN	None
Stands for Not a Number; it is a special floating-point value (np.nan)	Python's null object of type NoneType
Used for missing values in numeric columns	Used for missing values in object (text) columns
Arithmetic with it gives NaN (5 + NaN = NaN)	Arithmetic raises a TypeError
NaN == NaN is False; check with isnull()	None == None is True; check with 'is None' or isnull()

Pandas treats both as missing: isnull() returns True for both, and in a numeric column None is automatically converted to NaN. Newer versions of Pandas also provide pd.NA as a common missing-value marker.

Q5. How do you check and convert data types of columns in a DataFrame?

```
df.dtypes                                    # data type of each column
df.info()                                    # types plus non-null counts

df['Age'] = df['Age'].astype(int)            # convert type
df['Price'] = pd.to_numeric(df['Price'], errors='coerce')   # bad values -> NaN
df['Date'] = pd.to_datetime(df['Date'])      # string to datetime
df['Sex'] = df['Sex'].astype('category')     # to category
```

Q6. What does df.describe() return, and how is it useful in initial data analysis?

describe() returns a statistical summary of the numerical columns: **count, mean, std, min, 25%, 50% (median), 75% and max**. With include='all' it also summarises text columns (unique, top, freq).

- count shows whether any column has missing values (count less than the number of rows).
- mean compared with 50% shows skewness.
- min and max show the range and help spot outliers or wrong values (for example negative age).
- std shows how widely the values are spread.

```
df.describe()
df.describe(include='all')
```

Numerical from the problem statement (confusion matrix)

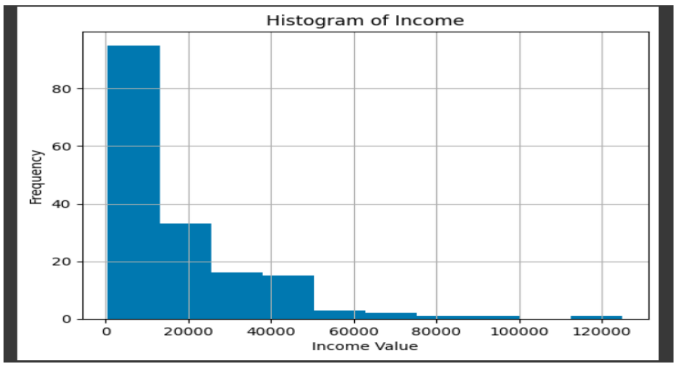
Total samples = 500. Predicted positive = 100, of which True Positive (TP) = 45, so False Positive (FP) = 100 - 45 = 55. Actual positive = 50, so False Negative (FN) = 50 - 45 = 5. True Negative (TN) = 500 - 45 - 55 - 5 = 395.

	Predicted Positive	**Predicted Negative**
Actual Positive	TP = 45	FN = 5
Actual Negative	FP = 55	TN = 395

- Accuracy = (TP + TN) / Total = (45 + 395) / 500 = **0.88**
- Precision = TP / (TP + FP) = 45 / 100 = **0.45**
- Recall = TP / (TP + FN) = 45 / 50 = **0.90**
- F1-score = 2 x P x R / (P + R) = 2 x 0.45 x 0.90 / 1.35 = **0.60**

CONCLUSION:
Signature and Date:

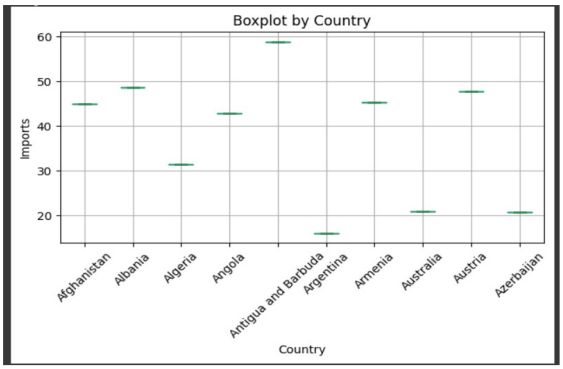
	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No: 4	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	
PROBLEM STATEMENT	Importing dataset (CSV file) and Python program to demonstrate the various plots using Matplotlib library on dataset.	
PREREQUISITES	Basic Pandas knowledge for handling datasets Understanding of chart types (line, bar, scatter, histogram, pie) and when to use each Basic concept of x-axis/y-axis and categorical vs numerical data	
OBJECTIVES:	To import a dataset and visualize numerical and categorical variables. To implement different plot types — line, bar, scatter, histogram, and pie chart. To interpret patterns and distributions in the dataset using visualizations.	
Tools Used	Python 3.x Jupyter Notebook Libraries: Pandas, matplotlib	
THEORY:		
Data Visualization with Pandas and Matplotlib		
	Pandas is very easy to use with **Matplotlib**, a powerful library used for creating basic plots and charts. With only a few lines of code, we can visualize our data and understand it better. Below are some simple examples to help you get started with plotting using Pandas and Matplotlib: # Import the library first import matplotlib.pyplot as plt **Histogram** A histogram shows the distribution of values in a column. df['income'].hist(bins=10) plt.title('Histogram of Income') plt.xlabel('Income Value') plt.ylabel('Frequency') plt.show()	



Box Plot

A **box plot** is useful to detect outliers and understand data spread.

```
df = df.head(10)
plt.figure(figsize=(20, 6))  # Increase width to make x-axis labels clearer
df.boxplot(column='imports', by='country')
plt.title('Boxplot by Country')
plt.suptitle('')  # Removes default title
plt.xlabel('Country')
plt.ylabel('Imports')
plt.xticks(rotation=45)  # Optional: Rotate x-axis labels for better visibility
plt.tight_layout()       # Adjust layout to avoid clipping
plt.show()
```



Scatter Plot

A **scatter plot** shows the relationship between two variables.

```
x = df["health"]
y = df["life_expec"]


plt.scatter(x, y, label="Data Points", color="m", marker="*", s=30)
```

plt.xlabel('Health')

plt.ylabel('Life Expectancy')

plt.title('Scatter Plot of Health vs Life Expectancy')

plt.legend()

plt.show()



Seaborn

While matplotlib is one of the most common data visualization libraries, we have another common library called - Seaborn. It is a library that has a high-level interface, and it is built on top of matplotlib, which means that it has added functionalities. The main advantage is that it provides more beautiful plots, with various color palettes making your data look pretty and pleasing to the eyes.

The seaborn library is generally used for statistical and multivariate (multiple variables) data visualization, and for plotting time series data. You can also fit and visualize linear regression models easily.

Matplotlib

It is a Python library for creating static, interactive and animated visualizations from data. It provides flexible and customizable plotting functions that help in understanding data patterns, trends and relationships effectively.

The parts of a Matplotlib figure include (as shown in the figure above):

- **Figure**: The overarching container that holds all plot elements, acting as the canvas for visualizations.
- **Axes**: The areas within the figure where data is plotted; each figure can contain multiple axes.
- **Axis**: Represents the x-axis and y-axis, defining limits, tick locations and labels for

data interpretation.

- **Lines and Markers**: Lines connect data points to show trends, while markers denote individual data points in plots like scatter plots.
- **Title and Labels**: The title provides context for the plot, while axis labels describe what data is being represented on each axis.

Matplotlib Pyplot

Pyplot is a module within Matplotlib that provides a MATLAB-like interface for making plots. It simplifies the process of adding plot elements such as lines, images and text to the axes of the current figure.

Steps to Use Pyplot:

- **Import Matplotlib:** Start by importing matplotlib.pyplot as plt.
- **Create Data:** Prepare your data in the form of lists or arrays.
- **Plot Data**: Use plt.plot() to create the plot.
- **Customize Plot**: Add titles, labels and other elements using methods like plt.title(), plt.xlabel() and plt.ylabel().
- **Display Plot**: Use plt.show() to display the plot.

Example-

Scatter plot:

```
# importing libraries
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
# loading the dataset
df = pd.read_csv("tips.csv")
sns.scatterplot(x='day', y='tip', data=df, hue='sex')
plt.show()
```

REFERENCES: Refer lab manual for the same

Practice Problem Statements:

1. An e-commerce company stores customer order information in a CSV file containing Date, Product, Category, Region, Quantity, Price, Sales and Profit. Import the dataset using Pandas and use Matplotlib to visualize monthly sales trends, category-wise sales, regional contribution, sales distribution, and the relationship between quantity sold and profit. Plots: Line, Bar, Pie, Histogram, Scatter.
2. A smart-city system records traffic information in a CSV file containing Date, Time, Location, Vehicle_Count, Average_Speed and Traffic_Density. Import the dataset and visualize traffic variation over time, location-wise vehicle count, vehicle-count distribution, and the relationship between vehicle count and average speed. Plots: Line, Bar, Histogram, Scatter.

Important Questions: -

1. What is the difference between plt.plot() and plt.scatter()?
2. How do you add labels, titles, and legends to a Matplotlib plot?
3. What is the significance of the 'bins' parameter in a histogram?
4. How can multiple plots be displayed in a single figure using subplots?
5. What is the difference between plt.show() and plt.savefig()?
6. When would you prefer a pie chart over a bar chart, and what are pie chart's limitations?

SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE - 411043

Department of Electronics & Computer Engineering

ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**

SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]

EXPT No:4

PROBLEM STATEMENT

Importing dataset (CSV file) and Python program to demonstrate the various plots using Matplotlib library on dataset.

ALGORITHM

Steps of plotting with Matplotlib:

1. Import **pandas, numpy** and `matplotlib.pyplot`; read Housing.csv and tesla_data.csv (convert Date with `pd.to_datetime`).
2. Select the chart type from the kind of data: trend over time → line; compare categories → bar; share of a whole → pie; distribution → histogram / box plot; relation of two numerical variables → scatter.
3. Prepare the data to be plotted (`value_counts()`, `groupby().mean()`, column selection).
4. Create the figure with `plt.figure(figsize=...)` or `plt.subplots(rows, cols)`.
5. Call the plotting function: `plt.plot, plt.bar, plt.pie, plt.hist, plt.scatter, plt.boxplot`.
6. Add the title, axis labels, legend, grid, colours and markers.
7. Arrange several plots in one figure and call `plt.tight_layout()`.
8. Display with `plt.show()` or save with `plt.savefig('name.png')`.
9. Interpret the patterns, spread and relationships visible in each plot.

ERRORS and TEST CASES, USE CASES

Common errors and their remedies

Error / symptom	Cause	Remedy
ValueError: x and y must have same first dimension	x and y have different lengths.	Check that `len(x) == len(y)`.
TypeError / could not convert string to float	A text column is plotted as numbers.	Convert with `pd.to_numeric()` or plot counts instead.
Dates unreadable on the x-axis	The Date column is still a string.	`pd.to_datetime` and `plt.xticks(rotation=45)`.
No plot is displayed	`plt.show()` missing or wrong backend.	Call `plt.show()`; use `%matplotlib inline` in Jupyter.
ValueError: Wedge sizes must be non-negative	Pie chart data contains negative or NaN values.	Clean the data with `dropna()` and check the values.
Plots overlap in one figure	A new figure was not started for each plot.	Use `plt.figure()` or subplots with `tight_layout()`.

Test cases

ID	Input / condition	Expected result
T1	`plt.plot([1,2,3],[2,4,6])`	A rising line through 3 points
T2	Bar chart of a column with 2 categories	2 bars whose heights equal the category counts
T3	Pie chart with `autopct='%1.1f%%'`	The percentages add up to 100
T4	Histogram of the 545 house prices with bins = 10	10 bars; the bar heights add up to 545
T5	Scatter plot of area vs price	545 points with an upward trend (r = 0.536)
T6	Line plot of Tesla closing price against Date	Date on the x-axis in order; one value per trading day
T7	(Negative test) `plt.plot([1,2,3],[1,2])`	ValueError (unequal lengths)
T8	`plt.savefig('plot.png')`	Image file is created in the working folder

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:4	
PROBLEM STATEMENT	Importing dataset (CSV file) and Python program to demonstrate the various plots using Matplotlib library on dataset.

Use cases

- Business dashboards: sales by category, share of each product.
- Time-series visualization such as stock prices or temperature.
- Exploring distributions and outliers before building a model.
- Survey and census results presented as bar and pie charts.
- Figures for reports and research papers.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 **Department of Electronics & Computer Engineering** ASSESSMENT YEAR: 2026-2027 | CLASS: BE SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No: 4	
PROBLEM STATEMENT	Importing dataset (CSV file) and Python program to demonstrate the various plots using Matplotlib library on dataset.

Libraries: pandas, numpy, matplotlib. Datasets: Housing.csv (bar, pie, histogram, scatter, box plots) and tesla_data.csv (line plot of time series data).

PROGRAM AND OUTPUT

Import dataset

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
pd.set_option("display.width", 85)
pd.set_option("display.max_columns", 20)

house = pd.read_csv("Housing.csv")
print("Housing.csv shape:", house.shape)
print(house.head())
print("\nColumns:", list(house.columns))

tesla = pd.read_csv("tesla_data.csv")
tesla["Date"] = pd.to_datetime(tesla["Date"])
print("\ntesla_data.csv shape:", tesla.shape)
print(tesla[["Date", "Open", "High", "Low", "Close", "Volume"]].head())
```

Output:

```
Housing.csv shape: (545, 13)
      price  area  bedrooms  bathrooms  stories mainroad guestroom basement  \
0  13300000  7420         4          2        3      yes        no       no
1  12250000  8960         4          4        4      yes        no       no
2  12250000  9960         3          2        2      yes        no      yes
3  12215000  7500         4          2        2      yes        no      yes
4  11410000  7420         4          1        2      yes       yes      yes

  hotwaterheating airconditioning  parking prefarea furnishingstatus
0              no             yes        2      yes        furnished
1              no             yes        3       no        furnished
2              no              no        2      yes   semi-furnished
3              no             yes        3      yes        furnished
4              no             yes        2       no        furnished

Columns: ['price', 'area', 'bedrooms', 'bathrooms', 'stories', 'mainroad', 'guestroom',
'basement', 'hotwaterheating', 'airconditioning', 'parking', 'prefarea', 'furnishingstatus']

tesla_data.csv shape: (251, 9)
        Date        Open        High         Low       Close    Volume
0 2022-08-29  282.829987  287.739990  280.700012  284.820007  41864700
1 2022-08-30  287.869995  288.480011  272.649994  277.700012  50541800
2 2022-08-31  280.619995  281.250000  271.809998  275.609985  52107300
3 2022-09-01  272.579987  277.579987  266.149994  277.160004  54287000
4 2022-09-02  281.070007  282.350006  269.079987  270.209991  50890100
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 **Department of Electronics & Computer Engineering** **ASSESSMENT YEAR: 2026-2027** | **CLASS: BE** **SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]**
EXPT No: 4	
PROBLEM STATEMENT	Importing dataset (CSV file) and Python program to demonstrate the various plots using Matplotlib library on dataset.

1. Line plot

Code:

```
plt.figure(figsize=(8, 3.8))
plt.plot(tesla["Date"], tesla["Close"], color="blue", linewidth=1.5, label="Close price")
plt.title("Line plot: Tesla closing price trend")
plt.xlabel("Date")
plt.ylabel("Closing price (USD)")
plt.grid(True)
plt.legend()
plt.show()
```

Output:



2. Bar plot

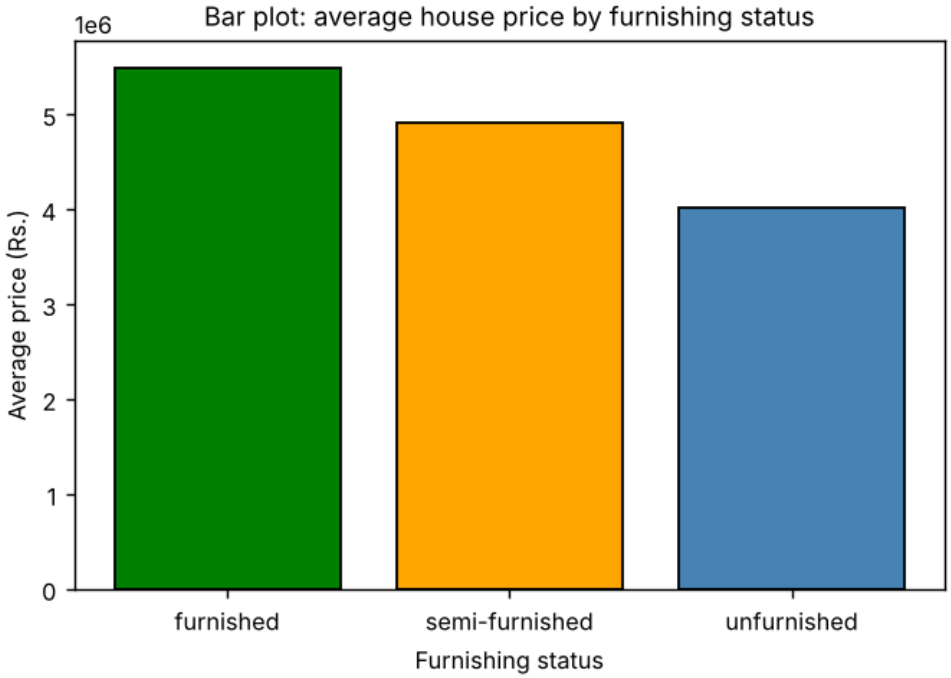
Code:

```
avg = house.groupby("furnishingstatus")["price"].mean()
print(avg.round(0))
plt.figure(figsize=(6, 3.8))
plt.bar(avg.index, avg.values, color=["green", "orange", "steelblue"], edgecolor="black")
plt.title("Bar plot: average house price by furnishing status")
plt.xlabel("Furnishing status")
plt.ylabel("Average price (Rs.)")
```

```
plt.show()
```

Output:

```
furnishingstatus
furnished          5495696.0
semi-furnished     4907524.0
unfurnished        4013831.0
Name: price, dtype: float64
```



3. Pie chart

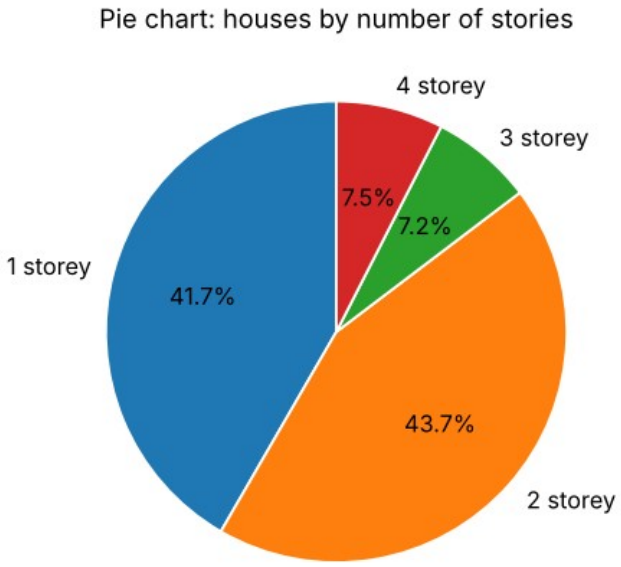
Code:

```
counts = house["stories"].value_counts().sort_index()
print(counts)
plt.figure(figsize=(5, 4))
plt.pie(counts.values, labels=[str(s) + " storey" for s in counts.index],
        autopct="%1.1f%%", startangle=90, wedgeprops={"edgecolor": "white"})
plt.title("Pie chart: houses by number of stories")
plt.show()
```

Output:

```
stories
1    227
2    238
```

```
3     39
4     41
Name: count, dtype: int64
```



4. Histogram

Code:

```
plt.figure(figsize=(6.5, 3.8))
plt.hist(house["price"], bins=15, color="skyblue", edgecolor="black")
plt.title("Histogram: distribution of house prices")
plt.xlabel("Price (Rs.)")
plt.ylabel("Frequency")
plt.show()
print("Mean price  :", round(house["price"].mean()))
print("Median price:", round(house["price"].median()))
```

Output:



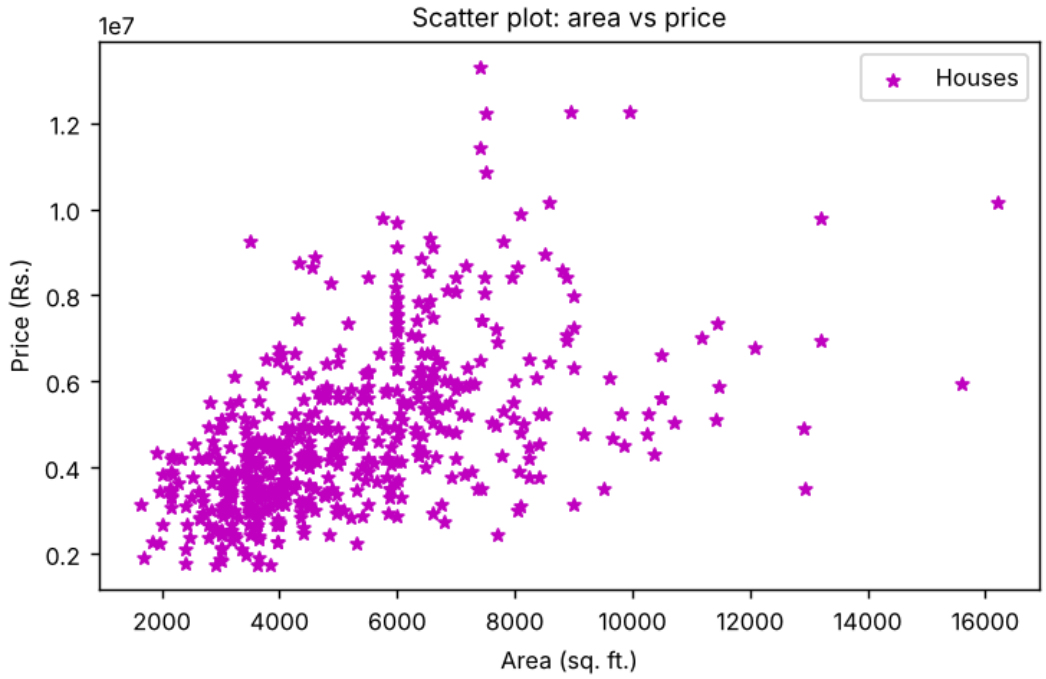

```
Mean price  : 4766729
Median price: 4340000
```

5. Scatter plot

Code:

```
plt.figure(figsize=(6.5, 3.8))
plt.scatter(house["area"], house["price"], color="m", marker="*", s=25, label="Houses")
plt.title("Scatter plot: area vs price")
plt.xlabel("Area (sq. ft.)")
plt.ylabel("Price (Rs.)")
plt.legend()
plt.show()
print("Correlation between area and price:", round(house["area"].corr(house["price"]), 3))
```

Output:




```
Correlation between area and price: 0.536
```

6. Box plot and subplots

Code:

```
fig, ax = plt.subplots(2, 2, figsize=(10, 7))
ax[0, 0].plot(tesla["Date"], tesla["Volume"], color="green")
ax[0, 0].set_title("Line: Tesla trading volume")
ax[0, 0].tick_params(axis="x", rotation=30)

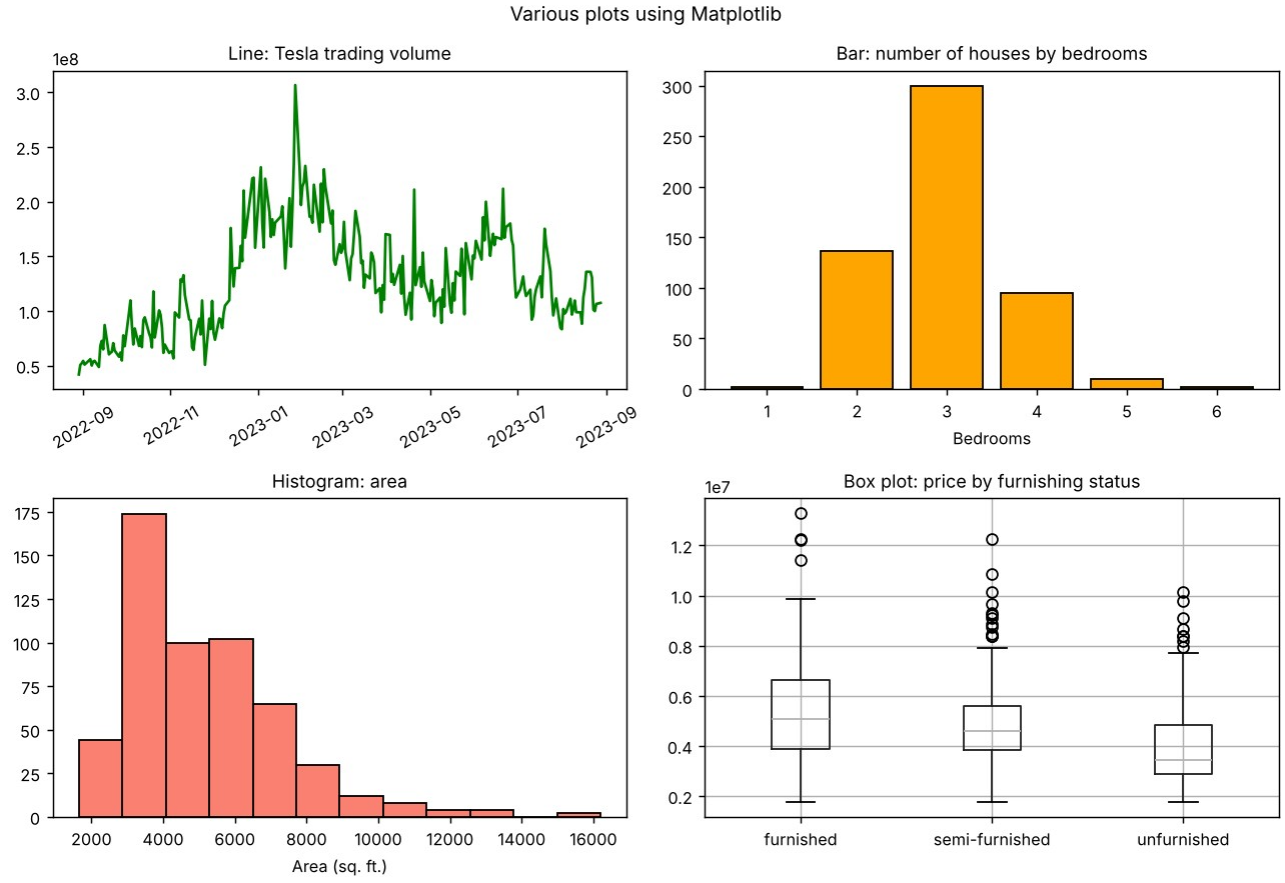
bed = house["bedrooms"].value_counts().sort_index()
ax[0, 1].bar(bed.index, bed.values, color="orange", edgecolor="black")
ax[0, 1].set_title("Bar: number of houses by bedrooms")
ax[0, 1].set_xlabel("Bedrooms")

ax[1, 0].hist(house["area"], bins=12, color="salmon", edgecolor="black")
ax[1, 0].set_title("Histogram: area")
ax[1, 0].set_xlabel("Area (sq. ft.)")

house.boxplot(column="price", by="furnishingstatus", ax=ax[1, 1])
ax[1, 1].set_title("Box plot: price by furnishing status")
ax[1, 1].set_xlabel("")
```

```
plt.suptitle("Various plots using Matplotlib")
plt.tight_layout()
plt.savefig("subplots.png", dpi=150)
plt.show()
print("Figure saved as subplots.png")
```

Output:




```
Figure saved as subplots.png
```

IMPORTANT QUESTIONS AND ANSWERS

Q1. What is the difference between plt.plot() and plt.scatter()?

plt.plot()	plt.scatter()
Draws a line joining the points (markers are optional)	Draws only individual points (markers)
Used for trends, time series and continuous data	Used to study the relationship or correlation between two variables
All points have the same style	Size (s), colour (c) and transparency (alpha) of each point can be different
Faster for a large number of points	Slower for a large number of points

```
plt.plot(x, y, marker='o')       # line plot
plt.scatter(x, y, c='red', s=40)  # scatter plot
```

Q2. How do you add labels, titles, and legends to a Matplotlib plot?

```
plt.plot(months, sales, label='Sales', color='blue')
plt.plot(months, profit, label='Profit', color='green')
plt.title('Monthly Sales and Profit')   # title
plt.xlabel('Month')                     # x-axis label
plt.ylabel('Amount (Rs.)')              # y-axis label
plt.legend(loc='upper left')            # legend uses the label= of each plot
plt.grid(True)
plt.show()
```

The label given inside each plot call is what appears in the legend. In the object-oriented style the same is done with ax.set_title(), ax.set_xlabel(), ax.set_ylabel() and ax.legend().

Q3. What is the significance of the 'bins' parameter in a histogram?

A histogram divides the range of the data into intervals called **bins** and counts how many values fall in each bin. The number of bins controls the level of detail of the plot.

- **Too few bins** hide the shape of the distribution (over-smoothing).
- **Too many bins** make the plot noisy, with gaps and spikes.
- A suitable number of bins shows the true shape (normal, skewed, bimodal) and the outliers.

bins can be an integer (bins=10), a list of edges (bins=[0, 20, 40, 60]) or a rule such as 'auto' or 'sturges'.

```
plt.hist(df['Sales'], bins=10, edgecolor='black')
```

Q4. How can multiple plots be displayed in a single figure using subplots?

plt.subplots(rows, cols) creates one figure with a grid of axes, and each plot is drawn on its own axes.

```
fig, ax = plt.subplots(2, 2, figsize=(10, 8))
ax[0, 0].plot(x, y);          ax[0, 0].set_title('Line')
ax[0, 1].bar(cat, val);       ax[0, 1].set_title('Bar')
ax[1, 0].hist(data, bins=10); ax[1, 0].set_title('Histogram')
ax[1, 1].scatter(x, y);       ax[1, 1].set_title('Scatter')
plt.tight_layout()            # avoids overlapping of titles and labels
plt.show()
```

The older method is plt.subplot(2, 2, 1), where the three numbers are rows, columns and the plot number.

Q5. What is the difference between plt.show() and plt.savefig()?

plt.show()	**plt.savefig()**
Displays the figure in a window or notebook	Saves the figure to a file (png, jpg, pdf, svg)
Nothing is stored on disk	Creates a file on disk
Clears the current figure after it is shown (in scripts)	Does not display the figure

savefig() must be called **before** show(); otherwise a blank image may be saved.

```
plt.savefig('sales.png', dpi=300, bbox_inches='tight')
plt.show()
```

Q6. When would you prefer a pie chart over a bar chart, and what are the pie chart's limitations?

A pie chart is preferred when showing the **share of each part in a whole** (parts add up to 100%) and there are only a few categories (about 2 to 5). Examples: region-wise share of total sales, category-wise sales contribution.

Limitations of a pie chart:

- It becomes unreadable with many categories or very small slices.
- It is difficult to compare angles/areas of similar-sized slices, whereas bar lengths are easy to compare.
- It cannot show negative values or changes over time.
- It cannot be used to compare several datasets side by side.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 Department of Electronics & Computer Engineering ASSESSMENT YEAR: 2026-2027 | CLASS: BE SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No: 4	
PROBLEM STATEMENT	Importing dataset (CSV file) and Python program to demonstrate the various plots using Matplotlib library on dataset.

A bar chart is better for comparing values across categories, ranking, and when there are many categories.

```
plt.pie(sales, labels=region, autopct='%1.1f%%')
```

CONCLUSION:
Signature and Date:

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:5	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	
PROBLEM STATEMENT	Stock market analysis using Python Pandas with suitable dataset.	
PREREQUISITES	Basic understanding of stock market terms (closing price, moving average, daily returns) Time-series data handling concepts Pandas DataFrame and indexing knowledge	
OBJECTIVES:	To import and preprocess stock market data using Pandas. To analyze trends such as moving averages and daily returns. To visualize stock price movement over time for decision-making insights.	
Tools Used	Python 3.x Jupyter Notebook Libraries: pandas, numpy, matplotlib	
THEORY:		
Dataset Information:		

Importing Libraries:

Python libraries make it very easy for us to handle the data and perform typical and complex tasks with a single line of code.

- Pandas - This library helps to load the data frame in a 2D array format and has multiple functions to perform analysis tasks in one go.
- Numpy - Numpy arrays are very fast and can perform large computations in a very short time.
- Matplotlib/Seaborn - This library is used to draw visualizations.
- Sklearn - This module contains multiple libraries having pre-implemented functions to perform tasks from data preprocessing to model development and evaluation.
- XGBoost - This contains the eXtreme Gradient Boosting machine learning algorithm which is one of the algorithms which helps us to achieve high accuracy on predictions.

The dataset we will use here to perform the analysis and build a predictive model is Tesla Stock Price data. We will use OHLC('Open', 'High', 'Low', 'Close') data from 1st January 2010 to 31st December 2017 which is for 8 years for the Tesla stocks.

- Commands To be Used to import CSV file

```
df = pd.read_csv('/content/Tesla.csv')
```

df.head(): we can see that data for some of the dates is missing the reason for that is on

weekends and holidays Stock Market remains closed hence no trading happens on these days
df.shape: From this, we got to know that there are 1692 rows of data available and for each row, we have 7 different features or columns.
df.describe(): It gives a statistical summary of numerical columns**.**
df.info():It gives the structural information about the DataFrame**.**

Exploratory Data Analysis

EDA is an approach to analyzing the data using visual techniques. It is used to discover trends, and patterns, or to check assumptions with the help of statistical summaries and graphical representations. While performing the EDA of the Tesla Stock Price data we will analyze how prices of the stock have moved over the period of time and how the end of the quarters affects the prices of the stock.

```
plt.figure(figsize=(15,5))
plt.plot(df['Close'])
plt.title('Tesla Close price.', fontsize=15)
plt.ylabel('Price in dollars.')
plt.show()
```

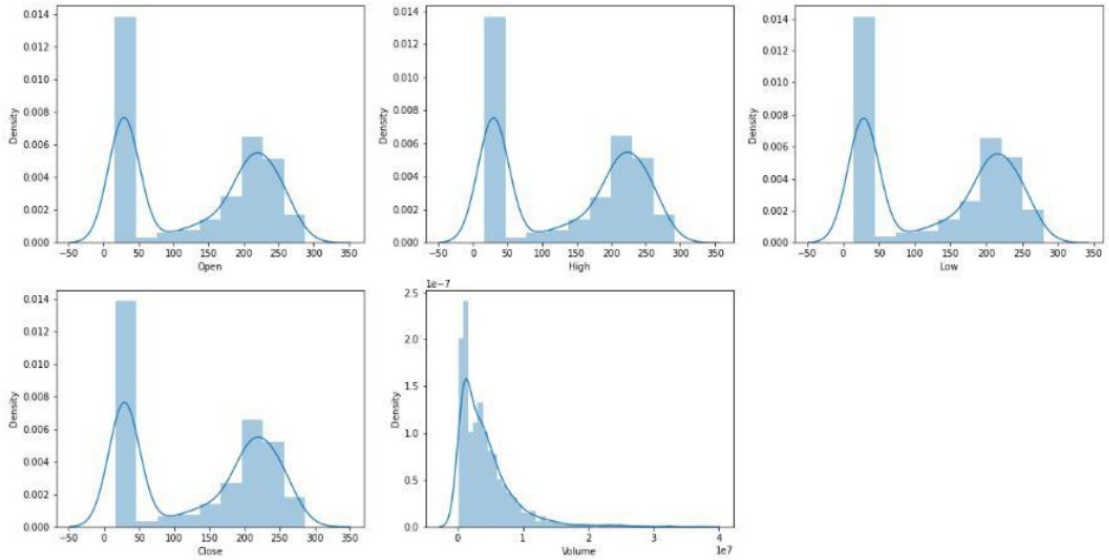
Output:



The prices of tesla stocks are showing an upward trend as depicted by the plot of the closing price of the stocks.
df.head():
df[df['Close'] == df['Adj Close']].shape: If we observe carefully we can see that the data in the 'Close' column and that available in the 'Adj Close' column is the same let's check whether this is the case with each row or not.
df = df.drop(['Adj Close'], axis=1): From here we can conclude that all the rows of columns 'Close' and 'Adj Close' have the same data. So, having redundant data in the dataset is not going to help so, we'll drop this column before further analysis.

```
features = ['Open', 'High', 'Low', 'Close', 'Volume']
plt.subplots(figsize=(20,10))
for i, col in enumerate(features):
  plt.subplot(2,3,i+1)
  sb.distplot(df[col])
plt.show()
```

Output:

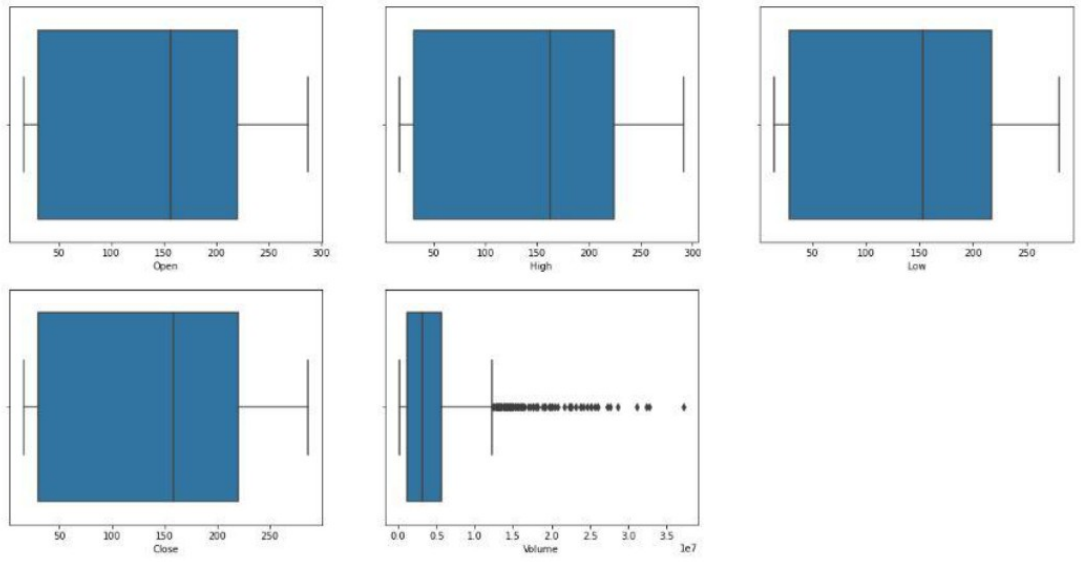


Distribution Plot of the Continuous Variable

In the distribution plot of OHLC data, we can see two peaks which means the data has varied significantly in two regions. And the Volume data is left-skewed.

```
plt.subplots(figsize=(20,10))
for i, col in enumerate(features):
  plt.subplot(2,3,i+1)
  sb.boxplot(df[col])
plt.show()
```

Output:



Box Plot of the Continuous Variable

From the above boxplots, we can conclude that only volume data contains outliers in it but the data in the rest of the columns are free from any outlier.

Feature Engineering

Feature Engineering helps to derive some valuable features from the existing ones. These extra

features sometimes help in increasing the performance of the model significantly and certainly help to gain deeper insights into the data.

```
splitted = df['Date'].str.split('/', expand=True)
df['day'] = splitted[1].astype('int')
df['month'] = splitted[0].astype('int')
df['year'] = splitted[2].astype('int')
df.head()
```

we have three more columns namely 'day', 'month' and 'year' all these three have been derived from the 'Date' column which was initially provided in the data.

```
df['is_quarter_end'] = np.where(df['month']%3==0,1,0)
df.head()
```

	Date	Open	High	Low	Close	Volume	day	month	year	is_quarter_end
0	6/29/2010	19.000000	25.00	17.540001	23.889999	18766300	29	6	2010	1
1	6/30/2010	25.790001	30.42	23.299999	23.830000	17187100	30	6	2010	1
2	7/1/2010	25.000000	25.92	20.270000	21.959999	8218800	1	7	2010	0
3	7/2/2010	23.000000	23.10	18.709999	19.200001	5139800	2	7	2010	0
4	7/6/2010	20.000000	20.00	15.830000	16.110001	6866900	6	7	2010	0

A quarter is defined as a group of three months. Every company prepares its quarterly results and publishes them publicly so, that people can analyze the company's performance. These quarterly results affect the stock prices heavily which is why we have added this feature because this can be a helpful feature for the learning model.

```
data_grouped = df.drop('Date', axis=1).groupby('year').mean()
plt.subplots(figsize=(20,10))

for i, col in enumerate(['Open', 'High', 'Low', 'Close']):
  plt.subplot(2,2,i+1)
  data_grouped[col].plot.bar()
plt.show()
```

REFERENCES: Refer lab manual for the same

Practice Problem Statements:

1. A financial analyst wants to study the historical performance of Tesla (TSLA) using daily stock-market data. Write a Python program using Pandas to find the highest and lowest closing prices and their corresponding dates, Calculate the daily percentage change in closing price. Display the 10 days having the highest daily gain and Plot the closing-price trend using Matplotlib.
2. Multi- A stock-market dataset contains historical daily records of different companies with the attributes Date, Symbol, Open, High, Low, Close, and Volume. Write a Python program using Pandas to analyze the performance and risk of the stocks: 1. Calculate the daily return for each stock. 2. Calculate the mean daily return and standard deviation of daily returns for each stock. 3. Rank the stocks according to their average return. 4. Identify the most volatile stock based on standard deviation. 5. Calculate the 20-day rolling average of the closing price. 6. Generate a correlation matrix of the

daily returns of all stocks. (Use A stock historical OHLCV dataset containing at least 3–5 companies)

3. You are provided with the file stock_market_small.csv, containing historical OHLCV data of three companies: AAPL, MSFT and GOOGL.Write a Python program using Pandas to perform the following: Find the maximum closing price for each company, Find the average trading volume for each company, Calculate the daily percentage change in closing price for each company, Display the company with the highest average closing price, Sort the records according to the closing price in descending order and Plot a comparison of the closing prices of the three companies using Matplotlib.

SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE - 411043

Department of Electronics & Computer Engineering

ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**

SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]

EXPT No:5

PROBLEM STATEMENT

Stock market analysis using Python Pandas with suitable dataset.

ALGORITHM

Steps of stock market analysis:

1. Import **pandas, numpy, matplotlib**; load the stock CSV (tesla_data.csv), convert Date with `pd.to_datetime` and sort in ascending order of date.
2. Inspect the data: `shape`, `head()`, `isnull().sum()`, `describe()`; drop Adj Close if it equals Close.
3. Optionally set Date as the index for time-based slicing.
4. Daily return = `Close.pct_change() * 100` (percentage change from the previous day).
5. Moving average = `Close.rolling(window=n).mean()` for n = 20 and 50 days.
6. Highest and lowest closing price with `idxmax()` / `idxmin()`; the 10 best days with `nlargest(10)`.
7. Volatility = standard deviation of daily returns; cumulative return = (1 + r).cumprod() − 1.
8. Multi-stock data: pivot Close by Symbol, rank the stocks by mean return, find the most volatile stock and the correlation matrix.
9. Plot the closing price with its moving averages and the histogram of returns; interpret the trend and risk.

ERRORS and TEST CASES, USE CASES

Common errors and their remedies

Error / symptom	Cause	Remedy
Wrong values in rolling / pct_change	Data is not sorted by date.	Use `sort_values('Date')` first.
x-axis shows strings or 0, 1, 2 ...	Date column is of object type.	Convert with `pd.to_datetime`.
NaN in the first rows	`pct_change` and `rolling` need previous values (window − 1 NaNs).	Expected behaviour; use `dropna()` or `min_periods`.
KeyError: 'Adj Close'	Column name differs or was already dropped.	Check `df.columns`.
Returns of different companies get mixed	Several symbols in a single column.	Use `groupby('Symbol')` before `pct_change`.
Dates appear to be missing	The market is closed on weekends and holidays.	Do not fill them; they are not data errors.

Test cases

ID	Input / condition	Expected result
T1	Close = [100, 110, 99]	Daily return: NaN, +10%, −10%
T2	Close = [10, 20, 30, 40], 3-day moving average	NaN, NaN, 20, 30
T3	20-day moving average on N rows	First 19 values are NaN, the rest are numbers
T4	tesla_data.csv loaded and sorted	251 daily records; Date is increasing
T5	`idxmax()` and `idxmin()` of Close	Highest close is greater than or equal to the lowest close, with their dates
T6	Correlation of a stock with itself	1.0 (the diagonal of the correlation matrix)
T7	Stock with the largest standard deviation of returns	Identified as the most volatile stock
T8	(Negative test) rolling average without sorting dates	Values are meaningless; `df['Date'].is_monotonic_increasing` is False

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:5		
PROBLEM STATEMENT	Stock market analysis using Python Pandas with suitable dataset.	

Use cases

- Investment analysis: identifying trends with moving averages.
- Risk assessment using the volatility (standard deviation) of returns.
- Trading strategies such as moving-average crossover.
- Portfolio diversification using the correlation of stocks.
- Financial dashboards and automatic report generation.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:5	
PROBLEM STATEMENT	Stock market analysis using Python Pandas with suitable dataset.

Libraries: pandas, numpy, matplotlib. Datasets: tesla_data.csv (TSLA) and stock_market_data_large.csv (second stock, named STOCK2, taken for the same period as TSLA).

PROGRAM AND OUTPUT

Import dataset

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
pd.set_option("display.width", 85)
pd.set_option("display.max_columns", 20)

df = pd.read_csv("tesla_data.csv")
df["Date"] = pd.to_datetime(df["Date"])
df = df[["Date", "Open", "High", "Low", "Close", "Adj Close", "Volume"]]
print("Shape:", df.shape)
print(df.head())
print("\nPeriod:", df["Date"].min().date(), "to", df["Date"].max().date())
print("\nMissing values:")
print(df.isnull().sum())
print("\nStatistical summary:")
print(df.drop(columns="Date").describe().round(2))
print("\nClose equals Adj Close in all rows:", (df["Close"] == df["Adj Close"]).all())
df = df.drop(columns=["Adj Close"])
```

Output:

```
Shape: (251, 7)
        Date        Open        High         Low       Close   Adj Close    Volume
0 2022-08-29  282.829987  287.739990  280.700012  284.820007  284.820007  41864700
1 2022-08-30  287.869995  288.480011  272.649994  277.700012  277.700012  50541800
2 2022-08-31  280.619995  281.250000  271.809998  275.609985  275.609985  52107300
3 2022-09-01  272.579987  277.579987  266.149994  277.160004  277.160004  54287000
4 2022-09-02  281.070007  282.350006  269.079987  270.209991  270.209991  50890100

Period: 2022-08-29 to 2023-08-28

Missing values:
Date         0
Open         0
High         0
Low          0
Close        0
Adj Close    0
Volume       0
dtype: int64

Statistical summary:
         Open    High     Low   Close  Adj Close        Volume
```

```
count  251.00  251.00  251.00  251.00     251.00  2.510000e+02
mean   208.54  213.04  203.65  208.43     208.43  1.274835e+08
std     48.22   48.61   47.72   48.15      48.15  4.724183e+07
min    103.00  111.75  101.81  108.10     108.10  4.186470e+07
25%    176.64  182.42  172.50  178.47     178.47  9.351960e+07
50%    198.54  203.00  192.89  197.58     197.58  1.219993e+08
75%    249.39  253.03  242.39  249.57     249.57  1.599710e+08
max    308.29  313.80  305.58  309.07     309.07  3.065906e+08

Close equals Adj Close in all rows: True
```

Problem 1: highest/lowest close

Code:

```
hi = df.loc[df["Close"].idxmax()]
lo = df.loc[df["Close"].idxmin()]
print("Highest closing price: %.2f on %s" % (hi["Close"], hi["Date"].date()))
print("Lowest closing price : %.2f on %s" % (lo["Close"], lo["Date"].date()))
```

Output:

```
Highest closing price: 309.07 on 2022-09-19
Lowest closing price : 108.10 on 2023-01-03
```

Problem 1: daily % change, top 10 gains

Code:

```
df["Daily_Change_%"] = df["Close"].pct_change() * 100
print(df[["Date", "Close", "Daily_Change_%"]].head(6))

top10 = df.nlargest(10, "Daily_Change_%")[["Date", "Close", "Daily_Change_%"]]
print("\nTop 10 days with highest daily gain:")
print(top10.to_string(index=False))
```

Output:

```
        Date       Close  Daily_Change_%
0 2022-08-29  284.820007             NaN
1 2022-08-30  277.700012       -2.499823
2 2022-08-31  275.609985       -0.752620
3 2022-09-01  277.160004        0.562396
4 2022-09-02  270.209991       -2.507581
5 2022-09-06  274.420013        1.558056

Top 10 days with highest daily gain:
      Date      Close  Daily_Change_%
2023-01-27 177.899994       11.000180
2023-01-26 160.270004       10.967259
2022-12-29 121.820000        8.082691
2022-11-23 183.199997        7.821784
2023-03-21 197.580002        7.819919
2023-01-23 143.750000        7.742469
2022-11-30 194.699997        7.670185
```

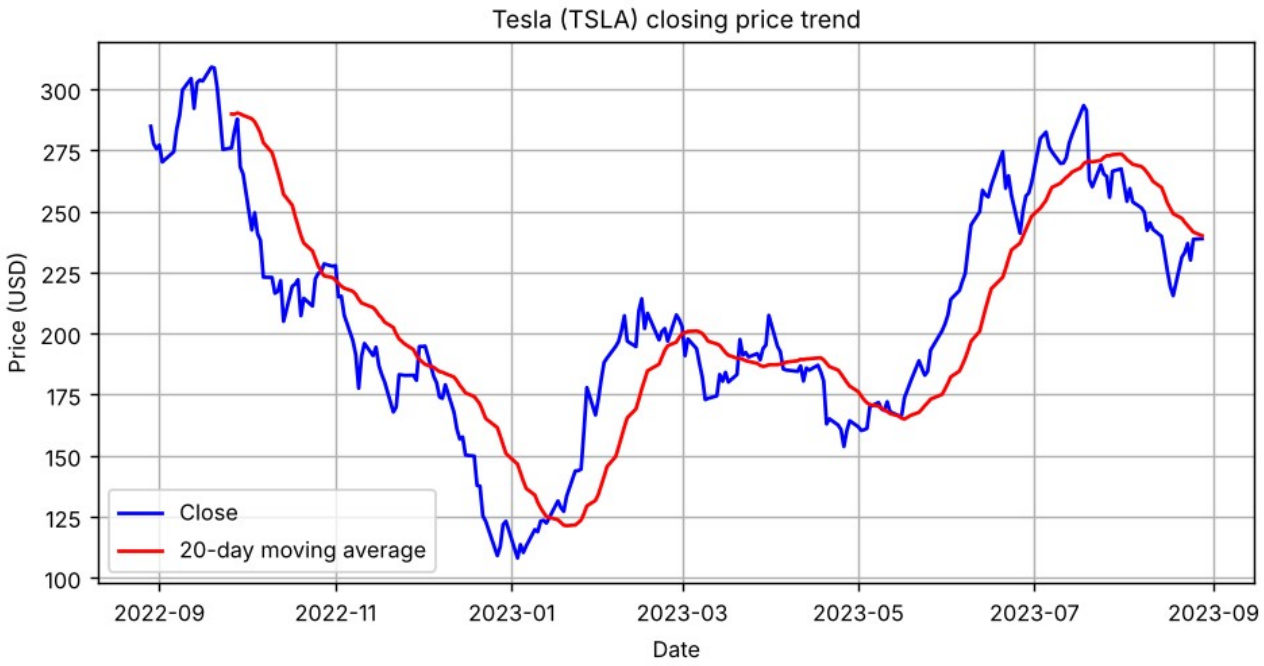
```
2023-02-14 209.250000        7.506166
2023-01-17 131.490005        7.426474
2022-11-10 190.720001        7.393437
```

Problem 1: closing price trend plot

Code:

```
df["MA20"] = df["Close"].rolling(window=20).mean()
plt.figure(figsize=(8.5, 4))
plt.plot(df["Date"], df["Close"], label="Close", color="blue")
plt.plot(df["Date"], df["MA20"], label="20-day moving average", color="red")
plt.title("Tesla (TSLA) closing price trend")
plt.xlabel("Date"); plt.ylabel("Price (USD)")
plt.legend(); plt.grid(True)
plt.show()
print("Mean daily return : %.4f %%" % df["Daily_Change_%"].mean())
print("Std. deviation    : %.4f %%" % df["Daily_Change_%"].std())
```

Output:




```
Mean daily return : 0.0012 %
Std. deviation    : 3.7812 %
```

Problem 2: multi-stock dataset

Code:

```
big = pd.read_csv("stock_market_data_large.csv")
big["Date"] = pd.to_datetime(big["Date"])
print("stock_market_data_large.csv shape:", big.shape)
print(big.head(3))

t = pd.read_csv("tesla_data.csv", parse_dates=["Date"])[["Date", "Open", "High", "Low",
"Close", "Volume"]]
t["Symbol"] = "TSLA"
b = big[(big["Date"] >= t["Date"].min()) & (big["Date"] <= t["Date"].max())][["Date",
"Open", "High", "Low", "Close", "Volume"]].copy()
b["Symbol"] = "STOCK2"
stocks = pd.concat([t, b], ignore_index=True)
print("\nCombined dataset shape:", stocks.shape)
print(stocks.groupby("Symbol")["Close"].count())
```

Output:

```
stock_market_data_large.csv shape: (13647, 10)
        Date        Open        High         Low       Close   Volume        RSI  \
0 2010-01-01  100.309172  102.741242   99.822837  101.381330   743835  75.905800
1 2010-01-04  101.147509  101.681257   99.244474  100.396541  2693069  81.444127
2 2010-01-05  101.567491  103.949668  101.509347  103.012156  3027306  71.544018

       MACD  Sentiment  Target
0  0.000000   0.058089       0
1 -0.078559   0.751219       1
2  0.069441   0.979723       1

Combined dataset shape: (512, 7)
Symbol
STOCK2    261
TSLA      251
Name: Close, dtype: int64
```

Problem 2: daily return, mean, std, rank

Code:

```
stocks = stocks.sort_values(["Symbol", "Date"])

stocks["Return"] = stocks.groupby("Symbol")["Close"].pct_change()
print(stocks[["Date", "Symbol", "Close", "Return"]].head(4))

summary = stocks.groupby("Symbol")["Return"].agg(["mean", "std"])
print("\nMean and standard deviation of daily returns:")
print(summary.round(5))

summary["Rank"] = summary["mean"].rank(ascending=False).astype(int)
print("\nRanking by average return:")
print(summary.sort_values("Rank").round(5))
```

```
print("\nMost volatile stock:", summary["std"].idxmax())
```

Output:

```
          Date  Symbol       Close    Return
251 2022-08-29  STOCK2  258.106090       NaN
252 2022-08-30  STOCK2  257.802607 -0.001176
253 2022-08-31  STOCK2  257.236569 -0.002196
254 2022-09-01  STOCK2  259.042438  0.007020

Mean and standard deviation of daily returns:
           mean      std
Symbol
STOCK2  0.00032  0.00887
TSLA    0.00001  0.03781

Ranking by average return:
           mean      std  Rank
Symbol
STOCK2  0.00032  0.00887     1
TSLA    0.00001  0.03781     2

Most volatile stock: TSLA
```

Problem 2: rolling average, correlation

Code:

```
stocks["MA20"] = stocks.groupby("Symbol")["Close"].transform(lambda s: s.rolling(20).mean())
print(stocks[stocks["Symbol"] == "TSLA"][["Date", "Close", "MA20"]].iloc[18:23])

ret = stocks.pivot(index="Date", columns="Symbol", values="Return")
print("\nCorrelation matrix of daily returns:")
print(ret.corr().round(4))

cl = stocks.pivot(index="Date", columns="Symbol", values="Close")
cl.plot(figsize=(8.5, 4), title="Comparison of closing prices")
plt.ylabel("Closing price"); plt.grid(True)
plt.show()
```

Output:

```
         Date       Close        MA20
18 2022-09-23  275.329987         NaN
19 2022-09-26  276.010010  289.867502
20 2022-09-27  282.940002  289.773502
21 2022-09-28  287.809998  290.279001
22 2022-09-29  268.209991  289.909001

Correlation matrix of daily returns:
Symbol  STOCK2    TSLA
Symbol
STOCK2  1.0000 -0.0346
```

```
TSLA   -0.0346  1.0000
```



Problem 3: per-company statistics and sorting

Code:

```
print("Maximum closing price of each company:")
print(stocks.groupby("Symbol")["Close"].max().round(2))
print("\nAverage trading volume of each company:")
print(stocks.groupby("Symbol")["Volume"].mean().round(0))
avg_close = stocks.groupby("Symbol")["Close"].mean()
print("\nAverage closing price:")
print(avg_close.round(2))
print("Company with highest average closing price:", avg_close.idxmax())

print("\nRecords sorted by closing price (descending), top 5:")
print(stocks.sort_values("Close", ascending=False)[["Date", "Symbol",
"Close"]].head(5).to_string(index=False))
```

Output:

```
Maximum closing price of each company:
Symbol
STOCK2    291.32
TSLA      309.07
```

```
Name: Close, dtype: float64

Average trading volume of each company:
Symbol
STOCK2      2489612.0
TSLA      127483453.0
Name: Volume, dtype: float64

Average closing price:
Symbol
STOCK2    273.94
TSLA      208.43
Name: Close, dtype: float64
Company with highest average closing price: STOCK2

Records sorted by closing price (descending), top 5:
       Date Symbol      Close
2022-09-19   TSLA 309.070007
2022-09-20   TSLA 308.730011
2022-09-12   TSLA 304.420013
2022-09-15   TSLA 303.750000
2022-09-16   TSLA 303.350006
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:5		
PROBLEM STATEMENT	Stock market analysis using Python Pandas with suitable dataset.	

IMPORTANT QUESTIONS AND ANSWERS

Note: the lab manual does not list important questions for this practical, so the following questions are based on its theory and practice problems.

Q1. What is a moving average? How is it calculated in Pandas and why is it used in stock analysis?

A moving (rolling) average is the average of the closing price over the last N days, recalculated for every day as the window moves forward. It smooths out day-to-day fluctuations so that the real trend is easier to see. A short window (for example 20 days) reacts quickly, while a long window (for example 200 days) shows the long-term trend. When the price moves above its moving average it is usually taken as an upward trend, and below it as a downward trend. The first N-1 values are NaN because the window is not yet full.

```
df['MA20'] = df['Close'].rolling(window=20).mean()
df[['Close', 'MA20']].plot(figsize=(12, 5))
```

Q2. How is the daily return (daily percentage change) calculated?

Daily return = (Close today - Close yesterday) / Close yesterday x 100. It tells how much the price gained or lost in one day in percentage, which makes different stocks comparable irrespective of their price level.

```
df['Return'] = df['Close'].pct_change() * 100
df['Return'].nlargest(10)           # 10 days with the highest gain
```

The first row is NaN because there is no previous day.

Q3. How do you find the highest and lowest closing price and the date on which they occurred?

```
df['Date'] = pd.to_datetime(df['Date'])
hi = df.loc[df['Close'].idxmax()]
lo = df.loc[df['Close'].idxmin()]
print(hi['Date'], hi['Close'])
print(lo['Date'], lo['Close'])
```

max() and min() give only the value, whereas idxmax() and idxmin() give the row label, from which the date can be taken.

Q4. Why are some dates missing in the stock dataset? Why was the 'Adj Close' column dropped?

The stock market is closed on Saturdays, Sundays and holidays, so there are no trading records for those dates. Because of this the Tesla file has only 1692 rows for 8 years instead of about 2900 calendar days.

'Adj Close' is the closing price adjusted for dividends and stock splits. In the Tesla data it was found to be identical to 'Close' in every row (checked with df[df['Close'] == df['Adj Close']].shape), so it is redundant and was dropped with df.drop(['Adj Close'], axis=1) to avoid duplicate information.

Q5. What is volatility, and how can the riskiest stock be identified?

Volatility is the degree of variation in the price of a stock. It is measured by the **standard deviation of the daily returns**. A higher standard deviation means larger price swings and therefore higher risk. Mean daily return shows the performance, and standard deviation shows the risk.

```
ret = df.groupby('Symbol')['Close'].pct_change()
df['Return'] = ret
summary = df.groupby('Symbol')['Return'].agg(['mean', 'std'])
summary.sort_values('mean', ascending=False)     # rank by average return
summary['std'].idxmax()                          # most volatile stock
```

Q6. What is the use of a correlation matrix of daily returns of different stocks?

It shows how strongly the returns of different companies move together. Stocks with a high positive correlation move in the same direction, so holding only such stocks does not reduce risk. Stocks with low or negative correlation are useful for **diversification** of a portfolio, because a loss in one may be offset by a gain in another.

```
returns = df.pivot(index='Date', columns='Symbol', values='Close').pct_change()
returns.corr()
sns.heatmap(returns.corr(), annot=True)
```

Q7. What do the EDA plots (distribution plot, box plot, yearly bar plot) show for the Tesla data?

- **Closing price line plot:** shows an upward trend of the Tesla stock over the period.
- **Distribution plots of Open, High, Low, Close:** show two peaks, which means the prices varied significantly in two different regions; the Volume data is skewed.
- **Box plots:** only Volume contains outliers; Open, High, Low and Close are free from outliers.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:5		
PROBLEM STATEMENT	Stock market analysis using Python Pandas with suitable dataset.	

- **Feature engineering:** day, month and year are extracted from the Date column, and a new column is_quarter_end is created with np.where(df['month'] % 3 == 0, 1, 0), because quarterly results affect the price.
- **Yearly bar plot (groupby('year').mean()):** shows how the average price changed from year to year.

CONCLUSIONS:
Signature and Date:

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:6	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	
PROBLEM STATEMENT	Titanic Dataset Exploratory Data Analysis (EDA) using Seaborn.	
PREREQUISITES	Concept of Exploratory Data Analysis (EDA) Basic understanding of categorical and numerical variables Familiarity with the Titanic dataset structure (survived, sex, age, pclass, etc.)	
OBJECTIVES:	To explore and understand the structure of the Titanic dataset. To visualize survival patterns based on gender, age, and passenger class using Seaborn. To identify relationships and missing data trends through EDA.	
Tools Used	Python 3.x Jupyter Notebook Libraries: seaborn, matplotlib, pandas	

THEORY:

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is an approach used by data scientists to analyze and summarize the main characteristics of a dataset, often using visual methods and summary statistics, before building machine learning models or testing formal hypotheses. Exploratory data analysis (EDA) is used by data scientists to analyze and investigate data sets and summarize their main characteristics, often employing data visualization methods. EDA helps determine how best to manipulate data sources to get the answers you need, making it easier for data scientists to discover patterns, spot anomalies, test a hypothesis, or check assumptions.

EDA is primarily used to see what data can reveal beyond the formal modeling or hypothesis testing task and provides a provides a better understanding of data set variables and the relationships between them. It can also help determine if the statistical techniques you are considering for data analysis are appropriate. Originally developed by American mathematician John Tukey in the 1970s, EDA techniques continue to be a widely used method in the data discovery process today.

Key Goals of EDA:

Understand Structure: Check the number of rows, columns, data types, and missing values.

Detect Patterns & Relationships: Use scatter plots, correlation matrices, and distributions to spot trends.

Identify Anomalies: Find outliers, duplicate records, or data entry errors.

Guide Feature Engineering:

Types of EDA:

- Univariate Analysis: Examines a single variable at a time (e.g., histograms, box plots, or

frequency counts).

- Bivariate Analysis: Compares two variables to look for relationships or correlations (e.g., scatter plots).
- Multivariate Analysis: Investigates interactions among three or more variables (e.g., heatmaps, pair plots, or principal component analysis).

Common Tools:

Python Libraries: Pandas for data manipulation, NumPy for numerical operations, and Matplotlib, Seaborn, or Plotly for data visualization.

Why is EDA important in data science?

The main purpose of EDA is to help look at data before making any assumptions. It can help identify obvious errors, as well as better understand patterns within the data, detect outliers or anomalous events, find interesting relations among the variables.

Data scientists can use exploratory analysis to ensure the results they produce are valid and applicable to any desired business outcomes and goals. EDA also helps stakeholders by confirming they are asking the right questions. EDA can help answer questions about standard deviations, categorical variables, and confidence intervals. Once EDA is complete and insights are drawn, its features can then be used for more sophisticated data analysis or modeling, including machine learning.

Specific statistical functions and techniques you can perform with EDA tools include:

- Clustering and dimension reduction techniques, which help create graphical displays of high-dimensional data containing many variables.
- Univariate visualization of each field in the raw dataset, with summary statistics.
- Bivariate visualizations and summary statistics that allow you to assess the relationship between each variable in the dataset and the target variable you're looking at.
- Multivariate visualizations, for mapping and understanding interactions between different fields in the data.
- K-means clustering, which is a clustering method in unsupervised learning where data points are assigned into K groups, i.e. the number of clusters, based on the distance from each group's centroid. The data points closest to a particular centroid will be clustered under the same category. K-means clustering is commonly used in market segmentation, pattern recognition, and image compression.
- Predictive models, such as linear regression, use statistics and data to predict outcomes.

Types of EDA

There are four primary types of EDA:

Univariate non-graphical, Univariate graphical,Multivariate non-graphical, Multivariate graphical

- Univariate non-graphical

This is simplest form of data analysis, where the data being analyzed consists of just one variable. Since it's a single variable, it doesn't deal with causes or relationships. The main purpose of univariate analysis is to describe the data and find patterns that exist within it.

- Univariate graphical

Non-graphical methods don't provide a full picture of the data. Graphical methods are therefore required. Common types of univariate graphics include:

Stem-and-leaf plots, which show all data values and the shape of the distribution.

Histograms, a bar plot in which each bar represents the frequency (count) or proportion (count/total count) of cases for a range of values.

	Box plots, which graphically depict the five-number summary of minimum, first quartile, median, third quartile, and maximum. • Multivariate nongraphical Multivariate data arises from more than one variable. Multivariate non-graphical EDA techniques generally show the relationship between two or more variables of the data through cross-tabulation or statistics. • **Multivariate graphical** Multivariate data uses graphics to display relationships between two or more sets of data. The most used graphic is a grouped bar plot or bar chart with each group representing one level of one of the variables and each bar within a group representing the levels of the other variable. Other common types of multivariate graphics include: Scatter plot, which is used to plot data points on a horizontal and a vertical axis to show how much one variable is affected by another. Multivariate chart, which is a graphical representation of the relationships between factors and a response. Run chart, which is a line graph of data plotted over time. Bubble chart, which is a data visualization that displays multiple circles (bubbles) in a two-dimensional plot. Heat map, which is a graphical representation of data where values are depicted by color.
REFERENCES: Refer lab manual for the same	

Practice Problem Statements:
1. Using the Titanic dataset, perform Exploratory Data Analysis (EDA) using Pandas and Seaborn. Analyze the distribution of passenger age, gender, passenger class, fare and survival status. Investigate the relationship between survival and passenger characteristics using appropriate Seaborn visualizations. Identify missing values and outliers, generate a correlation heatmap, and summarize at least five significant observations obtained from the analysis.
Important Questions: -
1. What is the purpose of Exploratory Data Analysis (EDA) before model building? 2. What is the difference between a countplot and a histplot? 3. How does the 'hue' parameter enhance Seaborn visualizations? 4. Why does the Titanic dataset have a high number of missing values in the 'deck' column, and how would you handle it? 5. What insights can be drawn from the correlation heatmap regarding survival and passenger class? 6. What is KDE (Kernel Density Estimate), and how does it complement a histogram?

SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE - 411043

Department of Electronics & Computer Engineering

ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**

SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]

EXPT No:6

PROBLEM STATEMENT

Titanic Dataset Exploratory Data Analysis (EDA) using Seaborn.

ALGORITHM

Steps of Exploratory Data Analysis:

1. Import **pandas, numpy, matplotlib, seaborn** and load Titanic-Dataset.csv.
2. Understand the data: `shape` (891, 12), `head()`, `info()`, `describe()`.
3. Find missing values with `isnull().sum()` (Age 177, Cabin 687, Embarked 2) and show them with a heatmap.
4. Univariate analysis: `countplot` for Survived, Sex and Pclass; `histplot(kde=True)` for Age and Fare; boxplot and the IQR rule for outliers.
5. Bivariate analysis: `countplot(hue=...)`, survival rate by Sex and by Pclass, boxplot of Fare by Pclass.
6. Multivariate analysis: correlation heatmap of the numerical columns.
7. Quantify the findings: survival rate, gender gap, class effect.
8. Summarise at least five significant observations.

ERRORS and TEST CASES, USE CASES

Common errors and their remedies

Error / symptom	Cause	Remedy
ValueError: could not convert string to float	`corr()` used on text columns such as Name.	`corr(numeric_only=True)`.
AttributeError / warning: distplot	`distplot` is removed from seaborn.	Use `histplot` or `displot`.
hue not applied	Column name misspelt or wrong dtype.	Check `df.columns`; convert to category if needed.
Statistics distorted by NaN	Age has 177 missing values.	Report the percentage missing; impute (median) or drop as needed.
Heatmap is unreadable	Too many columns or no formatting.	Increase `figsize`; use `annot=True, fmt='.2f'`.
Survival compared using counts	Groups of different size.	Compare the survival rate = mean of Survived.

Test cases

ID	Input / condition	Expected result
T1	`df.shape`	(891, 12)
T2	`df.isnull().sum()`	Age 177, Cabin 687, Embarked 2
T3	`df['Survived'].value_counts()`	549 did not survive, 342 survived
T4	`df['Survived'].mean()`	0.384, i.e. a survival rate of 38.4%
T5	Survival rate by Sex	Female 74.2%, male 18.9%
T6	Correlation of Survived with Pclass	Negative: higher class number gives lower survival
T7	IQR rule on Age	11 outliers (limits −6.69 and 64.81)
T8	(Negative test) `df.corr()` without `numeric_only=True`	ValueError because of text columns

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:6	
PROBLEM STATEMENT	Titanic Dataset Exploratory Data Analysis (EDA) using Seaborn.

Use cases

- First step of every data-science project before building a model.
- Detecting missing values, outliers and wrong entries.
- Understanding which factors influence an outcome (survival, churn, sales).
- Communicating findings to non-technical people through plots.
- Choosing the features to use in the model.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:6	
PROBLEM STATEMENT	Titanic Dataset Exploratory Data Analysis (EDA) using Seaborn.

Libraries: pandas, numpy, matplotlib, seaborn. Dataset: Titanic-Dataset.csv (891 passengers, 12 columns).

PROGRAM AND OUTPUT

Load dataset and basic information

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
pd.set_option("display.width", 85)
pd.set_option("display.max_columns", 20)
sns.set_style("whitegrid")

df = pd.read_csv("Titanic-Dataset.csv")
print("Shape of dataset:", df.shape)
print(df.drop(columns=["Name", "Ticket"]).head())
print()
df.info()
```

Output:

```
Shape of dataset: (891, 12)
   PassengerId  Survived  Pclass     Sex   Age  SibSp  Parch     Fare Cabin  \
0            1         0       3    male  22.0      1      0   7.2500   NaN
1            2         1       1  female  38.0      1      0  71.2833   C85
2            3         1       3  female  26.0      0      0   7.9250   NaN
3            4         1       1  female  35.0      1      0  53.1000  C123
4            5         0       3    male  35.0      0      0   8.0500   NaN

  Embarked
0        S
1        C
2        S
3        S
4        S

<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
```

```
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

Summary statistics and missing values

Code:

```
print("Statistical summary (numerical columns):")
print(df[["Survived", "Pclass", "Age", "SibSp", "Parch", "Fare"]].describe().round(2))

print("\nMissing values:")
miss = df.isnull().sum()
print(miss[miss > 0])
print("\nPercentage of missing values:")
print((miss[miss > 0] / len(df) * 100).round(2))
print("\nDuplicate rows:", df.duplicated().sum())
```

Output:

```
Statistical summary (numerical columns):
       Survived  Pclass     Age   SibSp   Parch    Fare
count    891.00  891.00  714.00  891.00  891.00  891.00
mean       0.38    2.31   29.70    0.52    0.38   32.20
std        0.49    0.84   14.53    1.10    0.81   49.69
min        0.00    1.00    0.42    0.00    0.00    0.00
25%        0.00    2.00   20.12    0.00    0.00    7.91
50%        0.00    3.00   28.00    0.00    0.00   14.45
75%        1.00    3.00   38.00    1.00    0.00   31.00
max        1.00    3.00   80.00    8.00    6.00  512.33

Missing values:
Age         177
Cabin       687
Embarked      2
dtype: int64

Percentage of missing values:
Age         19.87
Cabin       77.10
Embarked     0.22
dtype: float64

Duplicate rows: 0
```

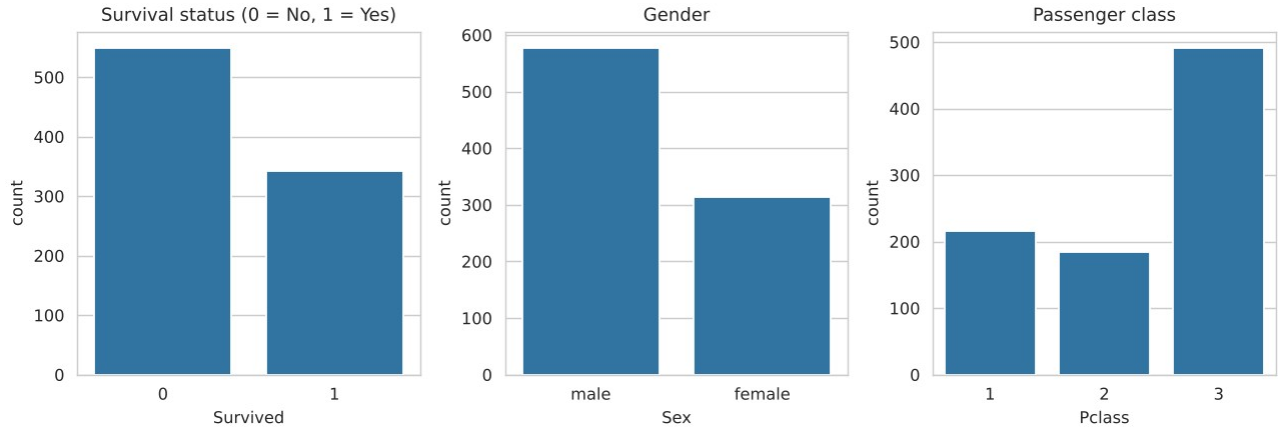
	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 Department of Electronics & Computer Engineering ASSESSMENT YEAR: 2026-2027 | CLASS: BE SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:6	
PROBLEM STATEMENT	Titanic Dataset Exploratory Data Analysis (EDA) using Seaborn.

Distribution of survival, gender and class

Code:

```
fig, ax = plt.subplots(1, 3, figsize=(10, 3.5))
sns.countplot(x="Survived", data=df, ax=ax[0])
ax[0].set_title("Survival status (0 = No, 1 = Yes)")
sns.countplot(x="Sex", data=df, ax=ax[1])
ax[1].set_title("Gender")
sns.countplot(x="Pclass", data=df, ax=ax[2])
ax[2].set_title("Passenger class")
plt.tight_layout()
plt.show()
print(df["Survived"].value_counts())
print("\nSurvival rate: %.1f %%" % (df["Survived"].mean() * 100))
print(df["Sex"].value_counts())
print(df["Pclass"].value_counts().sort_index())
```

Output:




```
Survived
0    549
1    342
Name: count, dtype: int64

Survival rate: 38.4 %
Sex
male      577
female    314
Name: count, dtype: int64
Pclass
1    216
2    184
3    491
Name: count, dtype: int64
```

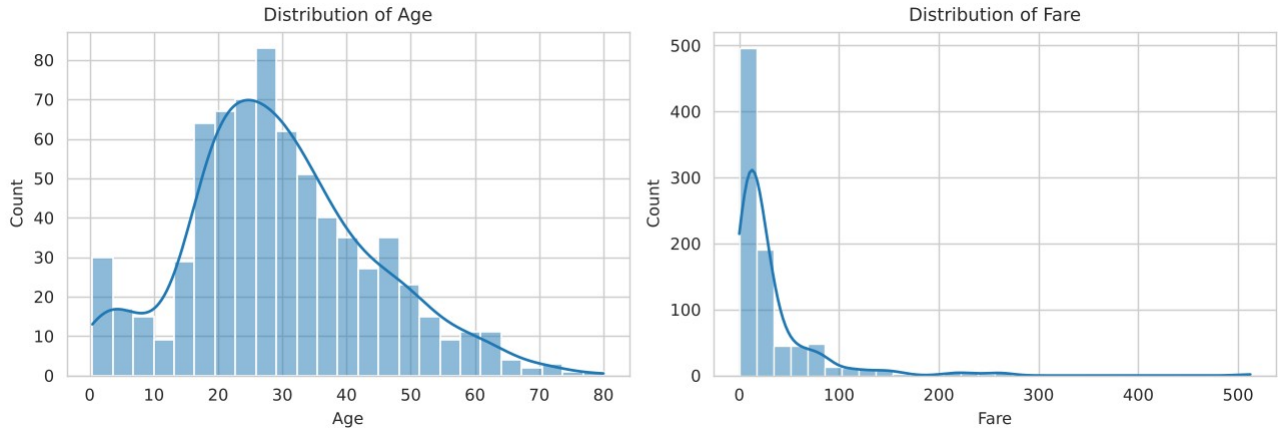
	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:6	
PROBLEM STATEMENT	Titanic Dataset Exploratory Data Analysis (EDA) using Seaborn.

Distribution of age and fare

Code:

```
fig, ax = plt.subplots(1, 2, figsize=(10, 3.5))
sns.histplot(df["Age"].dropna(), bins=25, kde=True, ax=ax[0])
ax[0].set_title("Distribution of Age")
sns.histplot(df["Fare"], bins=30, kde=True, ax=ax[1])
ax[1].set_title("Distribution of Fare")
plt.tight_layout()
plt.show()
print("Skewness of Age :", round(df["Age"].skew(), 3))
print("Skewness of Fare:", round(df["Fare"].skew(), 3))
```

Output:




```
Skewness of Age : 0.389
Skewness of Fare: 4.787
```

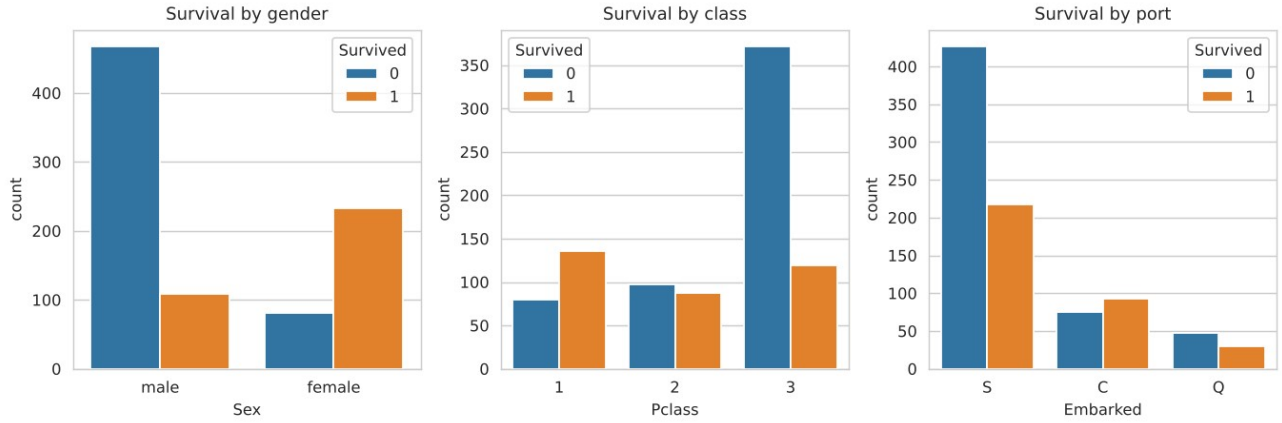
Survival vs gender, class and embarked

Code:

```
fig, ax = plt.subplots(1, 3, figsize=(10.5, 3.6))
sns.countplot(x="Sex", hue="Survived", data=df, ax=ax[0])
ax[0].set_title("Survival by gender")
sns.countplot(x="Pclass", hue="Survived", data=df, ax=ax[1])
ax[1].set_title("Survival by class")
sns.countplot(x="Embarked", hue="Survived", data=df, ax=ax[2])
ax[2].set_title("Survival by port")
plt.tight_layout()
plt.show()
print("Survival rate by gender (%):")
print((df.groupby("Sex")["Survived"].mean() * 100).round(2))
print("\nSurvival rate by passenger class (%):")
print((df.groupby("Pclass")["Survived"].mean() * 100).round(2))
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 Department of Electronics & Computer Engineering ASSESSMENT YEAR: 2026-2027 | CLASS: BE SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:6	
PROBLEM STATEMENT	Titanic Dataset Exploratory Data Analysis (EDA) using Seaborn.

Output:




```
Survival rate by gender (%):
Sex
female    74.20
male      18.89
Name: Survived, dtype: float64

Survival rate by passenger class (%):
Pclass
1    62.96
2    47.28
3    24.24
Name: Survived, dtype: float64
```

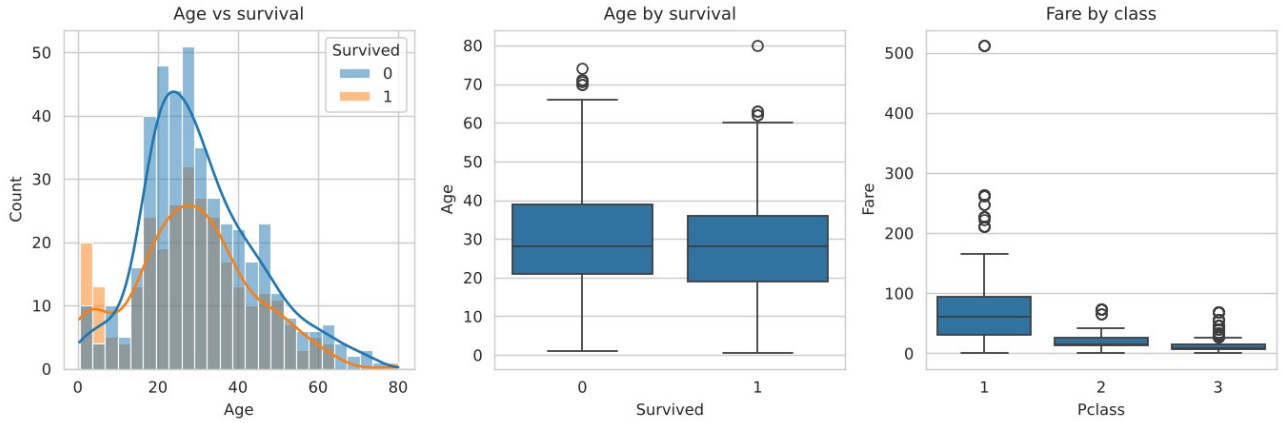
Survival vs age and fare

Code:

```
fig, ax = plt.subplots(1, 3, figsize=(10.5, 3.6))
sns.histplot(data=df, x="Age", hue="Survived", kde=True, bins=25, ax=ax[0])
ax[0].set_title("Age vs survival")
sns.boxplot(x="Survived", y="Age", data=df, ax=ax[1])
ax[1].set_title("Age by survival")
sns.boxplot(x="Pclass", y="Fare", data=df, ax=ax[2])
ax[2].set_title("Fare by class")
plt.tight_layout()
plt.show()
print("Average age of survivors     : %.2f" % df[df["Survived"] == 1]["Age"].mean())
print("Average age of non-survivors : %.2f" % df[df["Survived"] == 0]["Age"].mean())
print("\nAverage fare by class:")
print(df.groupby("Pclass")["Fare"].mean().round(2))
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 **Department of Electronics & Computer Engineering** **ASSESSMENT YEAR: 2026-2027** | **CLASS: BE** **SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]**
EXPT No:6	
PROBLEM STATEMENT	Titanic Dataset Exploratory Data Analysis (EDA) using Seaborn.

Output:




```
Average age of survivors     : 28.34
Average age of non-survivors : 30.63

Average fare by class:
Pclass
1    84.15
2    20.66
3    13.68
Name: Fare, dtype: float64
```

Outliers

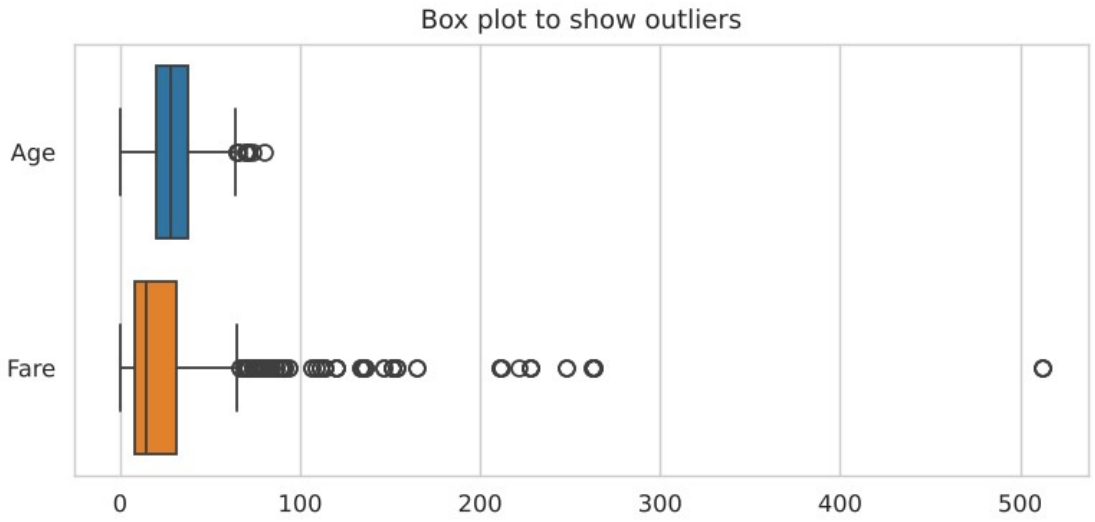
Code:

```
for col in ["Age", "Fare"]:
    q1, q3 = df[col].quantile(0.25), df[col].quantile(0.75)
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n = ((df[col] < low) | (df[col] > high)).sum()
    print("%s: Q1=%.2f Q3=%.2f IQR=%.2f  limits=(%.2f, %.2f)  outliers=%d" % (col, q1, q3,
iqr, low, high, n))

plt.figure(figsize=(7, 3))
sns.boxplot(data=df[["Age", "Fare"]], orient="h")
plt.title("Box plot to show outliers")
plt.show()
```

Output:

```
Age: Q1=20.12 Q3=38.00 IQR=17.88  limits=(-6.69, 64.81)  outliers=11
Fare: Q1=7.91 Q3=31.00 IQR=23.09  limits=(-26.72, 65.63)  outliers=116
```

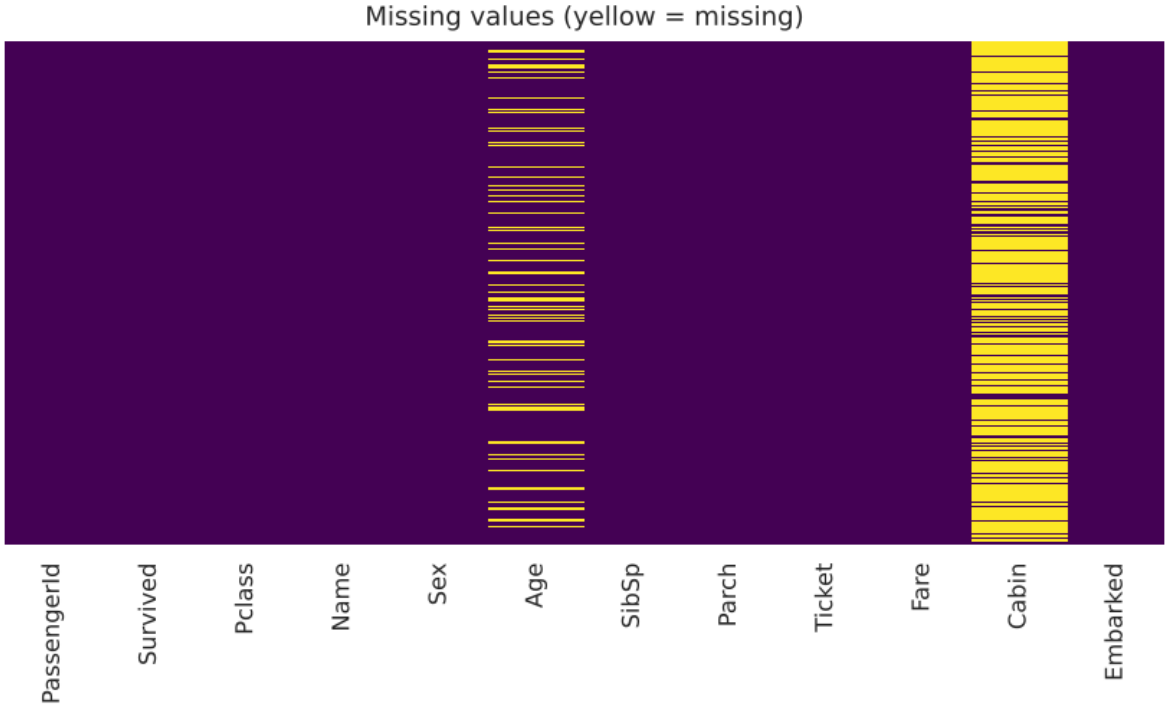


Missing values visualisation

Code:

```
plt.figure(figsize=(8, 3.5))
sns.heatmap(df.isnull(), cbar=False, yticklabels=False, cmap="viridis")
plt.title("Missing values (yellow = missing)")
plt.show()
```

Output:



Correlation heatmap

Code:

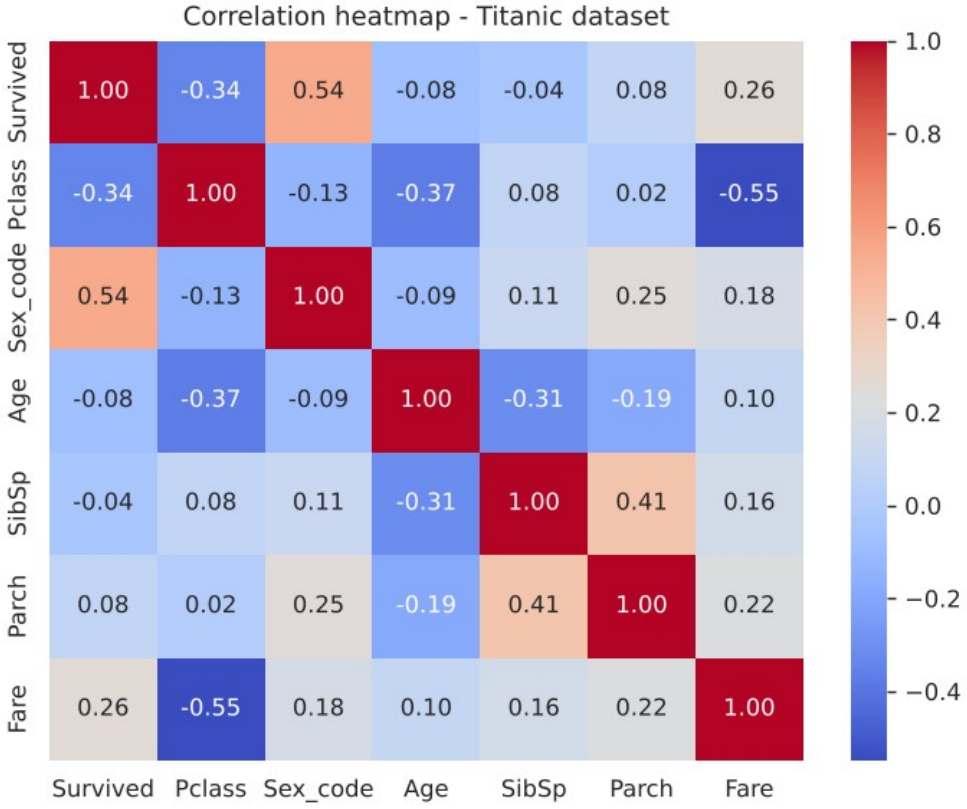
```
d2 = df.copy()
d2["Sex_code"] = d2["Sex"].map({"male": 0, "female": 1})
corr = d2[["Survived", "Pclass", "Sex_code", "Age", "SibSp", "Parch", "Fare"]].corr()
print(corr.round(3))
plt.figure(figsize=(6.5, 5))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation heatmap - Titanic dataset")
plt.show()
```

Output:

```
           Survived  Pclass  Sex_code    Age  SibSp  Parch   Fare
Survived     1.000  -0.338     0.543 -0.077 -0.035  0.082  0.257
Pclass      -0.338   1.000    -0.132 -0.369  0.083  0.018 -0.549
Sex_code     0.543  -0.132     1.000 -0.093  0.115  0.245  0.182
Age         -0.077  -0.369    -0.093  1.000 -0.308 -0.189  0.096
SibSp       -0.035   0.083     0.115 -0.308  1.000  0.415  0.160
Parch        0.082   0.018     0.245 -0.189  0.415  1.000  0.216
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 **Department of Electronics & Computer Engineering** **ASSESSMENT YEAR: 2026-2027** | **CLASS: BE** **SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]**
EXPT No:6	
PROBLEM STATEMENT	Titanic Dataset Exploratory Data Analysis (EDA) using Seaborn.

```
Fare          0.257  -0.549     0.182  0.096  0.160  0.216  1.000
```



Observations

Code:

```
print("Five significant observations:")
f = df[df["Sex"] == "female"]["Survived"].mean() * 100
m = df[df["Sex"] == "male"]["Survived"].mean() * 100
print("1. Gender: %.1f %% of females survived against only %.1f %% of males." % (f, m))
c = df.groupby("Pclass")["Survived"].mean() * 100
print("2. Class: survival was %.1f %% in 1st class, %.1f %% in 2nd and %.1f %% in 3rd
class." % (c[1], c[2], c[3]))
print("3. Age: %.1f %% of Age values are missing; median age is %.0f years." %
(df["Age"].isnull().mean() * 100, df["Age"].median()))
print("4. Fare is highly right-skewed (skew = %.2f) with %d outliers; 1st class paid the
highest fares." % (df["Fare"].skew(), (df["Fare"] > df["Fare"].quantile(0.75) + 1.5 *
(df["Fare"].quantile(0.75) - df["Fare"].quantile(0.25))).sum()))
```

```
print("5. Correlation: Survived vs Pclass = %.2f, Survived vs Fare = %.2f, Survived vs Sex =
%.2f." % (corr.loc["Survived", "Pclass"], corr.loc["Survived", "Fare"], corr.loc["Survived",
"Sex_code"]))
print("   Cabin is missing for %.1f %% of passengers." % (df["Cabin"].isnull().mean() *
100))
```

Output:

```
Five significant observations:
1. Gender: 74.2 % of females survived against only 18.9 % of males.
2. Class: survival was 63.0 % in 1st class, 47.3 % in 2nd and 24.2 % in 3rd class.
3. Age: 19.9 % of Age values are missing; median age is 28 years.
4. Fare is highly right-skewed (skew = 4.79) with 116 outliers; 1st class paid the highest
fares.
5. Correlation: Survived vs Pclass = -0.34, Survived vs Fare = 0.26, Survived vs Sex = 0.54.
   Cabin is missing for 77.1 % of passengers.
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:6		
PROBLEM STATEMENT	Titanic Dataset Exploratory Data Analysis (EDA) using Seaborn.	

IMPORTANT QUESTIONS AND ANSWERS

Q1. What is the purpose of Exploratory Data Analysis (EDA) before model building?

EDA is the process of summarising and visualising a dataset to understand it before applying any model. Its purposes are:

- To understand the structure of the data - number of rows and columns, data types, and missing values.
- To detect outliers, duplicates and data-entry errors so that the data can be cleaned.
- To find patterns and relationships between variables (correlations, trends, distributions).
- To check assumptions of the statistical technique or model that will be used.
- To guide feature selection and feature engineering, and to choose a suitable model.

Without EDA a model may be built on wrong, biased or irrelevant data. EDA was introduced by John Tukey in the 1970s.

Q2. What is the difference between a countplot and a histplot?

countplot()	**histplot()**
For **categorical** variables	For **numerical (continuous)** variables
Counts the occurrences of each category	Divides the values into bins and counts values per bin
One bar per category, with gaps between bars	Bars are adjacent (continuous scale)
sns.countplot(x='Pclass', data=df)	sns.histplot(df['Age'], bins=20, kde=True)

Q3. How does the 'hue' parameter enhance Seaborn visualizations?

hue adds an extra categorical variable to the plot by splitting the data into groups, each shown in a different colour with an automatic legend. In this way a third variable can be shown on a 2-D plot without drawing separate graphs.

```
sns.countplot(x='Pclass', hue='Survived', data=df)   # survival in each class
sns.scatterplot(x='Age', y='Fare', hue='Sex', data=df)
sns.histplot(data=df, x='Age', hue='Survived', kde=True)
```

For example, countplot with hue='Survived' directly shows that survival was much higher in first class than in third class.

Q4. Why does the Titanic dataset have a high number of missing values in the 'deck' column, and how would you handle it?

The deck is derived from the Cabin number. Only about 200 of the 891 passengers have a cabin record, so around **77% of the values are missing**. Cabin details were mostly recorded for first-class passengers, while most second- and third-class passengers had no assigned cabin or their records were lost (many passengers did not survive to report them).

Ways to handle it:

- **Drop the column**, as too much of it is missing to impute reliably (most common choice).
- **Fill with a new category** such as 'Unknown' / 'M' (missing), since missing itself carries information.
- **Create a flag** column has_cabin (1 or 0) and use it in place of the deck.
- Do not use the mean or mode to fill it, because that would add false information.

```
df.isnull().sum()
df['deck'] = df['deck'].cat.add_categories('Unknown').fillna('Unknown')
# or
df = df.drop(columns=['deck'])
```

Q5. What insights can be drawn from the correlation heatmap regarding survival and passenger class?

The heatmap is drawn with sns.heatmap(df.corr(numeric_only=True), annot=True). The main observations are:

- **Survived and Pclass: about -0.34** - a negative correlation. As the class number increases (1st to 3rd) the chance of survival decreases, so first-class passengers survived more.
- **Survived and Fare: about +0.26** - passengers who paid higher fares (first class) were more likely to survive.
- **Pclass and Fare: about -0.55** - a strong negative correlation, since first class tickets cost more.
- **Pclass and Age: about -0.37** - older passengers travelled more in first class.
- **Survived and Age: about -0.08** - almost no linear relationship.
- Sex is not numeric, so it appears in the heatmap only after encoding (female = 1); then it shows the strongest relationship with survival (about 0.54).

Correlation values are approximate for the standard 891-row dataset.

Q6. What is KDE (Kernel Density Estimate), and how does it complement a histogram?

KDE is a smooth curve that estimates the **probability density function** of a continuous variable. A small smooth curve (kernel, usually Gaussian) is placed on every data point and all of them are added up, giving a continuous estimate of the distribution.

A histogram depends on the number and position of bins, so its shape can change when bins are changed, and it appears blocky. The KDE is smooth and independent of bin edges, so it shows the true shape of the distribution (skewness, peaks, bimodality) more clearly. Plotting both together, with the histogram showing actual counts and the KDE showing the underlying shape, gives a better picture. Also, the KDEs of different groups (for example survived and not survived) can be overlaid easily for comparison.

```
sns.histplot(df['Age'].dropna(), kde=True)
sns.kdeplot(df['Age'].dropna())
```

CONCLUSIONS:
Signature and Date:

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:7	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	**Submission date:**
PROBLEM STATEMENT	Build training and testing dataset Titanic predict the probability of a survival of a person based on gender, age and passenger-class.	
PREREQUISITES	Concept of supervised learning and classification Understanding of train-test split and overfitting Basic knowledge of logistic regression Data preprocessing concepts (handling missing values, encoding categorical variables)	
OBJECTIVES:	To preprocess the Titanic dataset by handling missing values and encoding categorical features. To split the dataset into training and testing sets. To build a classification model predicting survival probability based on gender, age, and class.	
Tools Used	Python 3.x Jupyter Notebook Libraries: pandas,seaborn, scikit-learn	
THEORY:		
Machine Learning		
	Machine Learning (ML) is a branch of Artificial Intelligence that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed for every situation. In this experiment, historical Titanic passenger data is used to learn the relationship between passenger characteristics and survival. The problem can be represented as: $X = \{Sex, Age, Pclass\}$ $Y = Survived$ where: • Sex = Gender of passenger • Age = Age of passenger • Pclass = Passenger class • Survived = Target variable The target variable has two possible values: • 0 → Did not survive • 1 → Survived Therefore, this is a **binary classification problem**.	

Logistic Regression:

Logistic Regression is a supervised machine learning algorithm used primarily for **classification problems**.

Although its name contains "regression", it is commonly used for predicting categorical outcomes.

For this experiment:

$$Survived \in \{0, 1\}$$

Therefore, Logistic Regression can be used to predict whether a passenger survived or did not survive.

Logistic Regression Equation

Initially, a linear combination of input features is calculated:

$$z = b_0 + b_1 X_1 + b_2 X_2 + b_3 X_3$$

where:

- b_0 = intercept
- b_1, b_2, b_3 = model coefficients
- X_1 = Sex
- X_2 = Age
- X_3 = Pclass

The result is converted into a probability using the **Sigmoid function**:

$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

Decision Tree Classifier: A **Decision Tree Classifier** is a popular supervised machine learning algorithm used primarily for classification tasks, though it can also be adapted for regression. It breaks down a dataset into smaller and smaller subsets while simultaneously developing an associated flowchart-like tree structure. The model is highly favored in data science because it mimics human decision-making, requires minimal data preprocessing (like scaling), and is exceptionally easy to interpret.

REFERENCES: Refer lab manual for the same

Practice Problem Statements:

1. Build a Machine Learning model using the Titanic dataset to predict the probability of survival of a passenger based on Gender (Sex), Age, and Passenger Class (Pclass). Divide the dataset into training and testing datasets, preprocess the categorical variable, train a Logistic Regression classifier, and evaluate the model using accuracy, confusion matrix, and classification report. Finally, predict the survival probability for a new passenger.
2. Build a Machine Learning model using the Titanic dataset to predict the probability of survival of a passenger based on Gender (Sex), Age, and Passenger Class (Pclass). Divide the dataset into training and testing datasets, preprocess the categorical variable, train a Decision Tree Classifier, and evaluate the model using accuracy, confusion matrix, and classification report. Finally, predict the survival probability for a new passenger.

Important Questions: -

1. Why must categorical variables be encoded before training a machine learning model? 2. What is the difference between predict() and predict_proba()? 3. Why is random_state used in train_test_split()? 4. What is confusion matrix? Explain its significance. 5. What is the difference between accuracy, precision, and recall, and when would precision/recall matter more than accuracy? 6. Describe various classifiers available for above applications and compare the same.

SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE - 411043

Department of Electronics & Computer Engineering

ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**

SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]

EXPT No:7

PROBLEM STATEMENT

Build training and testing dataset Titanic predict the probability of a survival of a person based on gender, age and passenger-class.

ALGORITHM

Steps of building the classification model:

1. Load Titanic-Dataset.csv and keep the columns Survived, Pclass, Sex and Age.
2. Fill the missing Age values with the median: `fillna(df['Age'].median())`.
3. Encode the categorical variable Sex: male = 0, female = 1.
4. Define the features X = [Sex, Age, Pclass] and the target y = Survived.
5. Split into training and testing sets with `train_test_split` (fixed `random_state`).
6. Train `LogisticRegression`: $P(\text{survive}) = 1 / (1 + e^{-(b0 + b1 \cdot Sex + b2 \cdot Age + b3 \cdot Pclass)})$.
7. Predict the class with `predict()` and the probability of survival with `predict_proba()[:, 1]`.
8. Evaluate with `accuracy_score`, `confusion_matrix` and `classification_report`.
9. Predict the survival probability of a new passenger.
10. Repeat with `DecisionTreeClassifier` and compare the results.

ERRORS and TEST CASES, USE CASES

Common errors and their remedies

Error / symptom	Cause	Remedy
ValueError: could not convert string to float: 'male'	Sex was not encoded.	`map({'male': 0, 'female': 1})` or `get_dummies`.
ValueError: Input X contains NaN	Age still has missing values.	Fill them with the median before training.
ValueError: Expected 2D array, got 1D array	`predict()` given a flat list.	Pass `[[1, 25, 2]]` or a DataFrame.
ConvergenceWarning	The solver did not converge on unscaled features.	Increase `max_iter` or scale the features.
Feature-name / order mismatch	Prediction input has a different column order than the training data.	Keep the same names and order as in training.
Misleading accuracy / data leakage	Imbalanced classes, or the test data used for imputation.	Study the confusion matrix; fit imputers only on training data.

Test cases

ID	Input / condition	Expected result
T1	Age after median filling	0 missing values
T2	Encoding of Sex	male → 0, female → 1
T3	Confusion matrix of the decision tree in the practical	[[92, 13], [22, 52]]: sums to 179 test rows; accuracy (92 + 52)/179 = 0.8045
T4	`predict_proba(X_test)`	Every value is between 0 and 1 and each row adds up to 1
T5	Logistic regression coefficients (Sex, Age, Pclass)	+2.4742, −0.0255, −1.0356: being female raises, a higher class number lowers survival
T6	Female, 28, class 1 compared with male, 28, class 3	The first has a much higher survival probability
T7	`predict([[1, 25, 2]])`	A single class label, 0 or 1
T8	(Negative test) training with Sex as text	ValueError is raised

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:7	
PROBLEM STATEMENT	Build training and testing dataset Titanic predict the probability of a survival of a person based on gender, age and passenger-class.

Use cases

- Predicting loan default, customer churn or insurance claims.
- Medical diagnosis: probability that a patient has a disease.
- Spam filtering and fraud detection.
- Estimating risk scores that help to prioritise cases.
- Baseline model before trying more complex classifiers.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:7	
PROBLEM STATEMENT	Build training and testing dataset Titanic predict the probability of a survival of a person based on gender, age and passenger-class.

Libraries: pandas, numpy, seaborn, matplotlib, scikit-learn. Dataset: Titanic-Dataset.csv. Features: Sex, Age, Pclass; target: Survived.

PROGRAM AND OUTPUT

Load dataset and preprocessing

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
pd.set_option("display.width", 85)

df = pd.read_csv("Titanic-Dataset.csv")
data = df[["Survived", "Pclass", "Sex", "Age"]].copy()
print("Shape:", data.shape)
print(data.head())
print("\nMissing values before cleaning:")
print(data.isnull().sum())

data["Age"] = data["Age"].fillna(data["Age"].median())
data["Sex"] = data["Sex"].map({"male": 0, "female": 1})
print("\nMissing values after cleaning:")
print(data.isnull().sum())
print("\nPreprocessed data:")
print(data.head())
```

Output:

```
Shape: (891, 4)
   Survived  Pclass     Sex   Age
0         0       3    male  22.0
1         1       1  female  38.0
2         1       3  female  26.0
3         1       1  female  35.0
4         0       3    male  35.0

Missing values before cleaning:
Survived      0
Pclass        0
Sex           0
Age         177
dtype: int64

Missing values after cleaning:
Survived    0
Pclass      0
Sex         0
Age         0
dtype: int64
```

```
Preprocessed data:
   Survived  Pclass  Sex   Age
0         0       3    0  22.0
1         1       1    1  38.0
2         1       3    1  26.0
3         1       1    1  35.0
4         0       3    0  35.0
```

Train / test split

Code:

```
from sklearn.model_selection import train_test_split
X = data[["Sex", "Age", "Pclass"]]
y = data["Survived"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Training set :", X_train.shape, y_train.shape)
print("Testing set  :", X_test.shape, y_test.shape)
print("\nSurvival ratio in training set:", round(y_train.mean(), 3))
print("Survival ratio in testing set :", round(y_test.mean(), 3))
```

Output:

```
Training set : (712, 3) (712,)
Testing set  : (179, 3) (179,)

Survival ratio in training set: 0.376
Survival ratio in testing set : 0.413
```

Logistic Regression

Code:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

print("Coefficients (Sex, Age, Pclass):", lr.coef_.round(4))
print("Intercept:", lr.intercept_.round(4))
print("\nAccuracy:", round(accuracy_score(y_test, y_pred_lr), 4))
print("\nConfusion matrix:")
print(confusion_matrix(y_test, y_pred_lr))
print("\nClassification report:")
print(classification_report(y_test, y_pred_lr, target_names=["Not survived", "Survived"]))
```

Output:

```
Coefficients (Sex, Age, Pclass): [[ 2.4742 -0.0255 -1.0356]]
Intercept: [1.6358]

Accuracy: 0.8101
```

```
Confusion matrix:
[[91 14]
 [20 54]]

Classification report:
              precision    recall  f1-score   support

Not survived       0.82      0.87      0.84       105
    Survived       0.79      0.73      0.76        74

    accuracy                           0.81       179
   macro avg       0.81      0.80      0.80       179
weighted avg       0.81      0.81      0.81       179
```

LR predict probability for new passenger

Code:

```
new = pd.DataFrame({"Sex": [1], "Age": [28], "Pclass": [1]})
prob = lr.predict_proba(new)
print("Probability [not survived, survived]:", prob.round(4))
print("Predicted class:", lr.predict(new)[0])
print("Survival probability of female, age 28, 1st class: %.2f %%" % (prob[0][1] * 100))

new2 = pd.DataFrame({"Sex": [0], "Age": [40], "Pclass": [3]})
print("Survival probability of male, age 40, 3rd class  : %.2f %%" % (lr.predict_proba(new2)
[0][1] * 100))
```

Output:

```
Probability [not survived, survived]: [[0.0863 0.9137]]
Predicted class: 1
Survival probability of female, age 28, 1st class: 91.37 %
Survival probability of male, age 40, 3rd class  : 7.64 %
```

Decision Tree

Code:

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

dt = DecisionTreeClassifier(max_depth=3, random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, y_pred_dt), 4))
print("\nConfusion matrix:")
print(confusion_matrix(y_test, y_pred_dt))
print("\nClassification report:")
print(classification_report(y_test, y_pred_dt, target_names=["Not survived", "Survived"]))
print("Feature importances:")
for f, i in zip(X.columns, dt.feature_importances_):
    print("  %-7s %.4f" % (f, i))
```

Output:

```
Accuracy: 0.8045

Confusion matrix:
[[92 13]
 [22 52]]

Classification report:
              precision    recall  f1-score   support

Not survived       0.81      0.88      0.84       105
    Survived       0.80      0.70      0.75        74

    accuracy                           0.80       179
   macro avg       0.80      0.79      0.79       179
weighted avg       0.80      0.80      0.80       179

Feature importances:
  Sex     0.6536
  Age     0.0997
  Pclass  0.2467
```

DT tree plot and prediction

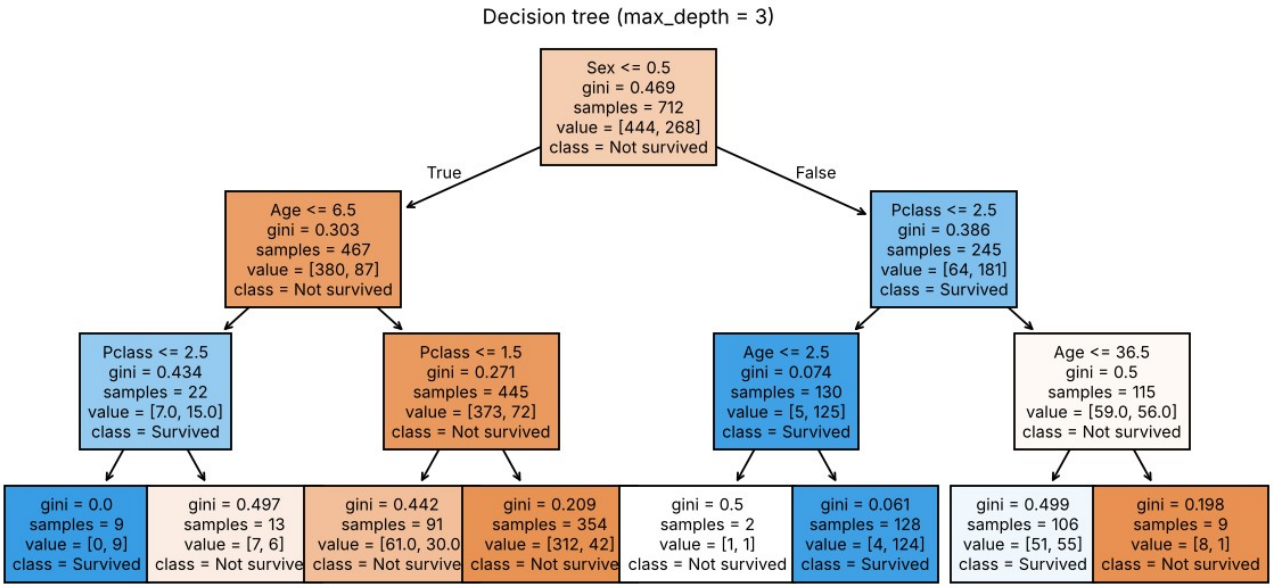
Code:

```
plt.figure(figsize=(11, 5))
plot_tree(dt, feature_names=list(X.columns), class_names=["Not survived", "Survived"],
filled=True, fontsize=8)
plt.title("Decision tree (max_depth = 3)")
plt.show()

prob_dt = dt.predict_proba(new)
print("Probability [not survived, survived]:", prob_dt.round(4))
print("Survival probability of female, age 28, 1st class: %.2f %%" % (prob_dt[0][1] * 100))
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:7	
PROBLEM STATEMENT	Build training and testing dataset Titanic predict the probability of a survival of a person based on gender, age and passenger-class.

Output:




```
Probability [not survived, survived]: [[0.0312 0.9688]]
Survival probability of female, age 28, 1st class: 96.88 %
```

Comparison and confusion matrix plot

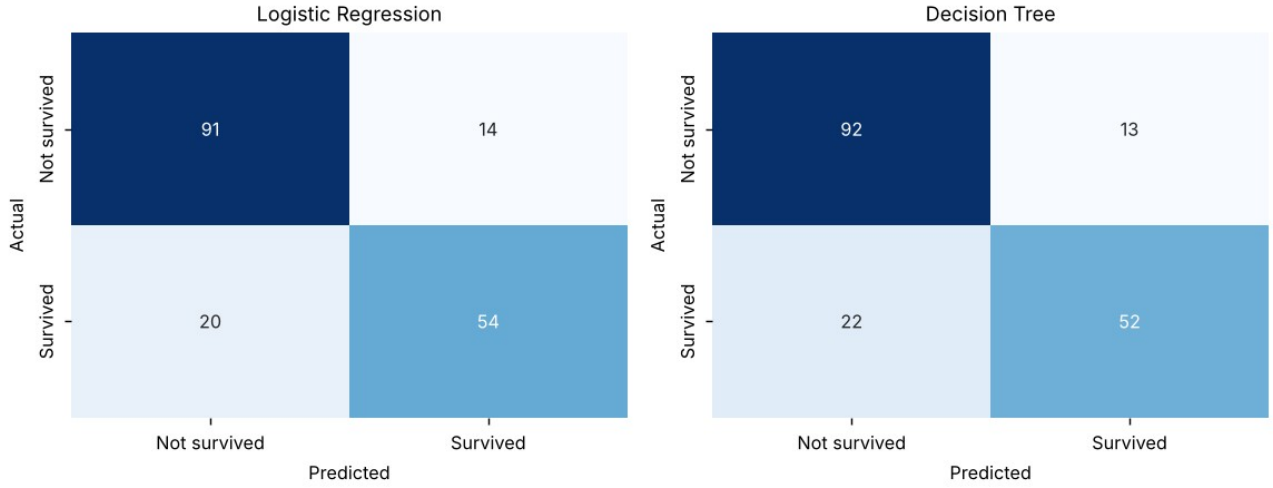
Code:

```
fig, ax = plt.subplots(1, 2, figsize=(9, 3.6))
for a, pred, name in zip(ax, [y_pred_lr, y_pred_dt], ["Logistic Regression", "Decision
Tree"]):
    sns.heatmap(confusion_matrix(y_test, pred), annot=True, fmt="d", cmap="Blues",
cbar=False, ax=a,
                xticklabels=["Not survived", "Survived"], yticklabels=["Not survived",
"Survived"])
    a.set_title(name); a.set_xlabel("Predicted"); a.set_ylabel("Actual")
plt.tight_layout()
plt.show()

print("Model comparison:")
print("Logistic Regression accuracy:", round(accuracy_score(y_test, y_pred_lr), 4))
print("Decision Tree accuracy      :", round(accuracy_score(y_test, y_pred_dt), 4))
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 **Department of Electronics & Computer Engineering** **ASSESSMENT YEAR: 2026-2027** | **CLASS: BE** **SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]**
EXPT No:7	
PROBLEM STATEMENT	Build training and testing dataset Titanic predict the probability of a survival of a person based on gender, age and passenger-class.

Output:




```
Model comparison:
Logistic Regression accuracy: 0.8101
Decision Tree accuracy      : 0.8045
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:7		
PROBLEM STATEMENT	Build training and testing dataset Titanic predict the probability of a survival of a person based on gender, age and passenger-class.	

IMPORTANT QUESTIONS AND ANSWERS

Q1. Why must categorical variables be encoded before training a machine learning model?

Machine learning algorithms work with numbers, because they perform mathematical operations (distances, dot products, gradients). Text values like 'male' and 'female' cannot be used in these calculations, so they are converted into numbers.

- **Label encoding:** male = 0, female = 1 (suitable for two categories or ordered categories).
- **One-hot encoding:** a separate 0/1 column for each category (suitable when there is no order, e.g. Embarked).

```
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
# or
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)
```

Note that label-encoding a non-ordered category with many values can mislead the model into thinking that one category is greater than another, so one-hot encoding is used there.

Q2. What is the difference between predict() and predict_proba()?

predict()	**predict_proba()**
Returns the final **class label** (0 or 1)	Returns the **probability** of each class
Output example: [1]	Output example: [[0.22, 0.78]]
Uses a default threshold of 0.5	Lets us choose our own threshold

Each row of predict_proba() has one column per class, in the order of model.classes_ (column 0 = did not survive, column 1 = survived), and each row adds up to 1. As the problem asks for the probability of survival, predict_proba()[:, 1] is used.

```
model.predict([[1, 25, 2]])           # array([1])
model.predict_proba([[1, 25, 2]])     # array([[0.22, 0.78]])
```

Q3. Why is random_state used in train_test_split()?

train_test_split() shuffles the data randomly before splitting it. random_state sets the seed of this random shuffle, so that the **same split is obtained every time** the code is run. This makes the results reproducible: accuracy and other results do not change from run to run, and different models or different people can be compared on exactly the same train and test data. The value itself (0, 42, etc.) does not matter; only that it is fixed.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0)
```

Q4. What is a confusion matrix? Explain its significance.

A confusion matrix is a table that compares the actual classes with the classes predicted by a classification model. For binary classification it is a 2 x 2 table:

	Predicted: Positive	**Predicted: Negative**
Actual: Positive	TP (True Positive)	FN (False Negative)
Actual: Negative	FP (False Positive)	TN (True Negative)

- **TP:** survived and predicted survived.
- **TN:** did not survive and predicted not survived.
- **FP:** did not survive but predicted survived (Type I error).
- **FN:** survived but predicted not survived (Type II error).

Significance: accuracy alone can be misleading, especially with imbalanced data. The confusion matrix shows exactly what kind of mistakes the model makes, and from it accuracy, precision, recall and F1-score are calculated.

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, y_pred)
```

Q5. What is the difference between accuracy, precision and recall, and when would precision/recall matter more than accuracy?

Metric	**Formula**	**Meaning**
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Fraction of all predictions that are correct
Precision	TP / (TP + FP)	Of all predicted positives, how many are really positive
Recall	TP / (TP + FN)	Of all actual positives, how many were found
F1-score	2 x P x R / (P + R)	Harmonic mean of precision and recall

- **Accuracy is not reliable on imbalanced data.** If 99% of emails are not spam, a model that always says 'not spam' has 99% accuracy but is useless.
- **Recall matters more** when missing a positive case is costly (FN is dangerous), for example cancer or COVID detection, fraud detection.
- **Precision matters more** when a false alarm is costly (FP is dangerous), for example spam filter (a genuine mail going to spam), or recommending a person for a loan.
- When both matter, the F1-score is used.

Q6. Describe various classifiers available for the above application and compare them.

Classifier	How it works	Advantages	Disadvantages
Logistic Regression	Applies a sigmoid function to a weighted sum of features and gives the probability	Simple, fast, gives probabilities, easy to interpret	Assumes a linear boundary; weak on complex data
Decision Tree	Splits data using a tree of if-else rules on features	Easy to understand, no scaling needed, handles non-linear data	Overfits easily; unstable
Random Forest	Many decision trees whose votes are combined	High accuracy, reduces overfitting	Slower, less interpretable
KNN	Classifies by majority class of K nearest points	Simple, no training phase	Slow on large data; needs scaling
SVM	Finds the hyperplane with maximum margin between classes	Good in high dimensions, works with kernels	Slow on large data; hard to tune
Naive Bayes	Uses Bayes' theorem assuming independent features	Very fast, works with small data	Independence assumption is rarely true

For the Titanic problem (3 features, ~891 rows), Logistic Regression and Decision Tree are suitable. Logistic Regression gives a smooth probability of survival and is easy to explain, while the Decision Tree captures rules such as 'female and class 1 or 2' directly but can overfit if its depth is not limited (max_depth).

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:7		
PROBLEM STATEMENT	Build training and testing dataset Titanic predict the probability of a survival of a person based on gender, age and passenger-class.	

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
lr = LogisticRegression().fit(X_train, y_train)
dt = DecisionTreeClassifier(max_depth=3, random_state=0).fit(X_train, y_train)
```

CONCLUSIONS:
Signature and Date:

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:8	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	
PROBLEM STATEMENT	Implementation of Simple and Multiple Linear Regression With scikit-learn in Python	
PREREQUISITES	Understanding of linear regression theory (line of best fit, slope, intercept) Concept of independent vs dependent variables Basic knowledge of model evaluation metrics (R^2 score)	
OBJECTIVES:	To implement simple linear regression to model the relationship between two variables. To implement multiple linear regression using several independent variables. To evaluate model performance using R^2 score and visualization.	
Tools Used	Python 3.x Jupyter Notebook Libraries: numpy,pandas, matplotlib,scikit-learn	
THEORY:		
linear regression		
	Linear regression analysis is used to predict the value of a variable based on the value of another variable. The variable you want to predict is called the dependent variable. The variable you are using to predict the other variable's value is called the independent variable. This form of analysis estimates the coefficients of the linear equation, involving one or more independent variables that best predict the value of the dependent variable. Linear regression fits a straight line or surface that minimizes the discrepancies between predicted and actual output values. There are simple linear regression calculators that use a "least squares" method to discover the best-fit line for a set of paired data. You then estimate the value of X (dependent variable) from Y (independent variable). Generate predictions more easily Why linear regression is important Linear-regression models are relatively simple and provide an easy-to-interpret mathematical formula that can generate predictions. Linear regression can be applied to various areas in business and academic study. You'll find that linear regression is used in everything from biological, behavioral, environmental and social sciences to business. Linear-regression models have become a proven way to scientifically and reliably predict the future. Because linear regression is a	

long-established statistical procedure, the properties of linear-regression models are well understood and can be trained very quickly. It is **A proven way to scientifically and reliably predict the future:** Business and organizational leaders can make better decisions by using linear regression techniques. Organizations collect masses of data, and linear regression helps them use that data to better manage reality, instead of relying on experience and intuition. You can take large amounts of raw data and transform it into actionable information.

You can also use linear regression to provide better insights by uncovering patterns and relationships that your business colleagues might have previously seen and thought they already understood. For example, performing an analysis of sales and purchase data can help you uncover specific purchasing patterns on particular days or at certain times. Insights gathered from regression analysis can help business leaders anticipate times when their company's products will be in high demand.

1. What is Regression?

Regression is a supervised learning technique used to find the relationship between:

- Independent variable(s) / Predictor(s) → X
- Dependent variable / Target → Y

The objective is to predict the value of **Y** using one or more values of **X**.

3. Simple Linear Regression

Simple Linear Regression uses only one independent variable to predict the dependent variable.

$$\hat{Y} = b_0 + b_1 X$$

4. Multiple Linear Regression

Multiple Linear Regression uses two or more independent variables to predict one dependent variable.

$$\hat{Y} = b_0 + b_1 X_1 + b_2 X_2 + \cdots + b_n X_n$$

REFERENCES: Refer lab manual for the same

Practice Problem Statements:

1. Using scikit-learn, implement Simple Linear Regression to predict a car's fuel efficiency (MPG) based on its engine displacement. Evaluate the model using R^2 and RMSE.
2. Using scikit-learn, implement Multiple Linear Regression to predict the selling price of a used car based on its age, kilometers driven, engine capacity, and mileage. Evaluate the model using MAE, RMSE, and R^2 score, and interpret the regression coefficients.

Important Questions: -

1. What is the difference between simple and multiple linear regression?
2. What does the R-squared value indicate about model performance?
3. What is the Ordinary Least Squares (OLS) method?
4. What is multicollinearity, and why is it a problem in multiple regression?
5. How do outliers affect a linear regression model?
6. What are the key assumptions of linear regression?

SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE - 411043

Department of Electronics & Computer Engineering

ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**

SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]

EXPT No:8

PROBLEM STATEMENT

Implementation of Simple and Multiple Linear Regression With scikit-learn in Python

ALGORITHM

Steps of linear regression:

1. Import **numpy, pandas, matplotlib** and the scikit-learn modules; load Housing.csv.
2. Check the shape, missing values and the correlation of price with the numerical columns.
3. Simple regression: independent variable X = `df[['area']]` (2-D), dependent variable y = price.
4. Split into training and testing data with `train_test_split` (fixed `random_state`).
5. Fit `LinearRegression()`: y = b0 + b1x, where Ordinary Least Squares minimises $\Sigma(y - \hat{y})^2$.
6. Predict the test data; read `intercept_` and `coef_`.
7. Evaluate: MAE, MSE, RMSE and $R^2 = 1 - SS_{res}/SS_{tot}$.
8. Plot the scatter of the data with the regression line.
9. Multiple regression: X = area, bedrooms, bathrooms, stories, parking; repeat the split, fit and evaluation; interpret the coefficients.
10. Compare the R^2 of simple and multiple regression.

ERRORS and TEST CASES, USE CASES

Common errors and their remedies

Error / symptom	Cause	Remedy
ValueError: Expected 2D array, got 1D array	X passed as a Series.	Use `df[['area']]` (double brackets) or `reshape(-1, 1)`.
ValueError: could not convert string to float: 'yes'	Text columns (mainroad, guestroom ...) used as features.	Encode them with `pd.get_dummies()`.
ValueError: Input contains NaN	Missing values in X or y.	Impute or drop the rows.
Very low or negative R^2	The model is no better than predicting the mean.	Add useful features; check linearity and outliers.
$R^2 = 1.0$ (suspicious)	Target leakage or a duplicated column.	Remove the leaking feature.
Good train score but poor test score	Evaluation done only on the training data / overfitting.	Always report the metrics on the test set.
Unstable coefficients	Multicollinearity between predictors.	Drop or combine correlated predictors.

Test cases

ID	Input / condition	Expected result
T1	x = [1, 2, 3, 4, 5], y = [3, 5, 7, 9, 11]	Intercept 1, coefficient 2, $R^2 = 1.0$
T2	Predict x = 6 with the model of T1	13
T3	Model that always predicts the mean of y	$R^2 = 0$
T4	Housing.csv: price on area	Positive coefficient (correlation 0.536)
T5	Housing.csv: multiple regression with 5 features	Test R^2 is usually higher than that of the simple model
T6	RMSE and MAE of any model	RMSE ≥ MAE, both in the units of price
T7	(Negative test) X given as a 1-D Series	ValueError (expected 2-D array)
T8	(Negative test) text column in X	ValueError (could not convert string to float)

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | **CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:8	
PROBLEM STATEMENT	Implementation of Simple and Multiple Linear Regression With scikit-learn in Python

Use cases

- House-price and used-car price prediction.
- Sales forecasting from advertising spend.
- Estimating fuel efficiency from engine size.
- Demand and revenue forecasting in business planning.
- Understanding the effect of each factor through the coefficients.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:8	
PROBLEM STATEMENT	Implementation of Simple and Multiple Linear Regression With scikit-learn in Python

Libraries: numpy, pandas, matplotlib, scikit-learn. Dataset: Housing.csv (dependent variable: price).

PROGRAM AND OUTPUT

Load dataset

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
pd.set_option("display.width", 85)
pd.set_option("display.max_columns", 20)

df = pd.read_csv("Housing.csv")
print("Shape:", df.shape)
print(df[["price", "area", "bedrooms", "bathrooms", "stories", "parking"]].head())
print("\nMissing values:", df.isnull().sum().sum())
print("\nCorrelation of price with other numerical columns:")
print(df[["price", "area", "bedrooms", "bathrooms", "stories", "parking"]].corr()
["price"].round(3))
```

Output:

```
Shape: (545, 13)
      price  area  bedrooms  bathrooms  stories  parking
0  13300000  7420         4          2        3        2
1  12250000  8960         4          4        4        3
2  12250000  9960         3          2        2        2
3  12215000  7500         4          2        2        3
4  11410000  7420         4          1        2        2

Missing values: 0

Correlation of price with other numerical columns:
price        1.000
area         0.536
bedrooms     0.366
bathrooms    0.518
stories      0.421
parking      0.384
Name: price, dtype: float64
```

Simple linear regression

Code:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X = df[["area"]]
y = df["price"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

slr = LinearRegression()
slr.fit(X_train, y_train)
y_pred = slr.predict(X_test)

print("Intercept (b0):", round(slr.intercept_, 2))
print("Slope (b1)    :", round(slr.coef_[0], 2))
print("Equation      : price = %.2f + %.2f x area" % (slr.intercept_, slr.coef_[0]))
print("\nMAE  :", round(mean_absolute_error(y_test, y_pred), 2))
print("RMSE :", round(np.sqrt(mean_squared_error(y_test, y_pred)), 2))
print("R2   :", round(r2_score(y_test, y_pred), 4))
```

Output:

```
Intercept (b0): 2512254.26
Slope (b1)    : 425.73
Equation      : price = 2512254.26 + 425.73 x area

MAE  : 1474748.13
RMSE : 1917103.7
R2   : 0.2729
```

Simple regression plot and prediction

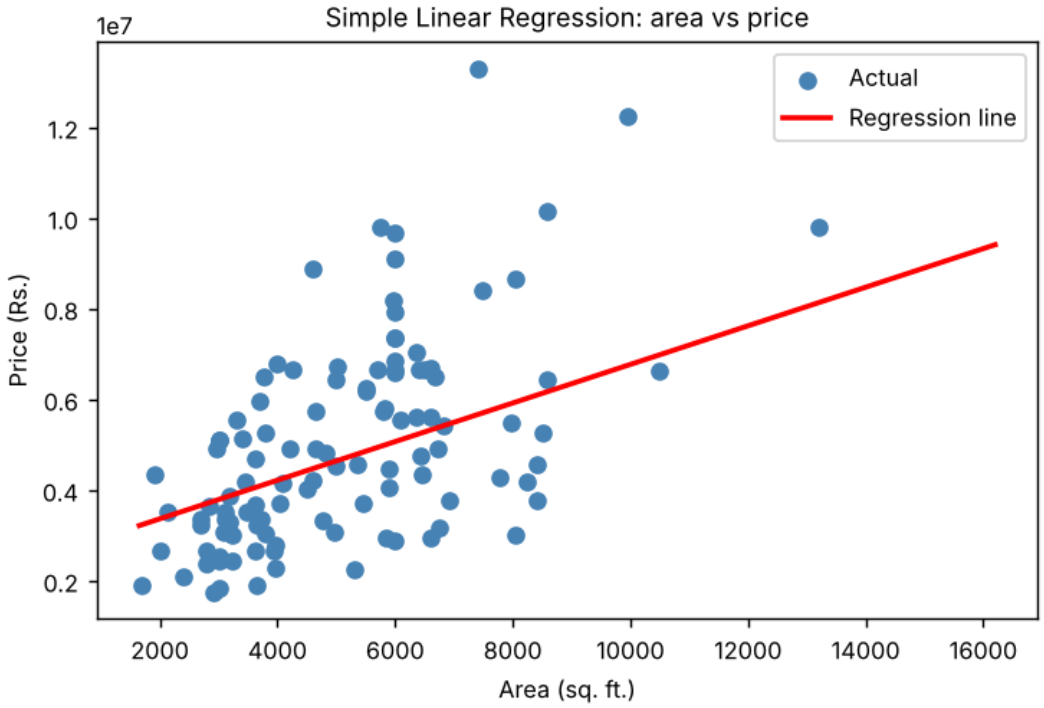
Code:

```
plt.figure(figsize=(6.5, 4))
plt.scatter(X_test, y_test, color="steelblue", label="Actual")
xs = np.linspace(X["area"].min(), X["area"].max(), 100).reshape(-1, 1)
plt.plot(xs, slr.predict(pd.DataFrame(xs, columns=["area"])), color="red", linewidth=2,
label="Regression line")
plt.xlabel("Area (sq. ft.)"); plt.ylabel("Price (Rs.)")
plt.title("Simple Linear Regression: area vs price")
plt.legend()
plt.show()

new = pd.DataFrame({"area": [5000]})
print("Predicted price for a house of 5000 sq. ft.: Rs. %.0f" % slr.predict(new)[0])
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 | CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:8	
PROBLEM STATEMENT	Implementation of Simple and Multiple Linear Regression With scikit-learn in Python

Output:




```
Predicted price for a house of 5000 sq. ft.: Rs. 4640903
```

Multiple linear regression

Code:

```
features = ["area", "bedrooms", "bathrooms", "stories", "parking"]
X = df[features]
y = df["price"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

mlr = LinearRegression()
mlr.fit(X_train, y_train)
y_pred = mlr.predict(X_test)

print("Intercept:", round(mlr.intercept_, 2))
print("\nCoefficients:")
for f, c in zip(features, mlr.coef_):
    print("  %-10s %12.2f" % (f, c))

print("\nMAE  :", round(mean_absolute_error(y_test, y_pred), 2))
print("RMSE :", round(np.sqrt(mean_squared_error(y_test, y_pred)), 2))
```

EXPT No:8	
PROBLEM STATEMENT	Implementation of Simple and Multiple Linear Regression With scikit-learn in Python

```
print("R2   :", round(r2_score(y_test, y_pred), 4))
```

Output:

```
Intercept: 51999.68

Coefficients:
  area              308.87
  bedrooms      151246.75
  bathrooms    1185731.71
  stories       495100.76
  parking       337660.83

MAE  : 1127483.35
RMSE : 1514173.55
R2   : 0.5464
```

Multiple regression plot, interpretation

Code:

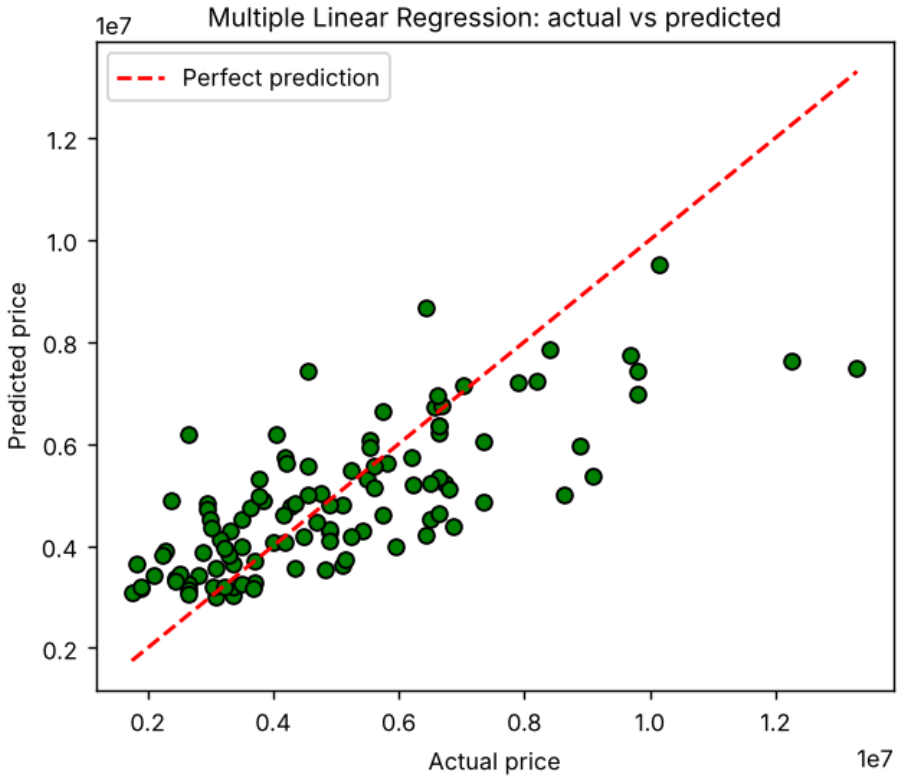
```
plt.figure(figsize=(5.5, 4.5))
plt.scatter(y_test, y_pred, color="green", edgecolor="black")
lim = [y_test.min(), y_test.max()]
plt.plot(lim, lim, "r--", label="Perfect prediction")
plt.xlabel("Actual price"); plt.ylabel("Predicted price")
plt.title("Multiple Linear Regression: actual vs predicted")
plt.legend()
plt.show()

print("Interpretation of coefficients:")
for f, c in zip(features, mlr.coef_):
    print(" - One unit increase in %s changes the price by about Rs. %.0f (other variables
constant)." % (f, c))

new = pd.DataFrame({"area": [6000], "bedrooms": [3], "bathrooms": [2], "stories": [2],
"parking": [1]})
print("\nPredicted price for 6000 sq.ft, 3 bed, 2 bath, 2 stories, 1 parking: Rs. %.0f" %
mlr.predict(new)[0])
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043 **Department of Electronics & Computer Engineering** **ASSESSMENT YEAR: 2026-2027** | **CLASS: BE** **SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]**
EXPT No:8	
PROBLEM STATEMENT	Implementation of Simple and Multiple Linear Regression With scikit-learn in Python

Output:




```
Interpretation of coefficients:
 - One unit increase in area changes the price by about Rs. 309 (other variables constant).
 - One unit increase in bedrooms changes the price by about Rs. 151247 (other variables
constant).
 - One unit increase in bathrooms changes the price by about Rs. 1185732 (other variables
constant).
 - One unit increase in stories changes the price by about Rs. 495101 (other variables
constant).
 - One unit increase in parking changes the price by about Rs. 337661 (other variables
constant).

Predicted price for 6000 sq.ft, 3 bed, 2 bath, 2 stories, 1 parking: Rs. 6058267
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:8		
PROBLEM STATEMENT	Implementation of Simple and Multiple Linear Regression With scikit-learn in Python	

IMPORTANT QUESTIONS AND ANSWERS

Q1. What is the difference between simple and multiple linear regression?

Simple Linear Regression	Multiple Linear Regression
One independent variable (X)	Two or more independent variables (X1, X2, ... Xn)
Equation: y = b0 + b1x	Equation: y = b0 + b1x1 + b2x2 + ... + bnxn
Fits a straight line in 2-D	Fits a plane / hyperplane in higher dimensions
Example: MPG from engine displacement	Example: price of a used car from age, km driven, engine capacity and mileage
Easy to visualise	Cannot be plotted directly; multicollinearity must be checked

Here b0 is the intercept and b1, b2, ... are the slopes (coefficients).

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
print(model.intercept_, model.coef_)
```

Q2. What does the R-squared value indicate about model performance?

R-squared (coefficient of determination) shows the **proportion of the variance of the dependent variable that is explained by the independent variables**.

R^2 = 1 - SSres / SStot, where SSres = sum of $(y - y_pred)^2$ and SStot = sum of $(y - y_mean)^2$.

- R^2 = 1 means a perfect fit; R^2 = 0.85 means 85% of the variation in y is explained by the model.
- R^2 = 0 means the model is no better than always predicting the mean.
- It can be negative on test data if the model is worse than the mean.
- R^2 never decreases when more features are added, so for multiple regression **adjusted R^2** is preferred.
- A high R^2 does not by itself mean that the model is good (it may be overfitting), so residuals should also be checked.

```
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)
```

Q3. What is the Ordinary Least Squares (OLS) method?

OLS is the method used by linear regression to find the best-fit line. It chooses the coefficients (b0, b1, ...) that **minimise the sum of the squared residuals**, where a residual is the difference between the actual value and the predicted value:

Minimise SSE = sum of (yi - y_predicted_i)²

Squaring makes all errors positive and gives larger weight to larger errors. For simple linear regression the solution is:

b1 = sum of (xi - x̄)(yi - ȳ) / sum of (xi - x̄)² b0 = ȳ - b1 x̄

For multiple regression it is given by the normal equation B = (X^T X)^-1 X^T y. scikit-learn's LinearRegression uses OLS.

Q4. What is multicollinearity, and why is it a problem in multiple regression?

Multicollinearity occurs when two or more independent variables are **highly correlated with each other** (for example, age of a car and kilometres driven).

Problems:

- The model cannot separate the individual effect of each correlated variable.
- The coefficients become unstable: small changes in data change them a lot, and they may even get the wrong sign.
- Standard errors increase, so important variables may look insignificant.
- The interpretation of coefficients becomes unreliable, although the overall prediction may still be acceptable.

Detection: correlation matrix/heatmap, and Variance Inflation Factor (VIF) - a VIF above 5 or 10 indicates a problem. Remedies: drop one of the correlated variables, combine them, use PCA, or use Ridge/Lasso regression.

Q5. How do outliers affect a linear regression model?

Linear regression minimises squared errors, so a point far away from the rest has a very large squared error and pulls the regression line towards itself.

- The slope and intercept get distorted and the line no longer represents the majority of the data.

- R^2 decreases and the prediction error (MSE/RMSE) increases.
- A high-leverage point (extreme x value) can have a very large influence on the fit.

Handling: detect them using box plots, scatter plots, z-score or IQR; then remove them if they are errors, cap/transform them (log transform), or use a robust method such as Huber regression.

Q6. What are the key assumptions of linear regression?

1. **Linearity:** the relationship between the independent variables and the dependent variable is linear.
2. **Independence of errors:** the residuals are independent of each other (no autocorrelation).
3. **Homoscedasticity:** the residuals have constant variance at all levels of the predicted value.
4. **Normality of residuals:** the residuals are approximately normally distributed.
5. **No multicollinearity:** the independent variables are not highly correlated with each other (for multiple regression).
6. No significant outliers or influential points.

These can be checked with a residual plot (residual vs predicted), a histogram or Q-Q plot of the residuals, and VIF.

CONCLUSION:
Signature and Date:

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:9	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	
PROBLEM STATEMENT	Implementation of K means Clustering in Python with suitable dataset	
PREREQUISITES	• Concept of unsupervised learning • Understanding of clustering and centroids • Basic knowledge of distance metrics (Euclidean distance)	
OBJECTIVES:	• To understand the concept of unsupervised learning through clustering. • To implement the K-Means algorithm on a real-time dataset. • To visualize and interpret the formed clusters and centroids.	
Tools Used	• Python 3.x • Jupyter Notebook • Libraries: numpy, matplotlib, scikit-learn	

THEORY:

K Means Clustering

K-Means Clustering groups similar data points into clusters without needing labeled data. It is used to uncover hidden patterns when the goal is to organize data based on similarity.

- Helps identify natural groupings in unlabeled datasets
- Works by grouping points based on distance to cluster centers
- Commonly used in customer segmentation, image compression and pattern discovery
- Useful when you need structure from raw, unorganized data

It is one of the most popular clustering methods used in machine learning. Unlike supervised learning, the training data that this algorithm uses is unlabeled, meaning that data points do not have a defined classification structure.

While various types of clustering algorithms exist, including exclusive, overlapping, hierarchical and probabilistic, the k-means clustering algorithm is an example of an exclusive or "hard" clustering method. This form of grouping stipulates that a data point can exist in just one cluster. This type of cluster analysis is commonly used in data science for market segmentation, document clustering, image segmentation and image compression. The k-means algorithm is a widely used method in cluster analysis because it is efficient, effective and simple.

K-means is an iterative, centroid-based clustering algorithm that partitions a dataset into similar groups based on the distance between their centroids. The centroid, or cluster center, is either the mean or median of all the points within the cluster depending on the characteristics of the data.

▪ How does k-means clustering work?

K-means clustering is an iterative process to minimize the sum of distances between the data points and their cluster centroids. The k-means clustering algorithm operates by categorizing data points into clusters by using a mathematical distance measure, usually euclidean, from the cluster center. The objective is to minimize the sum of distances between data points and their assigned clusters. Data points that are nearest to a centroid are grouped together within the same category. A higher k value, or the number of clusters, signifies smaller clusters with greater detail, while a lower k value results in larger clusters with less detail.

Initialize k

The conventional k-means clustering algorithm requires a few steps. The first step is to initialize k centroids where k is equal to the number of clusters chosen for a specific dataset. This approach uses either random selection or initial centroid sampling methods.

Assign centroids

The next step includes a two-step iterative process based on the expectation maximization machine learning algorithm. The expectation step assigns each data point to its closest centroid based on distance (again, usually euclidean). The maximization step computes the mean of all the points for each cluster and reassigns the cluster center, or centroid. This process repeats until the centroid positions have reached convergence or the maximum number of iterations have been reached.

K-means clustering is simple yet sensitive to initial conditions and outliers. It is important to optimize the centroid initialization and the number of clusters k, to achieve the most meaningful clusters. There are several ways to evaluate and optimize clustering components of the algorithm by using evaluation metrics and initial centroid sampling methods.

▪ Cluster evaluation metrics

Quality clusters contain at least two properties:

- All data points within a cluster should be similar.
- Clusters should be distinct from each other.

These properties are achieved by minimizing the intracluster distance and maximizing the intercluster distance of all data points in a dataset. In other words, the more compact and isolated a cluster is from other clusters, the better. The goal of the k-means clustering algorithm is to minimize the sum of squared errors (SSE). Computing the SSE of the squared euclidean distance of each point to its closest centroid evaluates the quality of the cluster assignments by measuring the total variation within each cluster.

Cluster evaluation metrics check the quality and provide different perspectives to analyze clustering results. This helps in selecting the optimal number of clusters and centroid initialization. The following evaluation metrics are common ways to measure the intra and intercluster distances in clustering.

Inertia

The k-means clustering algorithm aims to choose centroids, or cluster centers, that minimize inertia, an evaluation metric that measures how well a dataset has been clustered based on distance metrics. Inertia is calculated by measuring the distance between a datapoint and its centroid, squaring the distance and summing those squares for each data point in the cluster. The sum or inertial value is the intracluster distance. The lower the sum the better because it means that the datapoints within the cluster are compact or more similar.

The Dunn index

The second property is measured with the Dunn index. The Dunn index represents the relationship between the minimum intercluster distance and the maximum intracluster distance. Clusters with a high intercluster distance indicate better quality because it means that the clusters are as different from each other as possible.

▪ Optimizing k-means performance

Optimization is important when using k-means to achieve the best clustering results. The k-means algorithm is nondeterministic due to its random initialization step. This method implies that if the algorithm is performed twice on identical data, the cluster assignments might differ. To achieve optimal clustering results, properly selecting the initial centroids and the optimum number of clusters improves the accuracy and the speed of the k-means algorithm.

Initializing the cluster centroids

Each cluster is represented by a centroid, a data point that represents the cluster center. K-means groups together similar data points into clusters by minimizing the distance between data points in a cluster with their centroid or k mean value. The primary goal of the k-means algorithm is to minimize the total distances between points and their assigned cluster centroid. The algorithm operates iteratively, and initial partition selection can greatly impact the resulting clusters.

Random initialization risks yielding inconsistent results. Centroid initialization methods exist to mitigate these risks. A study by NUS Computing explains and compares methods such as the popular k-means++ to random initialization.

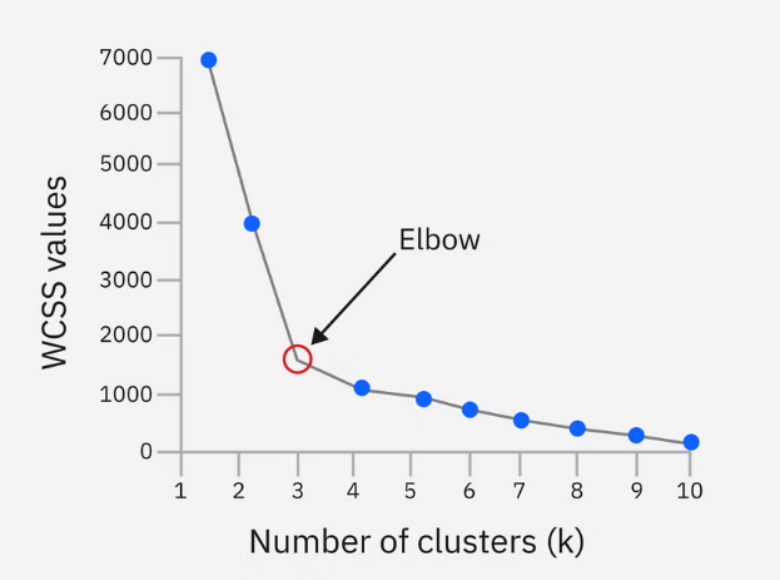
▪ **Choosing the optimal number of clusters**

Ideally, the k-means algorithm iterates until the optimal number of clusters are reached. The maximum number of iterations are met once the centroids have achieved convergence.

The elbow method

One method to achieve the optimal number of clusters is the elbow method. The elbow method is a graphical method for finding the optimum number of clusters within a k-means clustering algorithm. It measures the euclidean distance between each data point and its cluster center and chooses the number of clusters based on where change in "within cluster sum of squares" (WCSS) levels off. This value represents the total variance within each cluster that gets plotted against the number of clusters.

The first step of the elbow method is to calculate the WCSS for each cluster (k). Then, the WCSS value is plotted along the y-axis and the number of clusters is plotted on the x-axis. As the number of clusters increases, the plot points should form a consistent pattern. From this pattern, results a range for the optimum number of clusters. When deciding on the number of clusters, consider the computational costs. The higher the number of clusters, the more processing power is needed especially with large datasets.



Applications:

Customer segmentation: The practice of dividing a company's customers into groups based on common characteristics that reflect similarity. This strategy allows companies to target specific clusters or groups of customers for specific advertising campaigns.

Document classification: A procedure for allocating various classes or categories to documents. This method is used by many organizations to moderate content.

Image segmentation: A computer vision technique that divides a digital image into distinct sets of pixels. This research dives into how k-means models are used to help identify boundaries in medical images.

Recommendation engines: Applications all over the web use recommendation engines. Principal component analysis and k-means clustering techniques are used to build product recommendation for e-commerce businesses.

ALGORITHM

Steps of K-Means Clustering:

1. Import **numpy, pandas, matplotlib, seaborn** and **scikit-learn**; load Mall_Customers.csv.
2. Exploratory analysis: `shape` (200, 5), `head()`, `describe()`, `isnull().sum()` and the distributions of Age, Annual Income and Spending Score.
3. Select the features X = [Annual Income (k$), Spending Score (1-100)].
4. Scale the features with `StandardScaler` if they have different units or ranges.
5. Elbow method: for k = 1 to 10 run `KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)`, store the WCSS (inertia) = $\Sigma\|x - \mu_c\|^2$, plot k against WCSS and choose k at the elbow; confirm with the silhouette score.
6. Initialise k centroids (k-means++).
7. Assignment step: assign each point to the nearest centroid using Euclidean distance.
8. Update step: recompute every centroid as the mean of the points assigned to it.
9. Repeat the assignment and update steps until the centroids stop moving or `max_iter` is reached.
10. Add the cluster labels to the DataFrame and count the customers in each cluster.
11. Plot the clusters in different colours and mark the centroids.
12. Profile each cluster (average age, income, spending score) and interpret it for targeted marketing.

ERRORS and TEST CASES, USE CASES

Common errors and their remedies

Error / symptom	Cause	Remedy
ValueError: could not convert string to float: 'Male'	The text column Genre is included in X.	Use only numerical columns or encode the category.
ValueError: n_samples should be ≥ n_clusters	k is larger than the number of samples.	Choose k ≤ number of samples.
Different clusters on every run	Random centroid initialisation.	Use `init='k-means++'`, `n_init=10` and a fixed `random_state`.
One feature dominates the clusters	Features are on very different scales.	Standardise with `StandardScaler`.
No clear elbow	The WCSS curve decreases smoothly.	Use the silhouette score and business knowledge.
Misreading the cluster numbers	Labels 0 to 4 are arbitrary.	Interpret the clusters through their centroids, not their numbers.

Test cases

ID	Input / condition	Expected result
T1	Mall_Customers.csv loaded	Shape (200, 5); no missing values
T2	WCSS for k = 1 to 10	Decreases continuously from about 269981 (k = 1) to about 20072 (k = 10)
T3	Elbow curve and silhouette score	Elbow at k = 5; highest silhouette score 0.554 at k = 5
T4	KMeans k = 5, random_state = 42	Cluster sizes 81, 39, 22, 35 and 23 (total 200)
T5	Centroids (income, score)	(55.3, 49.5), (86.5, 82.1), (25.7, 79.4), (88.2, 17.1), (26.3, 20.9)
T6	`predict([[70, 80]])` (high income, high score)	Cluster 1 (high income, high spending)
T7	`predict([[20, 10]])` (low income, low score)	Cluster 4 (low income, low spending)
T8	Run twice with the same `random_state`	Identical labels both times
T9	(Negative test) include Genre (text) in X	ValueError

	Use cases • Customer segmentation for targeted marketing and offers. • Image compression and segmentation (e.g. satellite land-cover mapping). • Document and news clustering. • Detecting unusual customers or transactions (points far from every centroid). • Grouping products or users for recommendation engines.

REFERENCES: Refer lab manual for the same

Practice Problem Statements:

1. Implement K-Means clustering in Python to segment mall customers using Annual Income and Spending Score. Load the Mall Customers dataset, perform exploratory analysis, determine the optimal number of clusters using the Elbow Method, apply K-Means, visualize the clusters with centroids, and interpret the customer groups for targeted marketing. (Use https://github.com/SarahShafqat/Kaggle-Datasets/blob/master/Mall_Customers.csv or https://archive.ics.uci.edu/dataset/292/wholesale+customers)

Important Questions: -

1. What is the difference between supervised and unsupervised learning?
2. How does the K-Means algorithm decide the initial centroids, and why is this important?
3. What is the Elbow Method, and how is it used to choose the value of K?
4. What is inertia in the context of K-Means clustering?
5. What are the limitations of K-Means clustering (e.g., sensitivity to outliers, need to predefine K)?
6. Why is feature scaling important before applying K-Means?
7. Why is K-Means suitable for unsupervised image segmentation?
8. Why do we reshape an image into a two-dimensional array?
9. What does each row of the pixels array represent?
10. What is the role of the cluster centroid?
11. What happens when we change (K=4) to (K=6)?
12. Does K-Means automatically identify water, vegetation, and buildings?
13. How can satellite bands such as Near-Infrared (NIR) improve land-cover segmentation?

EXPT No:9

PROBLEM STATEMENT	Implementation of K means Clustering in Python with suitable dataset

Programs are written in Python using Pandas, NumPy, Matplotlib, Seaborn and scikit-learn and executed in Jupyter Notebook. Dataset: Mall_Customers.csv (200 customers; features used: Annual Income and Spending Score).

PROGRAM AND OUTPUT

Import libraries and load dataset

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
pd.set_option("display.width", 85)
sns.set_style("whitegrid")

df = pd.read_csv("Mall_Customers.csv")
print("Shape:", df.shape)
print(df.head())
print("\nMissing values:", df.isnull().sum().sum())
```

Output:

```
Shape: (200, 5)
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Missing values: 0
```

Exploratory data analysis

Code:

```
print(df.describe().round(2))
print("\nGenre counts:")
print(df["Genre"].value_counts())

fig, ax = plt.subplots(1, 3, figsize=(10, 3.2))
for a, c in zip(ax, ["Age", "Annual Income (k$)", "Spending Score (1-100)"]):
    sns.histplot(df[c], bins=15, kde=True, ax=a, color="#4c72b0")
    a.set_title(c, fontsize=9)
plt.tight_layout(); plt.show()
```

Output:

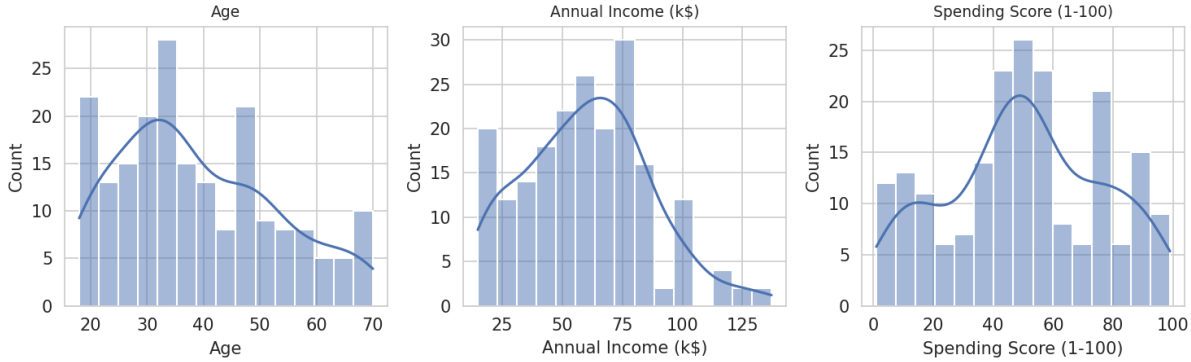
```
       CustomerID     Age  Annual Income (k$)  Spending Score (1-100)
count      200.00  200.00              200.00                  200.00
mean       100.50   38.85               60.56                   50.20
std         57.88   13.97               26.26                   25.82
min          1.00   18.00               15.00                    1.00
25%         50.75   28.75               41.50                   34.75
50%        100.50   36.00               61.50                   50.00
75%        150.25   49.00               78.00                   73.00
max        200.00   70.00              137.00                   99.00

Genre counts:
```

EXPT No:9

PROBLEM STATEMENT	Implementation of K means Clustering in Python with suitable dataset

```
Genre
Female    112
Male       88
Name: count, dtype: int64
```



Observation: there are 200 customers (112 female, 88 male) and no missing values. Annual income ranges from 15 to 137 k$ (mean 60.56) and the spending score from 1 to 99 (mean 50.20). The ID column carries no information and is not used.

Elbow method and silhouette score

Code:

```
X = df[["Annual Income (k$)", "Spending Score (1-100)"]].values

wcss, sil = [], []
for k in range(1, 11):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42).fit(X)
    wcss.append(km.inertia_)
    if k > 1:
        sil.append(silhouette_score(X, km.labels_))

print("k   WCSS         Silhouette")
for k in range(1, 11):
    score = "-" if k == 1 else "%.3f" % sil[k - 2]
    print("%-3d %-12.2f %s" % (k, wcss[k - 1], score))

fig, ax = plt.subplots(1, 2, figsize=(10, 3.6))
ax[0].plot(range(1, 11), wcss, "o-"); ax[0].axvline(5, ls="--", c="red")
ax[0].set_title("Elbow Method"); ax[0].set_xlabel("Number of clusters (k)")
ax[0].set_ylabel("WCSS")
ax[1].plot(range(2, 11), sil, "s-", c="green")
ax[1].set_title("Silhouette score"); ax[1].set_xlabel("Number of clusters (k)")
plt.tight_layout(); plt.show()
```

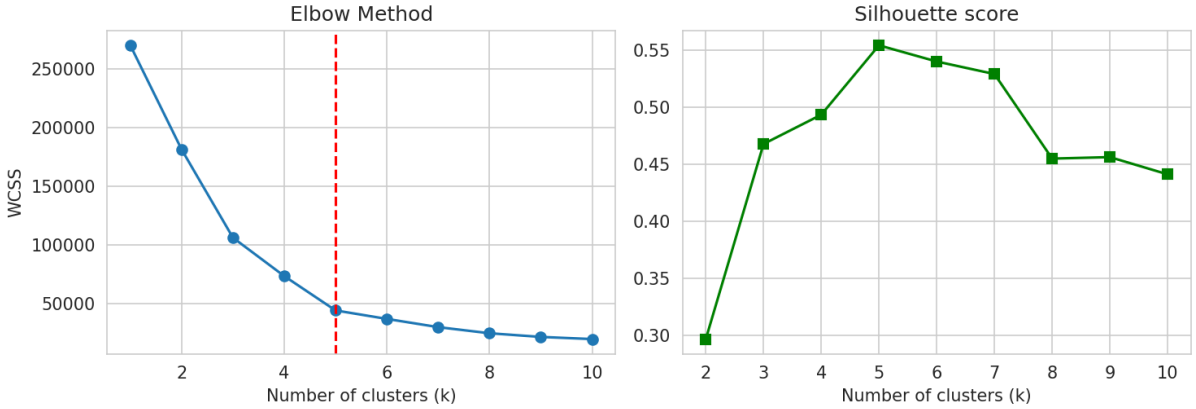
Output:

```
k   WCSS         Silhouette
1   269981.28    -
2   181363.60    0.297
3   106348.37    0.468
4   73679.79     0.493
5   44448.46     0.554
6   37233.81     0.540
7   30241.34     0.529
8   25036.42     0.455
```

EXPT No:9

PROBLEM STATEMENT	Implementation of K means Clustering in Python with suitable dataset

```
9   21916.79     0.456
10  20072.07     0.441
```



Interpretation: the WCSS falls steeply up to k = 5 (106348 at k = 3, 73680 at k = 4, 44448 at k = 5) and then flattens (37234 at k = 6). The bend (elbow) is therefore at **k = 5**. The silhouette score is also highest at k = 5 (0.554), which confirms the choice.

Apply K-Means with k = 5

Code:

```
km = KMeans(n_clusters=5, init="k-means++", n_init=10, random_state=42)
df["Cluster"] = km.fit_predict(X)

print("Centroids (Income, Score):")
print(np.round(km.cluster_centers_, 2))
print("Inertia:", round(km.inertia_, 2), " Iterations:", km.n_iter_,
      " Silhouette: %.3f" % silhouette_score(X, df["Cluster"]))
print("\nCustomers per cluster:")
print(df["Cluster"].value_counts().sort_index())
```

Output:

```
Centroids (Income, Score):
[[55.3  49.52]
 [86.54 82.13]
 [25.73 79.36]
 [88.2  17.11]
 [26.3  20.91]]
Inertia: 44448.46  Iterations: 4  Silhouette: 0.554

Customers per cluster:
Cluster
0    81
1    39
2    22
3    35
4    23
Name: count, dtype: int64
```

Visualise the clusters with centroids

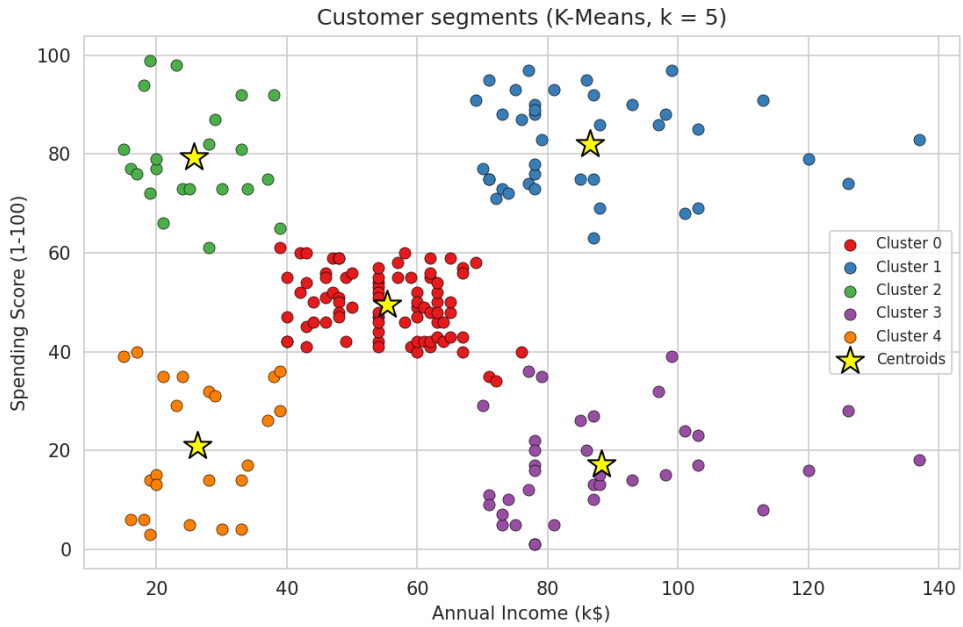
Code:

```
colors = ["#e41a1c", "#377eb8", "#4daf4a", "#984ea3", "#ff7f00"]
```

EXPT No:9

PROBLEM STATEMENT	Implementation of K means Clustering in Python with suitable dataset

```
plt.figure(figsize=(7.5, 5))
for c in range(5):
    m = df["Cluster"] == c
    plt.scatter(X[m, 0], X[m, 1], s=40, c=colors[c], label="Cluster %d" % c,
                edgecolor="k", linewidth=0.3)
plt.scatter(km.cluster_centers_[:, 0], km.cluster_centers_[:, 1], s=250, c="yellow",
            marker="*", edgecolor="k", label="Centroids")
plt.xlabel("Annual Income (k$)"); plt.ylabel("Spending Score (1-100)")
plt.title("Customer segments (K-Means, k = 5)"); plt.legend(fontsize=8)
plt.tight_layout(); plt.show()
```



Cluster profile for targeted marketing

Code:

```
summary = df.groupby("Cluster").agg(Customers=("CustomerID", "count"), Avg_Age=("Age", "mean"),
                                    Avg_Income=("Annual Income (k$)", "mean"),
                                    Avg_Score=("Spending Score (1-100)", "mean")).round(1)
print(summary)
```

Output:

```
         Customers  Avg_Age  Avg_Income  Avg_Score
Cluster
0               81     42.7        55.3       49.5
1               39     32.7        86.5       82.1
2               22     25.3        25.7       79.4
3               35     41.1        88.2       17.1
4               23     45.2        26.3       20.9
```

EXPT No:9

PROBLEM STATEMENT	Implementation of K means Clustering in Python with suitable dataset

Cluster	Income (k$) / Score (customers)	Segment	Marketing strategy
0	55.3 / 49.5 (81)	Average customers	Standard offers, loyalty points, seasonal discounts to keep them engaged.
1	86.5 / 82.1 (39)	Premium target group (high income, high spending)	Priority customers: premium and new-arrival products, VIP membership, early access.
2	25.7 / 79.4 (22)	Young enthusiastic spenders (low income, high spending)	Trendy low-price products, coupons, festival and EMI offers, social-media campaigns.
3	88.2 / 17.1 (35)	Careful high-income customers (high income, low spending)	Potential for growth: personalised campaigns, premium quality assurance, product recommendations.
4	26.3 / 20.9 (23)	Budget-conscious customers (low income, low spending)	Essential and low-cost items, clearance sales; avoid costly campaigns.

Interpretation: the five groups differ clearly in income and spending. The premium group (cluster 1) and the young enthusiastic spenders (cluster 2) respond best to promotions, while the high-income low-spending group (cluster 3) is the biggest opportunity to increase sales. Cluster numbers come from `random_state=42`.

Effect of scaling and prediction for new customers

Code:

```
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import adjusted_rand_score

Xs = StandardScaler().fit_transform(X)
km_scaled = KMeans(n_clusters=5, n_init=10, random_state=42).fit(Xs)
ari = adjusted_rand_score(df["Cluster"], km_scaled.labels_)
print("Agreement between raw and scaled clustering (ARI): %.3f" % ari)

print("New customer (income 70k$, score 80) -> cluster", km.predict([[70, 80]])[0])
print("New customer (income 20k$, score 10) -> cluster", km.predict([[20, 10]])[0])
```

Output:

```
Agreement between raw and scaled clustering (ARI): 1.000
New customer (income 70k$, score 80) -> cluster 1
New customer (income 20k$, score 10) -> cluster 4
```

Note: both features here are on similar scales (15–137 and 1–99), so scaling gives exactly the same clusters (ARI = 1.0). Scaling becomes essential when the features have very different ranges (for example Age and income in rupees). Cluster numbers are arbitrary labels and may differ on another machine.

IMPORTANT QUESTIONS AND ANSWERS

Q1. What is the difference between supervised and unsupervised learning?

Supervised learning	Unsupervised learning
Data is labeled (inputs with known outputs)	Data is unlabeled (only inputs)
Learns a mapping from inputs to the known output	Finds hidden structure or groups in the data
Tasks: classification, regression	Tasks: clustering, dimensionality reduction, association
Examples: logistic regression, decision tree	Examples: K-Means, hierarchical clustering, PCA
Evaluated against the true labels (accuracy, R^2)	No true labels; evaluated with inertia, silhouette score

In this practical no label such as 'customer type' exists, so K-Means discovers the groups by itself.

Q2. How does the K-Means algorithm decide the initial centroids, and why is this important?

In the simplest version k data points are chosen at random as the initial centroids. In **k-means++** (the default in scikit-learn) the first centroid is chosen randomly and every next centroid is chosen with a probability proportional to its squared distance from the nearest centroid already chosen, so the initial centroids are spread far apart.

This matters because K-Means only finds a *local* minimum of the inertia. Poor starting points (for example two centroids in the same group) give poor clusters, slower convergence and different results on every run. k-means++, `n_init=10` (the best of 10 runs is kept) and a fixed `random_state` make the result good and reproducible.

Q3. What is the Elbow Method, and how is it used to choose the value of K?

The Elbow Method runs K-Means for k = 1, 2, 3 ... and plots k against the WCSS (inertia). The WCSS always decreases when k increases, but after the correct number of clusters the decrease becomes small. The point where the curve bends like an elbow is taken as k.

In this practical the WCSS drops from 269981 (k = 1) to 44448 (k = 5) and then decreases slowly (37234 at k = 6), so k = 5. When the elbow is not clear, the silhouette score is used as a second check (highest at k = 5 with 0.554).

Q4. What is inertia in the context of K-Means clustering?

Inertia (WCSS or SSE) is the sum of the squared Euclidean distances between every data point and the centroid of its own cluster:

```
Inertia = sum over all clusters c, sum over all points x in c of  || x - mu_c ||^2
```

A small inertia means compact clusters. K-Means tries to minimise it; it is available as `km.inertia_` (44448.46 for k = 5 here). Inertia cannot be used alone to choose k because it keeps falling as k grows (it is 0 when k equals the number of points).

Q5. What are the limitations of K-Means clustering?

- The number of clusters k must be decided in advance (elbow / silhouette help).
- Sensitive to the initial centroids (local minimum) and to **outliers**, because the centroid is a mean and is pulled towards extreme points.
- Assumes clusters are roughly spherical and of similar size; it fails on elongated, ring-shaped or very unequal clusters (DBSCAN or Gaussian mixtures are better).
- Works only with numerical features and Euclidean distance; categorical data needs K-Modes / K-Prototypes.
- Needs feature scaling and becomes less meaningful in very high dimensions.
- Hard clustering: every point belongs to exactly one cluster.

Q6. Why is feature scaling important before applying K-Means?

K-Means uses Euclidean distance, so a feature with a large range dominates the distance. If income is in rupees (thousands to lakhs) and age is 18–70, the clusters would be formed almost only by income. Standardisation (`StandardScaler`: z = (x − mean)/std) gives every feature the same influence.

In the Mall Customers practical both features (income 15–137, score 1–99) have similar ranges, so the clusters with and without scaling are identical (ARI = 1.0); with Age added, scaling should be done.

Q7. Why is K-Means suitable for unsupervised image segmentation?

An image is a set of pixels and every pixel is a point described by its colour values (R, G, B or satellite bands). Pixels of the same object (water, vegetation, buildings) have similar colour values, so they form groups in the colour space. K-Means finds these groups without any labelled training image, is fast on millions of pixels and is simple to implement; each pixel is then replaced by the colour of its cluster centre to obtain the segmented image.

Q8. Why do we reshape an image into a two-dimensional array?

A colour image has the shape (height, width, channels), but `KMeans.fit()` expects a 2-D array of shape (n_samples, n_features). Reshaping to (height × width, channels) gives one row per pixel and one column per band; after clustering the labels are reshaped back to (height, width).

```
h, w, c = image.shape                      # e.g. (400, 600, 3)
pixels = image.reshape(-1, c)                # (240000, 3): one row per pixel
km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(pixels)
segmented = km.cluster_centers_[km.labels_].reshape(h, w, c)
```

Q9. What does each row of the pixels array represent?

Each row is one pixel of the image and its columns are the values of that pixel in each band, for example [R, G, B] = [34, 120, 56]. For a satellite image with four bands it is [Blue, Green, Red, NIR]. A 400 × 600 image therefore gives 240000 rows.

Q10. What is the role of the cluster centroid?

The centroid is the mean of all points in the cluster, i.e. its representative point. It is used (1) to assign a point to the nearest cluster, (2) to be recomputed in every iteration until convergence, and (3) to describe and interpret the cluster. In image segmentation the centroid is the average colour that replaces all pixels of the cluster; for customers it is the typical income and spending score of the group (for example (86.5, 82.1) for the premium group).

Q11. What happens when we change (K=4) to (K=6)?

- The image / data is divided into more and smaller segments, so more detail is shown (for example different types of vegetation or shadows become separate).
- The inertia decreases (it always decreases when k increases) and the segmented image looks closer to the original.
- Clusters may be split without real meaning (over-segmentation), the interpretation becomes harder and the computation time grows.
- With K = 4 several land-cover types may be merged into one cluster (under-segmentation). The best k is selected using the elbow method, the silhouette score and the purpose of the application.

Q12. Does K-Means automatically identify water, vegetation, and buildings?

No. K-Means groups pixels only by similarity of their values and gives arbitrary numbers 0, 1, 2 ... to the clusters. It does not know what water or a building is. A person (or a reference map / ground truth) must examine the centroid values and the segmented image and give the names, for example the cluster with a very low red and NIR value is water. Different objects with similar colour (dark water and shadow) can fall in the same cluster.

Q13. How can satellite bands such as Near-Infrared (NIR) improve land-cover segmentation?

Different surfaces reflect each band differently. Healthy vegetation reflects strongly in NIR and absorbs red light, water absorbs almost all NIR, and buildings / bare soil reflect NIR moderately. Adding the NIR band to R, G, B gives K-Means extra features in which these classes are clearly separated, which is not possible with the visible bands alone. Indices computed from it make the separation even easier:

```
NDVI = (NIR - Red) / (NIR + Red)     # high for vegetation, near 0 or negative for water and built-up areas
```

Used as an additional column of the pixels array, NDVI improves the segmentation of vegetation and water.

CONCLUSION:
Signature:

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:10	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	
PROBLEM STATEMENT	Implementation of decision tree classifier using python with suitable dataset	
PREREQUISITES	• Concept of classification problems • Basic understanding of decision tree structure (nodes, branches, leaves) • Knowledge of entropy/Gini index (for splitting criteria)	
OBJECTIVES:	• To understand the working of a decision tree for classification problems. • To implement a decision tree classifier using scikit-learn on a real-time dataset. • To evaluate the model's accuracy and visualize the decision tree structure.	
Tools Used	• Python 3.x • Jupyter Notebook • Libraries: scikit-learn, matplotlib	

THEORY:

Decision Tree classifier

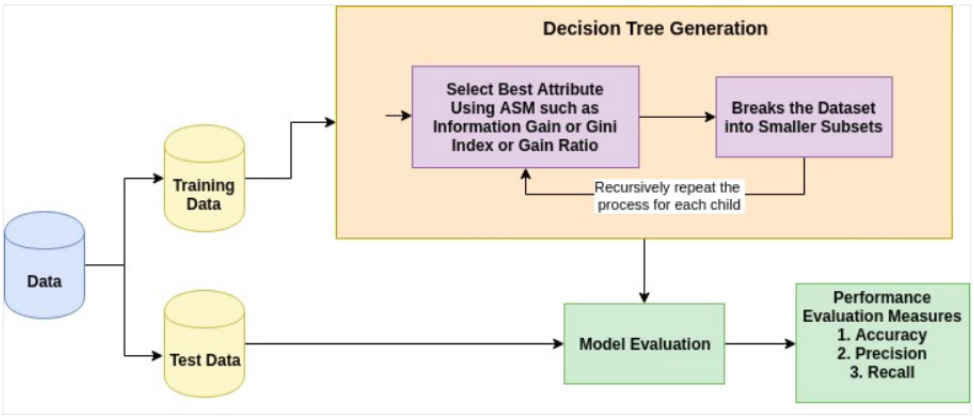
A Decision Tree classifier creates an upside-down tree to make predictions, starting at the top with a question about an important feature in your data, then branches out based on the answers. As you follow these branches down, each stop asks another question, narrowing down the possibilities. This question-and-answer game continues until you reach the bottom a leaf node where you get your final prediction or classification. A decision tree is a white box type of ML algorithm. It shares internal decision-making logic, which is not available in the black box type of algorithms such as with a neural network. Its training time is faster compared to the neural network algorithm.

The time complexity of decision trees is a function of the number of records and attributes in the given data. The decision tree is a distribution-free or non-parametric method which does not depend upon assumptions about the underlying probability distribution of the data. Decision trees can handle high-dimensional data with good accuracy.

How Does the Decision Tree Algorithm Work?

The basic idea behind any decision tree algorithm is as follows:

- Select the best attribute using Attribute Selection Measures (ASM) to split the records.
- Make that attribute a decision node and breaks the dataset into smaller subsets.
- Start tree building by repeating this process recursively for each child until one of the conditions will match:
  - All the tuples belong to the same attribute value.
  - There are no more remaining attributes.
  - There are no more instances.



Attribute Selection Measures

Attribute selection measure is a heuristic for selecting the splitting criterion that partitions data in the best possible manner. It is also known as splitting rules because it helps us to determine breakpoints for tuples on a given node. ASM provides a rank to each feature (or attribute) by explaining the given dataset. The best score attribute will be selected as a splitting attribute. In the case of a continuous-valued attribute, split points for branches also need to define. The most popular selection measures are Information Gain, Gain Ratio, and Gini Index.

Information gain

Claude Shannon invented the concept of entropy, which measures the impurity of the input set. In physics and mathematics, entropy is referred to as the randomness or the impurity in a system. In information theory, it refers to the impurity in a group of examples. Information gain is the decrease in entropy. Information gain computes the difference between entropy before the split and average entropy after the split of the dataset based on given attribute values. ID3 (Iterative Dichotomiser) decision tree algorithm uses information gain.

$$\text{Info}(D) = -\sum_{i=1}^{m} pi \log_2 pi$$

Where Pi is the probability that an arbitrary tuple in D belongs to class Ci.

$$\text{Info}_A(D) = \sum_{j=1}^{V} \frac{|Dj|}{|D|} \quad \text{X Info}(D_j)$$

$$\text{Gain}(A) = \text{Info}(D) - \text{Info}_A(D)$$

Where: Info(D) is the average amount of information needed to identify the class label of a tuple in D. |Dj|/|D| acts as the weight of the jth partition. $\text{Info}_A(D)$ is the expected information required to classify a tuple from D based on the partitioning by A. The attribute A with the highest information gain, Gain(A), is chosen as the splitting attribute at node N().

Gain ratio

Information gain is biased for the attribute with many outcomes. It means it prefers the attribute with a large number of distinct values. For instance, consider an attribute with a unique identifier, such as customer_ID, that has zero info(D) because of pure partition. This maximizes the information gain and creates useless partitioning. C4.5, an improvement of ID3, uses an extension to information gain known as the gain ratio. Gain ratio handles the issue of bias by normalizing the information gain using Split Info. Java implementation of the C4.5 algorithm is known as J48, which is available in WEKA data mining tool.

$$SplitInfo_A(D) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \times \log_2\left(\frac{|D_j|}{|D|}\right)$$

Where |Dj|/|D| acts as the weight of the jth partition and v is the number of discrete values in attribute A. The gain ratio can be defined as

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)}$$

Gini index

Another decision tree algorithm CART (Classification and Regression Tree) uses the Gini method to create split points.

$$Gini(D) = 1 - \sum_{i=1}^{m} Pi^2$$

Where pi is the probability that a tuple in D belongs to class Ci. The Gini Index considers a binary split for each attribute. You can compute a weighted sum of the impurity of each partition. If a binary split on attribute A partitions data D into D1 and D2, the Gini index of D is:

In the case of a discrete-valued attribute, the subset that gives the minimum gini index for that chosen is selected as a splitting attribute. In the case of continuous-valued attributes, the strategy is to select each pair of adjacent values as a possible split point, and a point with a smaller gini index is chosen as the splitting point.

$$\Delta Gini(A) = Gini(D) - Gini_A(D).$$

Classification Step

1. Start at the root node of the trained decision tree.
2. Evaluate the feature and split condition at the current node.
3. Repeat step 2 at each subsequent node until reaching a leaf node.
4. The class label of the leaf node becomes the prediction for the new instance.

Pros:

1. **Interpretability:** Easy to understand and visualize the decision-making process.
2. **No Feature Scaling:** Can handle both numerical and categorical data without normalization.
3. **Handles Non-linear Relationships:** Can capture complex patterns in the data.
4. **Feature Importance:** Provides a clear indication of which features are most important for prediction.

Cons:

1. **Overfitting:** Prone to creating overly complex trees that don't generalize well, especially with small datasets.
2. **Instability:** Small changes in the data can result in a completely different tree being generated.
3. **Biased with Imbalanced Datasets:** Can be biased towards dominant classes.
4. **Inability to Extrapolate:** Cannot make predictions beyond the range of the training data.

ALGORITHM

Steps of the Decision Tree classifier:

1. Import **pandas, matplotlib** and the scikit-learn modules; load the Iris dataset (150 samples, 4 features, 3 classes).
2. Separate the features X and the target y; split with `train_test_split(test_size=0.25, random_state=0, stratify=y)`.
3. Create `DecisionTreeClassifier(criterion='entropy' or 'gini', max_depth=3, random_state=0)`.
4. Train with `fit(X_train, y_train)`. At every node choose the attribute and threshold with the highest information gain (lowest Gini index), where Entropy = $-\Sigma\ p \log_2 p$, Gain = Entropy(parent) $-\ \Sigma\ (|D_j|/|D|)$ Entropy(D_j), Gini = $1 - \Sigma\ p^2$.
5. Repeat the split recursively until the node is pure, `max_depth` is reached or no attribute is left; the node then becomes a leaf with a class label.
6. Predict the test data with `predict(X_test)` (start at the root, follow the conditions down to a leaf).
7. Evaluate with `accuracy_score`, `confusion_matrix` and `classification_report`.
8. Visualise the tree with `plot_tree` and print its rules with `export_text`.
9. Compare Gini and entropy, vary `max_depth` from 1 to 10 and read `feature_importances_`.
10. Predict the class of a new sample.

ERRORS and TEST CASES, USE CASES

Common errors and their remedies

Error / symptom	Cause	Remedy
ValueError: could not convert string to float	Text features passed to the tree.	Encode with `get_dummies` or `LabelEncoder`.
ValueError: Input contains NaN	Missing values in the features (not accepted by every version).	Impute or drop the rows before `fit`.
Overfitting: train 100%, test much lower	The tree grows to full depth.	Limit `max_depth`, `min_samples_leaf` or use pruning (`ccp_alpha`).
Underfitting: both scores low	The tree is too shallow (e.g. max_depth = 1).	Increase `max_depth`.
A different tree on each run	Ties between equally good splits are broken randomly.	Fix `random_state`.
plot_tree is unreadable	Figure too small.	Increase `figsize` and `fontsize`, or limit the plotted depth.
Misleading accuracy	Imbalanced classes.	Study the confusion matrix, precision and recall.

Test cases

ID	Input / condition	Expected result
T1	Iris dataset loaded	150 rows, 4 features, 3 classes with 50 samples each
T2	Split with test_size = 0.25	112 training rows and 38 testing rows
T3	Entropy tree, max_depth = 3	Train accuracy 0.9643, test accuracy 0.9474
T4	Confusion matrix of T3	[[13, 0, 0], [0, 13, 0], [0, 2, 10]]: two virginica flowers predicted as versicolor
T5	Unrestricted Gini tree vs entropy tree	Test accuracy 0.9737 (Gini) and 0.9474 (entropy)
T6	max_depth = 1 and max_depth ≥ 4	Test accuracy 0.6579 (underfit); training accuracy 1.0 from depth 4
T7	`feature_importances_`	Petal width (0.9767) is the most important feature
T8	Predict [5.1, 3.5, 1.4, 0.2] and [6.7, 3.0, 5.2, 2.3]	setosa and virginica
T9	(Negative test) a text feature in X	ValueError

	Use cases • Medical diagnosis: rules that doctors can read and verify. • Credit and loan approval decisions that must be explained. • Customer churn and fraud detection. • Fault diagnosis in machines and networks. • Extracting human-readable if-else rules from data (white-box model).

REFERENCES: Refer lab manual for the same

Practice Problem Statements:

1. Train a decision tree using criterion='gini' instead of 'entropy' and compare accuracy.
2. Vary max_depth from 1 to 10 and plot accuracy vs depth to observe overfitting/underfitting.
3. Implement a decision tree classifier on the Titanic dataset (sex, age, pclass) and compare accuracy with logistic regression (Practical 7).
4. Print feature importance scores using model.feature_importances_ and identify the most influential feature.
5. Generate and display a confusion matrix for the Iris classification results

Important Questions: -

1. What is the difference between Gini Index and Entropy as splitting criteria?
2. What is Information Gain, and how is it calculated?
3. How does max_depth help prevent overfitting in decision trees?
4. What is pruning in decision trees, and why is it used?
5. What are the advantages and disadvantages of decision tree classifiers compared to logistic regression?
6. How does a decision tree handle both numerical and categorical features?

Programs are written in Python using Pandas, Matplotlib and scikit-learn and executed in Jupyter Notebook. Dataset: Iris (150 flowers, 4 features, 3 classes), loaded with sklearn.datasets.load_iris().

PROGRAM AND OUTPUT

Load the Iris dataset and split it

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report,
                             ConfusionMatrixDisplay)
pd.set_option("display.width", 85)

iris = load_iris(as_frame=True)
X, y = iris.data, iris.target
print("Shape:", X.shape, " Classes:", iris.target_names.tolist())
print(X.head())
print("\nClass counts:", y.value_counts().sort_index().to_dict())

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
                                                    random_state=0, stratify=y)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

Output:

```
Shape: (150, 4)  Classes: ['setosa', 'versicolor', 'virginica']
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

Class counts: {0: 50, 1: 50, 2: 50}
Train: (112, 4)  Test: (38, 4)
```

Train the decision tree and evaluate it

Code:

```
dt = DecisionTreeClassifier(criterion="entropy", max_depth=3, random_state=0)
dt.fit(X_train, y_train)
pred = dt.predict(X_test)

print("Train accuracy: %.4f   Test accuracy: %.4f" %
      (dt.score(X_train, y_train), accuracy_score(y_test, pred)))
print("\nConfusion matrix:")
print(confusion_matrix(y_test, pred))
print("\nClassification report:")
print(classification_report(y_test, pred, target_names=iris.target_names))
```

Output:

```
Train accuracy: 0.9643   Test accuracy: 0.9474

Confusion matrix:
```

```
[[13  0  0]
 [ 0 13  0]
 [ 0  2 10]]

Classification report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        13
  versicolor       0.87      1.00      0.93        13
   virginica       1.00      0.83      0.91        12

    accuracy                           0.95        38
   macro avg       0.96      0.94      0.95        38
weighted avg       0.95      0.95      0.95        38
```

Visualise the tree and print its rules

Code:

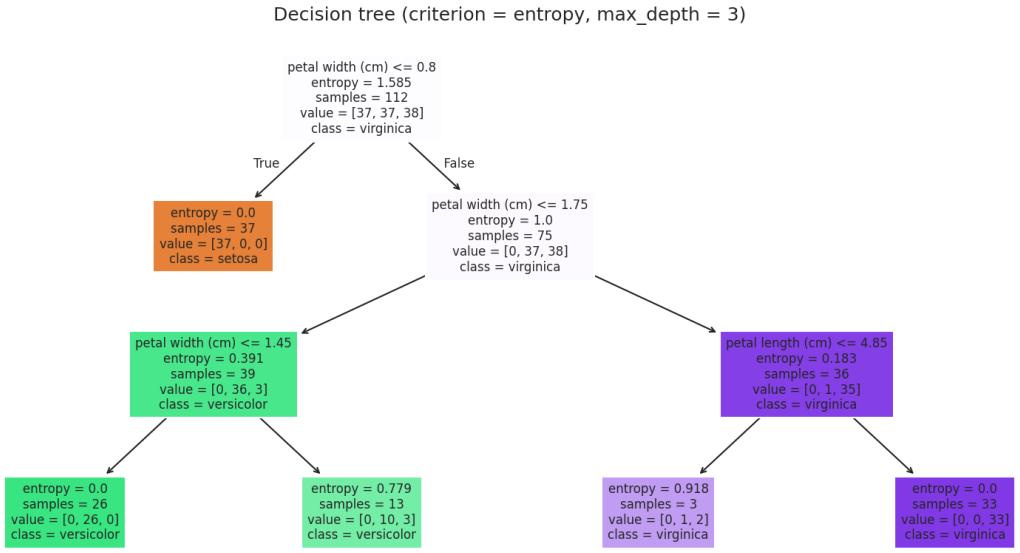
```
print(export_text(dt, feature_names=list(X.columns)))

plt.figure(figsize=(11, 5.5))
plot_tree(dt, feature_names=X.columns, class_names=iris.target_names, filled=True, fontsize=8)
plt.title("Decision tree (criterion = entropy, max_depth = 3)")
plt.tight_layout(); plt.show()
```

Output:

```
|--- petal width (cm) <= 0.80
|   |--- class: 0
|--- petal width (cm) >  0.80
|   |--- petal width (cm) <= 1.75
|   |   |--- petal width (cm) <= 1.45
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.45
|   |   |   |--- class: 1
|   |--- petal width (cm) >  1.75
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- class: 2
|   |   |--- petal length (cm) >  4.85
|   |   |   |--- class: 2
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043
	Department of Electronics & Computer Engineering
	ASSESSMENT YEAR: 2026-2027 / **CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]
EXPT No:10	
PROBLEM STATEMENT	Implementation of decision tree classifier using python with suitable dataset



Interpretation: the root question is *petal width ≤ 0.80 cm*, which separates all setosa flowers at once. Petal width and petal length then separate versicolor from virginica. The only errors on the test set are two virginica flowers predicted as versicolor.

Practice problem 1: Gini index instead of entropy

Code:

```
for c in ["gini", "entropy"]:
    m = DecisionTreeClassifier(criterion=c, random_state=0).fit(X_train, y_train)
    print("%-8s depth=%d leaves=%d test accuracy=%.4f" %
          (c, m.get_depth(), m.get_n_leaves(), m.score(X_test, y_test)))
```

Output:

```
gini     depth=4 leaves=8 test accuracy=0.9737
entropy  depth=7 leaves=10 test accuracy=0.9474
```

Observation: on this split the Gini tree gives 0.9737 and the entropy tree 0.9474 (one test sample difference). Both criteria usually give very similar trees; Gini is slightly faster because it needs no logarithm.

Practice problem 2: accuracy versus max_depth

Code:

```
depths = range(1, 11)
train_acc, test_acc = [], []
for d in depths:
    m = DecisionTreeClassifier(max_depth=d, random_state=0).fit(X_train, y_train)
    train_acc.append(m.score(X_train, y_train))
    test_acc.append(m.score(X_test, y_test))

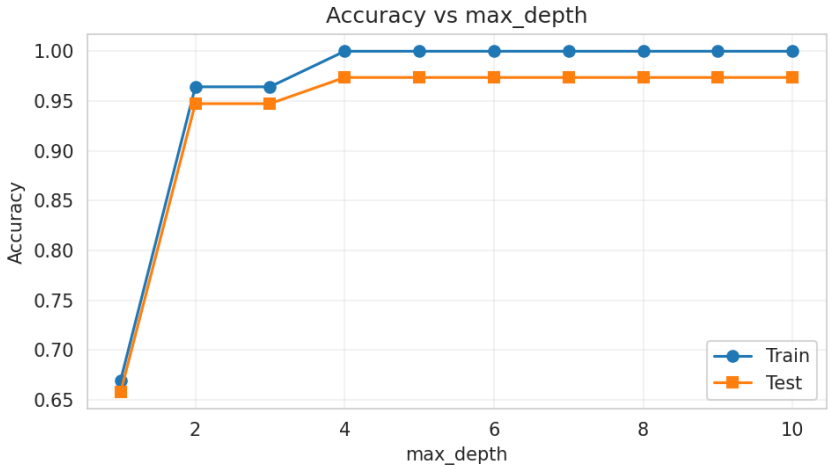
print("Depth  Train   Test")
for d, a, b in zip(depths, train_acc, test_acc):
    print("%-6d %.4f  %.4f" % (d, a, b))

plt.figure(figsize=(6.5, 3.8))
plt.plot(depths, train_acc, "o-", label="Train")
```

```
plt.plot(depths, test_acc, "s-", label="Test")
plt.xlabel("max_depth"); plt.ylabel("Accuracy"); plt.title("Accuracy vs max_depth")
plt.legend(); plt.grid(alpha=.3)
plt.tight_layout(); plt.show()
```

Output:

```
Depth  Train   Test
1      0.6696  0.6579
2      0.9643  0.9474
3      0.9643  0.9474
4      1.0000  0.9737
5      1.0000  0.9737
6      1.0000  0.9737
7      1.0000  0.9737
8      1.0000  0.9737
9      1.0000  0.9737
10     1.0000  0.9737
```



Interpretation: with max_depth = 1 the tree can separate only two of the three classes (test accuracy 0.658): **underfitting**. From depth 2 to 3 the accuracy is 0.947. From depth 4 the training accuracy is 1.0, i.e. the tree memorises the training data; on this small and clean dataset the test accuracy does not fall, but the tree becomes more complex (up to 10 leaves) without any gain. A gap between training and test accuracy is the sign of **overfitting**.

Practice problem 3: decision tree versus logistic regression (Titanic)

Code:

```
from sklearn.linear_model import LogisticRegression

t = pd.read_csv("Titanic-Dataset.csv")[["Survived", "Pclass", "Sex", "Age"]]
t["Age"] = t["Age"].fillna(t["Age"].median())
t["Sex"] = t["Sex"].map({"male": 0, "female": 1})
Xt, yt = t[["Sex", "Age", "Pclass"]], t["Survived"]
Xtr, Xte, ytr, yte = train_test_split(Xt, yt, test_size=0.2, random_state=0)

dtc = DecisionTreeClassifier(criterion="gini", max_depth=3, random_state=0).fit(Xtr, ytr)
lr = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
print("Decision tree accuracy      :", round(dtc.score(Xte, yte), 4))
print("Logistic regression accuracy:", round(lr.score(Xte, yte), 4))
```

Output:

```
  Decision tree accuracy      : 0.7989
  Logistic regression accuracy: 0.8101
```

Practice problem 4: feature importance

Code:

```
for name, value in sorted(zip(X.columns, dt.feature_importances_), key=lambda t: -t[1]):
    print("  %-20s %.4f" % (name, value))
```

Output:

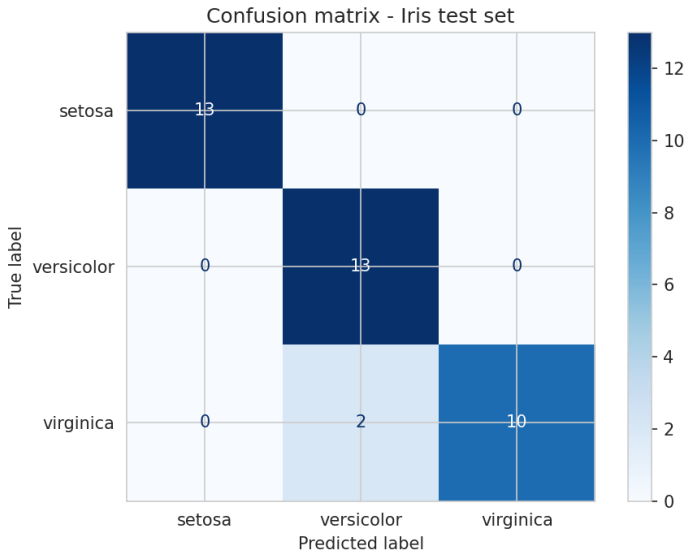
```
  petal width (cm)     0.9767
  petal length (cm)    0.0233
  sepal length (cm)    0.0000
  sepal width (cm)     0.0000
```

Observation: petal width (0.9767) is by far the most influential feature; petal length contributes 0.0233 and the sepal measurements are not used by this tree.

Practice problem 5: confusion matrix display

Code:

```
ConfusionMatrixDisplay.from_predictions(y_test, pred, display_labels=iris.target_names,
                                        cmap="Blues")
plt.title("Confusion matrix - Iris test set")
plt.tight_layout(); plt.show()
```



Prediction for new flowers

Code:

```
new = pd.DataFrame([[5.1, 3.5, 1.4, 0.2], [6.7, 3.0, 5.2, 2.3]], columns=X.columns)
for row, label in zip(new.values, dt.predict(new)):
    print(row, "->", iris.target_names[label])
```

Output:

```
[5.1 3.5 1.4 0.2] -> setosa
```

```
[6.7 3.  5.2 2.3] -> virginica
```

IMPORTANT QUESTIONS AND ANSWERS

Q1. What is the difference between Gini Index and Entropy as splitting criteria?

Property	Gini index	Entropy (information gain)
Formula	Gini = $1 - \Sigma\, p_i^2$	Entropy = $-\Sigma\, p_i \log_2 p_i$
Value for 2 equal classes (p = 0.5, 0.5)	0.5 (maximum)	1 bit (maximum)
Value for a pure node	0	0
Used by	CART (scikit-learn default, criterion='gini')	ID3, C4.5 (criterion='entropy')
Computation	Faster (no logarithm)	Slightly slower
Result	Usually almost the same tree; Gini tends to isolate the largest class	Tends to give slightly more balanced trees

In this practical the Gini tree gave a test accuracy of 0.9737 and the entropy tree 0.9474, i.e. a difference of one test sample.

Q2. What is Information Gain, and how is it calculated?

Information gain is the decrease in entropy obtained by splitting the data on an attribute A: Gain(A) = Info(D) − $\text{Info}_A(D)$, where Info(D) = $-\Sigma\, p_i \log_2 p_i$ and $\text{Info}_A(D) = \Sigma\, (|D_j|/|D|) \times \text{Info}(D_j)$. The attribute with the highest gain is chosen for the split.

Example: 14 records, 9 Yes and 5 No. Info(D) = $-(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.940$. Splitting on Outlook gives Sunny (2 Yes, 3 No) with entropy 0.971, Overcast (4 Yes, 0 No) with entropy 0 and Rain (3 Yes, 2 No) with entropy 0.971. $\text{Info}_{\text{Outlook}}(D)$ = (5/14)(0.971) + (4/14)(0) + (5/14)(0.971) = 0.694, so Gain(Outlook) = 0.940 − 0.694 = **0.247**.

Information gain prefers attributes with many distinct values (such as an ID); the gain ratio of C4.5 corrects this.

Q3. How does max_depth help prevent overfitting in decision trees?

A tree that is allowed to grow to full depth keeps splitting until every leaf is pure, so it memorises the noise of the training data and generalises badly. `max_depth` limits the number of questions from the root to a leaf, so the tree stays simple and learns only the main patterns. Too small a value causes underfitting.

In the practical (Iris) max_depth = 1 gives only 0.658 test accuracy (underfitting), max_depth = 2 or 3 gives 0.947 with a small tree, while from depth 4 the training accuracy is 100% and the tree only becomes larger. The best depth is chosen where the test (or cross-validation) accuracy is highest with the simplest tree.

Q4. What is pruning in decision trees, and why is it used?

Pruning removes branches that give little predictive power, to reduce the size of the tree and avoid overfitting.

- **Pre-pruning (early stopping):** stop the growth while building: `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`.
- **Post-pruning:** grow the full tree and then cut weak branches. scikit-learn uses cost-complexity pruning with the parameter `ccp_alpha`; a larger value prunes more.

A pruned tree is smaller, faster, easier to interpret and generally gives better accuracy on unseen data.

Q5. What are the advantages and disadvantages of decision tree classifiers compared to logistic regression?

Aspect	Decision tree	Logistic regression
Decision boundary	Non-linear, axis-parallel rectangles	Linear boundary
Interpretability	If-else rules that can be drawn (white box)	Coefficients / odds ratios

Aspect	Decision tree	Logistic regression
Feature scaling	Not needed	Recommended
Categorical data	Handled easily (after encoding in scikit-learn)	Needs encoding
Probability output	Rough (fraction of a leaf)	Smooth, well-calibrated probability
Overfitting	High risk; needs max_depth / pruning	Lower risk; regularisation (C)
Stability	Small data change can give a different tree	Stable
Best when	Non-linear relations and interactions	Roughly linear relation, small data

Q6. How does a decision tree handle both numerical and categorical features?

Numerical features: the tree sorts the values, takes the mid-point between each pair of adjacent values as a candidate threshold and selects the threshold with the best Gini / information gain; the question is of the form 'petal width ≤ 0.80?'. No scaling is needed because only the order of the values matters.

Categorical features: theoretically the tree splits on the categories (one branch per value in ID3, or two groups of values in CART). scikit-learn's trees accept only numbers, so the categories are first converted using label encoding (ordered categories) or one-hot encoding (`pd.get_dummies`), and the tree then asks questions such as 'Sex_female ≤ 0.5?'.

CONCLUSION:
Signature:

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:11	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	
PROBLEM STATEMENT	Implementation of AND/NAND gate using feed forward Neural Network	
PREREQUISITES	Basic understanding of logic gates and truth tables (AND, NAND) Concept of artificial neural networks (neurons, weights, activation functions) Basic idea of supervised learning and training iterations (epochs)	
OBJECTIVES:	To understand the structure and working of a feed-forward neural network. To implement a neural network that learns the AND and NAND logic gates. To verify the network's output against the truth table	
Tools Used	Python 3.x Jupyter Notebook Libraries: numpy, scikit-learn	
THEORY:		

Perceptron Learning Algorithm: The Perceptron is the simplest form of an artificial neural network, acting as a linear binary classifier. It takes a set of input features, multiplies them by weights, adds a bias, and passes the result through a step activation function to yield a binary output (0 or 1, or -1 and 1).

Step-by-Step Algorithm

1. **Initialize parameters:** Set all weights W and the bias b to 0 or small random numbers. Define a learning rate L, usually between 0 and 1).
2. **Compute the output:** For a training sample with inputs $x = [x_1, x_2, ..., x_n]$

$$\hat{y} = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

Where f(z) is a hard step activation function:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

3. **Update parameters:** Compare the predicted output

$$w_i \leftarrow w_i + \eta \cdot (y - \hat{y}) \cdot x_i$$

$$b \leftarrow b + \eta \cdot (y - \hat{y})$$

4. **Iterate:** Repeat steps 2 and 3 for all training examples (one epoch) until the network classifies all items correctly or reaches a maximum iteration limit.

ALGORITHM

Steps:

1. Write the truth table: inputs X = [[0,0],[0,1],[1,0],[1,1]]; targets AND = [0, 0, 0, 1], NAND = [1, 1, 1, 0].
2. Initialise the weights w1 = w2 = 0 and the bias b = 0; choose the learning rate L = 0.1.
3. For one training row compute z = w1*A + w2*B + b and the output y' = f(z), where f(z) = 1 if z >= 0, otherwise 0 (step activation).
4. Find the error = target - y'.
5. Update the parameters: w = w + L * error * x and b = b + L * error.
6. Repeat steps 3-5 for all four rows (one epoch) until an epoch has no errors or the maximum number of epochs is reached.
7. Verify: apply the trained network to all four inputs and compare the outputs with the truth table.
8. Repeat with a feed-forward network (scikit-learn MLPClassifier, 4 hidden neurons, backpropagation) and plot the decision line.

ERRORS and TEST CASES, USE CASES

Errors and how to avoid them

Error / symptom	Cause and remedy
Every output is 1, accuracy 0.25	learning rate set to 0, so the weights never change
ValueError: matmul dimension mismatch	an input with 3 values was given - the gate has exactly 2 inputs (A, B)
ConvergenceWarning: maximum iterations reached	increase max_iter (or use solver='lbfgs' for tiny data)
Not converged within the epochs	epochs too small - AND / NAND are linearly separable, so more epochs will converge

Test cases

No.	Input / condition	Expected output
TC1	Inputs (0,0), (0,1), (1,0), (1,1) to the trained AND network	outputs 0, 0, 0, 1 - truth table verified
TC2	The same four inputs to the trained NAND network	outputs 1, 1, 1, 0 - truth table verified
TC3	Train AND / NAND with learning rate 0.1	AND converges in 4 epochs (w = [0.2, 0.1], b = -0.2); NAND in 6 epochs
TC4	Learning rate = 0 (AND)	weights stay 0, every output is 1, accuracy 0.25
TC5	MLPClassifier(max_iter = 5)	ConvergenceWarning: maximum iterations (5) reached
TC6	MLP predict() with a 3-value input	ValueError: X has 3 features, but MLPClassifier is expecting 2 features

Use cases

- Digital logic: a single neuron can realise AND / NAND; NAND is a universal gate, so networks of NAND neurons can build any Boolean circuit.
- Understanding the building block (weights, bias, activation) of larger neural networks.
- Teaching the perceptron learning rule and the idea of linear separability.

REFERENCES: Refer lab manual for the same

Important Questions: -

1. What is a perceptron, and how does it relate to a feed-forward neural network?
2. Why are AND and NAND gates considered linearly separable problems?
3. What is the role of the activation function in a neural network?
4. What is backpropagation, and how does it help in training the model?
5. What is the difference between MLPClassifier and a simple single-layer Perceptron in scikit-learn?
6. Why might increasing max_iter be necessary for some logic gate models to converge?

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:11	LAB Ref: BE/2026-27/	Starting date:
	Roll No:	
PROBLEM STATEMENT	Implementation of AND/NAND gate using feed forward Neural Network	

Libraries: numpy, matplotlib, scikit-learn. The AND and NAND gates are learned by (a) a single neuron (perceptron) written from scratch with the perceptron learning rule and (b) a feed-forward network (MLPClassifier) from scikit-learn.

PROGRAM AND OUTPUT

Import libraries and create the truth tables

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import Perceptron

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = np.array([0, 0, 0, 1])
y_nand = np.array([1, 1, 1, 0])

print("A B | AND NAND")
for (a, b), t1, t2 in zip(X, y_and, y_nand):
    print(a, b, "|", t1, "  ", t2)
```

Output:

```
A B | AND NAND
0 0 | 0    1
0 1 | 0    1
1 0 | 0    1
1 1 | 1    0
```

Perceptron (single neuron) class and verification function

Code:

```
class SimplePerceptron:
    """Single neuron with a step activation, trained with the perceptron learning rule."""
    def __init__(self, lr=0.1, epochs=100):
        self.lr = lr
        self.epochs = epochs

    def step(self, z):
        return np.where(z >= 0, 1, 0)

    def predict(self, X):
        return self.step(X @ self.w + self.b)

    def fit(self, X, y, verbose=True):
        self.w = np.zeros(X.shape[1])
        self.b = 0.0
        for epoch in range(1, self.epochs + 1):
            errors = 0
            for xi, target in zip(X, y):
                out = self.step(xi @ self.w + self.b)
                update = self.lr * (target - out)
                self.w = self.w + update * xi
                self.b = self.b + update
                errors += int(update != 0)
            if verbose:
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:11	LAB Ref: BE/2026-27/	Starting date:
	Roll No:	
PROBLEM STATEMENT	Implementation of AND/NAND gate using feed forward Neural Network	

```
                print("Epoch", epoch, "-> errors:", errors, " w =", self.w.round(2), " b =", round(self.b,
2))
            if errors == 0:
                print("Converged after", epoch, "epochs")
                break
        return self


def verify(model, X, y, name):
    pred = model.predict(X)
    print(name, "- truth table verification")
    print("A B | Expected | Predicted | Result")
    for (a, b), t, p in zip(X, y, pred):
        print(a, b, "|    ", t, "   |    ", p, "    |", "Correct" if t == p else "WRONG")
    print("Accuracy:", (pred == y).mean())
```

Output:

Train the network for the AND gate

Code:

```
and_model = SimplePerceptron(lr=0.1, epochs=50)
and_model.fit(X, y_and)
print("\nFinal weights:", and_model.w.round(2), " bias:", round(and_model.b, 2), "\n")
verify(and_model, X, y_and, "AND")
```

Output:

```
Epoch 1 -> errors: 2  w = [0.1 0.1]  b = 0.0
Epoch 2 -> errors: 3  w = [0.2 0.1]  b = -0.1
Epoch 3 -> errors: 3  w = [0.2 0.1]  b = -0.2
Epoch 4 -> errors: 0  w = [0.2 0.1]  b = -0.2
Converged after 4 epochs

Final weights: [0.2 0.1]  bias: -0.2

AND - truth table verification
A B | Expected | Predicted | Result
0 0 |     0    |     0     | Correct
0 1 |     0    |     0     | Correct
1 0 |     0    |     0     | Correct
1 1 |     1    |     1     | Correct
Accuracy: 1.0
```

Observation: the neuron learns the AND gate in 4 epochs. The final rule 0.2A + 0.1B - 0.2 >= 0 is true only when A = B = 1.

Train the network for the NAND gate

Code:

```
nand_model = SimplePerceptron(lr=0.1, epochs=50)
nand_model.fit(X, y_nand)
print("\nFinal weights:", nand_model.w.round(2), " bias:", round(nand_model.b, 2), "\n")
verify(nand_model, X, y_nand, "NAND")
```

Output:

```
Epoch 1 -> errors: 1  w = [-0.1 -0.1]  b = -0.1
Epoch 2 -> errors: 3  w = [-0.2 -0.1]  b = 0.0
Epoch 3 -> errors: 3  w = [-0.2 -0.1]  b = 0.1
Epoch 4 -> errors: 2  w = [-0.2 -0.2]  b = 0.1
```

```
Epoch 5 -> errors: 1  w = [-0.2 -0.1]  b = 0.2
Epoch 6 -> errors: 0  w = [-0.2 -0.1]  b = 0.2
Converged after 6 epochs

Final weights: [-0.2 -0.1]  bias: 0.2

NAND - truth table verification
A B | Expected | Predicted | Result
0 0 |     1    |     1     | Correct
0 1 |     1    |     1     | Correct
1 0 |     1    |     1     | Correct
1 1 |     0    |     0     | Correct
Accuracy: 1.0
```

Observation: NAND is learned in 6 epochs. The weights are negative and the bias positive: -0.2A - 0.1B + 0.2 >= 0 is false only when A = B = 1.

Feed-forward neural network with scikit-learn (MLPClassifier)

Code:

```
for name, target in [("AND", y_and), ("NAND", y_nand)]:
    mlp = MLPClassifier(hidden_layer_sizes=(4,), activation="logistic", solver="lbfgs",
                        max_iter=1000, random_state=1)
    mlp.fit(X, target)
    print(name, "gate: iterations =", mlp.n_iter_, " final loss =", round(mlp.loss_, 4))
    print("Probability of output 1:", mlp.predict_proba(X)[:, 1].round(3))
    verify(mlp, X, target, name)
    print()
```

Output:

```
AND gate: iterations = 20  final loss = 0.0035
Probability of output 1: [0.    0.001 0.001 0.999]
AND - truth table verification
A B | Expected | Predicted | Result
0 0 |     0    |     0     | Correct
0 1 |     0    |     0     | Correct
1 0 |     0    |     0     | Correct
1 1 |     1    |     1     | Correct
Accuracy: 1.0

NAND gate: iterations = 23  final loss = 0.0033
Probability of output 1: [1.    1.    0.999 0.001]
NAND - truth table verification
A B | Expected | Predicted | Result
0 0 |     1    |     1     | Correct
0 1 |     1    |     1     | Correct
1 0 |     1    |     1     | Correct
1 1 |     0    |     0     | Correct
Accuracy: 1.0
```

Observation: the multi-layer network also reproduces both truth tables; its output probabilities are close to 0 or 1, and the lbfgs solver needed only 20 and 23 iterations.

scikit-learn Perceptron for comparison

Code:

```
for name, target in [("AND", y_and), ("NAND", y_nand)]:
```

```
    p = Perceptron(max_iter=1000, tol=None, random_state=0).fit(X, target)
    print(name, "-> accuracy:", p.score(X, target), " weights:", p.coef_[0], " bias:", p.intercept_[0])
```

Output:

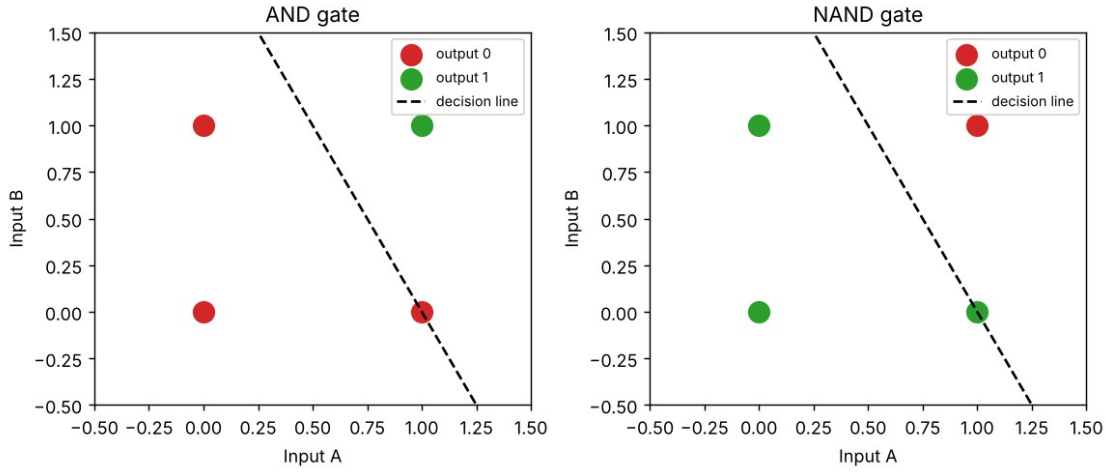
```
AND -> accuracy: 1.0  weights: [3. 2.]  bias: -4.0
NAND -> accuracy: 1.0  weights: [-3. -2.]  bias: 4.0
```

Decision boundary learned by the neuron

Code:

```
fig, ax = plt.subplots(1, 2, figsize=(9, 4))
for a, name, model, target in [(ax[0], "AND", and_model, y_and), (ax[1], "NAND", nand_model, y_nand)]:
    a.scatter(X[target == 0, 0], X[target == 0, 1], s=140, color="tab:red", label="output 0")
    a.scatter(X[target == 1, 0], X[target == 1, 1], s=140, color="tab:green", label="output 1")
    xs = np.linspace(-0.5, 1.5, 50)
    ys = -(model.w[0] * xs + model.b) / model.w[1]
    a.plot(xs, ys, "k--", label="decision line")
    a.set_xlim(-0.5, 1.5)
    a.set_ylim(-0.5, 1.5)
    a.set_title(name + " gate")
    a.set_xlabel("Input A")
    a.set_ylabel("Input B")
    a.legend(loc="upper right", fontsize=8)
plt.tight_layout()
plt.show()
```

Output:



Observation: one straight (dashed) line separates the output-0 points from the output-1 points for both gates, so AND and NAND are linearly separable.

IMPORTANT QUESTIONS AND ANSWERS

Q1. What is a perceptron, and how does it relate to a feed-forward neural network?

A perceptron is a single artificial neuron: it computes the weighted sum z = w1*A + w2*B + b of its inputs and passes it

through a step activation to give a 0 / 1 output. A feed-forward neural network is made of layers of such neurons (input layer, one or more hidden layers, output layer) in which the signals travel only forward, never in a loop. The perceptron is therefore the simplest feed-forward network (no hidden layer) and the basic building block of larger ones such as MLPClassifier.

Q2. Why are AND and NAND gates considered linearly separable problems?

A problem is linearly separable if one straight line can separate the inputs whose output is 0 from those whose output is 1. For AND only the point (1, 1) gives 1, so a line such as 0.2A + 0.1B - 0.2 = 0 (the one learned in this experiment) puts (1, 1) on one side and (0, 0), (0, 1), (1, 0) on the other. NAND is the complement of AND, so the same line with the signs reversed (-0.2A - 0.1B + 0.2 = 0) separates it. Because they are linearly separable, the perceptron convergence theorem guarantees that the learning rule finds the weights in a finite number of epochs (4 for AND and 6 for NAND here).

Q3. What is the role of the activation function in a neural network?

The activation function decides the output of a neuron from its weighted sum and adds non-linearity. Without it, any number of layers would collapse into a single linear function. The step function (0 / 1) is used by the perceptron; sigmoid / logistic, tanh and ReLU are used in multi-layer networks because they are differentiable, which backpropagation needs. In this experiment the from-scratch neuron uses a step function and MLPClassifier uses the logistic (sigmoid) function.

Q4. What is backpropagation, and how does it help in training the model?

Backpropagation computes the gradient of the loss with respect to every weight by applying the chain rule from the output layer back to the input layer. The weights are then moved against the gradient (w = w - L * dLoss/dw), and the process is repeated until the loss is small. It lets multi-layer networks be trained efficiently. The simple perceptron rule used for the single neuron does not need it, but MLPClassifier trains its weights this way.

Q5. What is the difference between MLPClassifier and a simple single-layer Perceptron in scikit-learn?

	Perceptron	**MLPClassifier**
Layers	Single layer (no hidden layer)	One or more hidden layers
Decision boundary	Linear (one straight line / plane)	Non-linear
Training	Perceptron update rule	Backpropagation with sgd, adam or lbfgs
Output	Class only	Class and probabilities (predict_proba)
Can solve XOR?	No	Yes (with a hidden layer)

Q6. Why might increasing max_iter be necessary for some logic gate models to converge?

Iterative solvers (sgd, adam, lbfgs) stop either when the loss stops improving or when max_iter is reached. If max_iter is too small the loss is still high when training stops and scikit-learn raises a ConvergenceWarning (in our test, MLPClassifier(max_iter = 5) gave 'Maximum iterations (5) reached and the optimization hasn't converged'); the weights are then not trained properly and the truth table may be wrong. For logic gates the data set has only four rows, so a small learning rate or an unlucky initialisation can need many more iterations. Increasing max_iter (or the learning rate, or using solver = 'lbfgs', which converged in only 20 and 23 iterations here) allows training to finish.

CONCLUSION:
Signature & Date:

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:12	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	
PROBLEM STATEMENT	Implementation of OR/NOR gate using feed forward Neural Network	
PREREQUISITES	Basic understanding of logic gates and truth tables (OR/NOR) Concept of artificial neural networks (neurons, weights, activation functions) Basic idea of supervised learning and training iterations (epochs)	
OBJECTIVES:	To understand the structure and working of a feed-forward neural network. To implement a neural network that learns the OR/NOR logic gates. To verify the network's output against the truth table	
Tools Used	Python 3.x Jupyter Notebook Libraries: numpy, scikit-learn	
THEORY:		

Perceptron Learning Algorithm: The Perceptron is the simplest form of an artificial neural network, acting as a linear binary classifier. It takes a set of input features, multiplies them by weights, adds a bias, and passes the result through a step activation function to yield a binary output (0 or 1, or -1 and 1).

Step-by-Step Algorithm

1. **Initialize parameters:** Set all weights W and the bias b to 0 or small random numbers. Define a learning rate L, usually between 0 and 1).
2. **Compute the output:** For a training sample with inputs $x = [x_1, x_2, ..., x_n]$

$$\hat{y} = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

Where f(z) is a hard step activation function:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

3. **Update parameters:** Compare the predicted output

$$w_i \leftarrow w_i + \eta \cdot (y - \hat{y}) \cdot x_i$$

$$b \leftarrow b + \eta \cdot (y - \hat{y})$$

4. **Iterate:** Repeat steps 2 and 3 for all training examples (one epoch) until the network classifies all items correctly or reaches a maximum iteration limit.

ALGORITHM

Steps:

1. Write the truth table: inputs X = [[0,0],[0,1],[1,0],[1,1]]; targets OR = [0, 1, 1, 1], NOR = [1, 0, 0, 0].
2. Initialise the weights w1 = w2 = 0 and the bias b = 0; choose the learning rate L = 0.1.
3. For one training row compute z = w1*A + w2*B + b and the output y' = f(z), where f(z) = 1 if z >= 0, otherwise 0 (step activation).
4. Find the error = target - y'.
5. Update the parameters: w = w + L * error * x and b = b + L * error.
6. Repeat steps 3-5 for all four rows (one epoch) until an epoch has no errors or the maximum number of epochs is reached.
7. Verify: apply the trained network to all four inputs and compare the outputs with the truth table.
8. Repeat with a feed-forward network (scikit-learn MLPClassifier, 4 hidden neurons, backpropagation) and plot the decision line.

ERRORS and TEST CASES, USE CASES

Errors and how to avoid them

Error / symptom	Cause and remedy
Every output is 1 (accuracy 0.75 for OR, 0.25 for NOR)	learning rate set to 0, so the weights never change
ValueError: matmul dimension mismatch	an input with 3 values was given - the gate has exactly 2 inputs (A, B)
ConvergenceWarning: maximum iterations reached	increase max_iter (or use solver='lbfgs' for tiny data)
Not converged within the epochs	epochs too small - OR / NOR are linearly separable, so more epochs will converge

Test cases

No.	Input / condition	Expected output
TC1	Inputs (0,0), (0,1), (1,0), (1,1) to the trained OR network	outputs 0, 1, 1, 1 - truth table verified
TC2	The same four inputs to the trained NOR network	outputs 1, 0, 0, 0 - truth table verified
TC3	Train OR / NOR with learning rate 0.1	OR: 4 epochs (w = [0.1, 0.1], b = -0.1); NOR: 4 epochs (w = [-0.1, -0.1], b = 0)
TC4	Learning rate = 0 (OR)	weights stay 0, every output is 1, accuracy 0.75
TC5	MLPClassifier(max_iter = 5)	ConvergenceWarning: maximum iterations (5) reached
TC6	MLP predict() with a 3-value input	ValueError: X has 3 features, but MLPClassifier is expecting 2 features

Use cases

- Alarm and safety systems: OR logic raises an alarm when any one sensor is active.
- Digital logic: NOR is a universal gate, so NOR neurons can build any Boolean circuit.
- Understanding weights, bias and activation of a neuron before moving to deeper networks.

REFERENCES: Refer lab manual for the same

Important Questions: -

1. What is a perceptron, and how does it relate to a feed-forward neural network?
2. Why are AND and NAND gates considered linearly separable problems?
3. What is the role of the activation function in a neural network?
4. What is backpropagation, and how does it help in training the model?
5. What is the difference between MLPClassifier and a simple single-layer Perceptron in scikit-learn?
6. Why might increasing max_iter be necessary for some logic gate models to converge?

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:12	LAB Ref: BE/2026-27/	Starting date:
	Roll No:	
PROBLEM STATEMENT	Implementation of OR/NOR gate using feed forward Neural Network	

Libraries: numpy, matplotlib, scikit-learn. The OR and NOR gates are learned by (a) a single neuron (perceptron) written from scratch with the perceptron learning rule and (b) a feed-forward network (MLPClassifier) from scikit-learn.

PROGRAM AND OUTPUT

Import libraries and create the truth tables

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import Perceptron

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_or = np.array([0, 1, 1, 1])
y_nor = np.array([1, 0, 0, 0])

print("A B | OR NOR")
for (a, b), t1, t2 in zip(X, y_or, y_nor):
    print(a, b, "|", t1, "  ", t2)
```

Output:

```
A B | OR NOR
0 0 | 0    1
0 1 | 1    0
1 0 | 1    0
1 1 | 1    0
```

Perceptron (single neuron) class and verification function

Code:

```
class SimplePerceptron:
    """Single neuron with a step activation, trained with the perceptron learning rule."""
    def __init__(self, lr=0.1, epochs=100):
        self.lr = lr
        self.epochs = epochs

    def step(self, z):
        return np.where(z >= 0, 1, 0)

    def predict(self, X):
        return self.step(X @ self.w + self.b)

    def fit(self, X, y, verbose=True):
        self.w = np.zeros(X.shape[1])
        self.b = 0.0
        for epoch in range(1, self.epochs + 1):
            errors = 0
            for xi, target in zip(X, y):
                out = self.step(xi @ self.w + self.b)
                update = self.lr * (target - out)
                self.w = self.w + update * xi
                self.b = self.b + update
                errors += int(update != 0)
            if verbose:
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:12	LAB Ref: BE/2026-27/	Starting date:
	Roll No:	
PROBLEM STATEMENT	Implementation of OR/NOR gate using feed forward Neural Network	

```
                print("Epoch", epoch, "-> errors:", errors, " w =", self.w.round(2), " b =", round(self.b,
2))
            if errors == 0:
                print("Converged after", epoch, "epochs")
                break
        return self


def verify(model, X, y, name):
    pred = model.predict(X)
    print(name, "- truth table verification")
    print("A B | Expected | Predicted | Result")
    for (a, b), t, p in zip(X, y, pred):
        print(a, b, "|    ", t, "   |    ", p, "    |", "Correct" if t == p else "WRONG")
    print("Accuracy:", (pred == y).mean())
```

Output:

Train the network for the OR gate

Code:

```
or_model = SimplePerceptron(lr=0.1, epochs=50)
or_model.fit(X, y_or)
print("\nFinal weights:", or_model.w.round(2), " bias:", round(or_model.b, 2), "\n")
verify(or_model, X, y_or, "OR")
```

Output:

```
Epoch 1 -> errors: 2  w = [0.  0.1]  b = 0.0
Epoch 2 -> errors: 2  w = [0.1 0.1]  b = 0.0
Epoch 3 -> errors: 1  w = [0.1 0.1]  b = -0.1
Epoch 4 -> errors: 0  w = [0.1 0.1]  b = -0.1
Converged after 4 epochs

Final weights: [0.1 0.1]  bias: -0.1

OR - truth table verification
A B | Expected | Predicted | Result
0 0 |     0    |     0     | Correct
0 1 |     1    |     1     | Correct
1 0 |     1    |     1     | Correct
1 1 |     1    |     1     | Correct
Accuracy: 1.0
```

Observation: the neuron learns the OR gate in 4 epochs. The final rule 0.1A + 0.1B - 0.1 >= 0 is false only when A = B = 0.

Train the network for the NOR gate

Code:

```
nor_model = SimplePerceptron(lr=0.1, epochs=50)
nor_model.fit(X, y_nor)
print("\nFinal weights:", nor_model.w.round(2), " bias:", round(nor_model.b, 2), "\n")
verify(nor_model, X, y_nor, "NOR")
```

Output:

```
Epoch 1 -> errors: 1  w = [ 0.  -0.1]  b = -0.1
Epoch 2 -> errors: 2  w = [-0.1 -0.1]  b = -0.1
Epoch 3 -> errors: 1  w = [-0.1 -0.1]  b = 0.0
Epoch 4 -> errors: 0  w = [-0.1 -0.1]  b = 0.0
```

```
Converged after 4 epochs

Final weights: [-0.1 -0.1]  bias: 0.0

NOR - truth table verification
A B | Expected | Predicted | Result
0 0 |     1    |     1     | Correct
0 1 |     0    |     0     | Correct
1 0 |     0    |     0     | Correct
1 1 |     0    |     0     | Correct
Accuracy: 1.0
```

Observation: NOR is learned in 4 epochs. The rule -0.1A - 0.1B >= 0 is true only when A = B = 0 (the output 1 for input (0, 0) comes from z = 0, which the step function maps to 1).

Feed-forward neural network with scikit-learn (MLPClassifier)

Code:

```
for name, target in [("OR", y_or), ("NOR", y_nor)]:
    mlp = MLPClassifier(hidden_layer_sizes=(4,), activation="logistic", solver="lbfgs",
                        max_iter=1000, random_state=1)
    mlp.fit(X, target)
    print(name, "gate: iterations =", mlp.n_iter_, " final loss =", round(mlp.loss_, 4))
    print("Probability of output 1:", mlp.predict_proba(X)[:, 1].round(3))
    verify(mlp, X, target, name)
    print()
```

Output:

```
OR gate: iterations = 24  final loss = 0.0031
Probability of output 1: [0.001 1.    1.    1.   ]
OR - truth table verification
A B | Expected | Predicted | Result
0 0 |     0    |     0     | Correct
0 1 |     1    |     1     | Correct
1 0 |     1    |     1     | Correct
1 1 |     1    |     1     | Correct
Accuracy: 1.0

NOR gate: iterations = 26  final loss = 0.0033
Probability of output 1: [0.999 0.    0.    0.   ]
NOR - truth table verification
A B | Expected | Predicted | Result
0 0 |     1    |     1     | Correct
0 1 |     0    |     0     | Correct
1 0 |     0    |     0     | Correct
1 1 |     0    |     0     | Correct
Accuracy: 1.0
```

Observation: the multi-layer network also reproduces both truth tables, needing 24 and 26 iterations with the lbfgs solver.

scikit-learn Perceptron for comparison

Code:

```
for name, target in [("OR", y_or), ("NOR", y_nor)]:
    p = Perceptron(max_iter=1000, tol=None, random_state=0).fit(X, target)
    print(name, "-> accuracy:", p.score(X, target), " weights:", p.coef_[0], " bias:", p.intercept_[0])
```

Output:

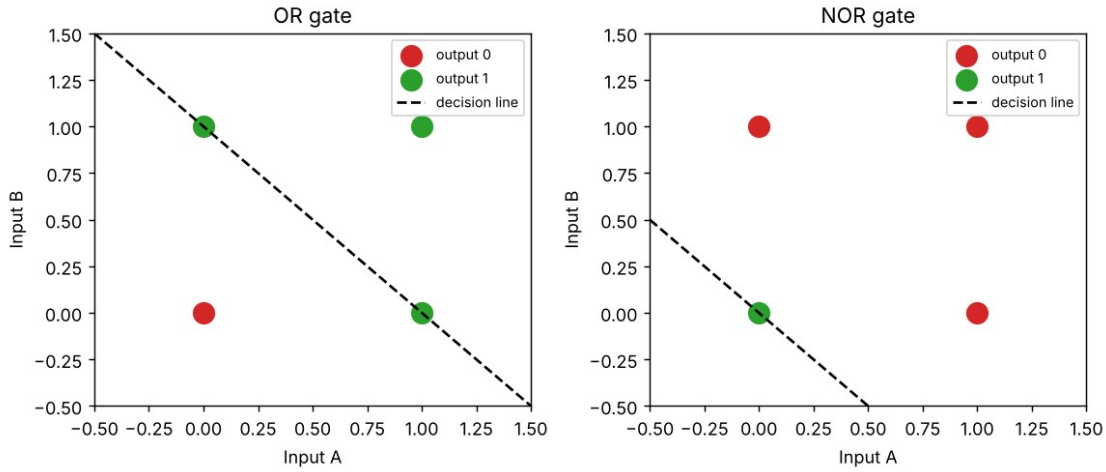
```
OR -> accuracy: 1.0  weights: [2. 2.]  bias: -1.0
NOR -> accuracy: 1.0  weights: [-2. -2.]  bias: 1.0
```

Decision boundary learned by the neuron

Code:

```
fig, ax = plt.subplots(1, 2, figsize=(9, 4))
for a, name, model, target in [(ax[0], "OR", or_model, y_or), (ax[1], "NOR", nor_model, y_nor)]:
    a.scatter(X[target == 0, 0], X[target == 0, 1], s=140, color="tab:red", label="output 0")
    a.scatter(X[target == 1, 0], X[target == 1, 1], s=140, color="tab:green", label="output 1")
    xs = np.linspace(-0.5, 1.5, 50)
    ys = -(model.w[0] * xs + model.b) / model.w[1]
    a.plot(xs, ys, "k--", label="decision line")
    a.set_xlim(-0.5, 1.5)
    a.set_ylim(-0.5, 1.5)
    a.set_title(name + " gate")
    a.set_xlabel("Input A")
    a.set_ylabel("Input B")
    a.legend(loc="upper right", fontsize=8)
plt.tight_layout()
plt.show()
```

Output:



Observation: one straight (dashed) line separates the output-0 points from the output-1 points for both gates, so OR and NOR are linearly separable.

IMPORTANT QUESTIONS AND ANSWERS

Q1. What is a perceptron, and how does it relate to a feed-forward neural network?

A perceptron is a single artificial neuron: it computes the weighted sum z = w1*A + w2*B + b of its inputs and passes it through a step activation to give a 0 / 1 output. A feed-forward neural network is made of layers of such neurons (input layer, one or more hidden layers, output layer) in which the signals travel only forward, never in a loop. The perceptron is therefore the simplest feed-forward network (no hidden layer) and the basic building block of larger ones such as

MLPClassifier.

Q2. Why are AND and NAND gates considered linearly separable problems?

A problem is linearly separable if one straight line can separate the inputs whose output is 0 from those whose output is 1. For AND only the point (1, 1) gives 1, so a line such as 0.2A + 0.1B - 0.2 = 0 puts (1, 1) on one side and the other three points on the other; NAND is its complement, so the same line with reversed signs separates it. The gates of this experiment behave in the same way: for OR only (0, 0) gives 0, so the line 0.1A + 0.1B - 0.1 = 0 separates it from the other three points, and NOR is the complement. Since all four gates are linearly separable, the perceptron rule is guaranteed to converge (4 epochs for both OR and NOR here).

Q3. What is the role of the activation function in a neural network?

The activation function decides the output of a neuron from its weighted sum and adds non-linearity. Without it, any number of layers would collapse into a single linear function. The step function (0 / 1) is used by the perceptron; sigmoid / logistic, tanh and ReLU are used in multi-layer networks because they are differentiable, which backpropagation needs. In this experiment the from-scratch neuron uses a step function and MLPClassifier uses the logistic (sigmoid) function.

Q4. What is backpropagation, and how does it help in training the model?

Backpropagation computes the gradient of the loss with respect to every weight by applying the chain rule from the output layer back to the input layer. The weights are then moved against the gradient (w = w - L * dLoss/dw), and the process is repeated until the loss is small. It lets multi-layer networks be trained efficiently. The simple perceptron rule used for the single neuron does not need it, but MLPClassifier trains its weights this way.

Q5. What is the difference between MLPClassifier and a simple single-layer Perceptron in scikit-learn?

	Perceptron	**MLPClassifier**
Layers	Single layer (no hidden layer)	One or more hidden layers
Decision boundary	Linear (one straight line / plane)	Non-linear
Training	Perceptron update rule	Backpropagation with sgd, adam or lbfgs
Output	Class only	Class and probabilities (predict_proba)
Can solve XOR?	No	Yes (with a hidden layer)

Q6. Why might increasing max_iter be necessary for some logic gate models to converge?

Iterative solvers (sgd, adam, lbfgs) stop either when the loss stops improving or when max_iter is reached. If max_iter is too small the loss is still high when training stops and scikit-learn raises a ConvergenceWarning (in our test, MLPClassifier(max_iter = 5) gave 'Maximum iterations (5) reached and the optimization hasn't converged'); the weights are then not trained properly and the truth table may be wrong. For logic gates the data set has only four rows, so a small learning rate or an unlucky initialisation can need many more iterations. Increasing max_iter (or the learning rate, or using solver = 'lbfgs', which converged in only 24 and 26 iterations here) allows training to finish.

CONCLUSION:
Signature & Date:

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:13	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	
PROBLEM STATEMENT	Implementation of EX-OR gate using feed forward Neural Network	
PREREQUISITES	Basic understanding of logic gates and truth tables (AND, NAND) Concept of artificial neural networks (neurons, weights, activation functions) Basic idea of supervised learning and training iterations (epochs)	
OBJECTIVES:	To understand the structure and working of a feed-forward neural network. To implement a neural network that learns the EX-OR logic gates. To verify the network's output against the truth table	
Tools Used	Python 3.x Jupyter Notebook Libraries: numpy, scikit-learn	
THEORY:		

Perceptron Learning Algorithm: The Perceptron is the simplest form of an artificial neural network, acting as a linear binary classifier. It takes a set of input features, multiplies them by weights, adds a bias, and passes the result through a step activation function to yield a binary output (0 or 1, or -1 and 1).

Step-by-Step Algorithm

1. **Initialize parameters:** Set all weights W and the bias b to 0 or small random numbers. Define a learning rate L, usually between 0 and 1).
2. **Compute the output:** For a training sample with inputs $x = [x_1, x_2, ..., x_n]$

$$\hat{y} = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

Where f(z) is a hard step activation function:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

3. **Update parameters:** Compare the predicted output

$$w_i \leftarrow w_i + \eta \cdot (y - \hat{y}) \cdot x_i$$

$$b \leftarrow b + \eta \cdot (y - \hat{y})$$

4. **Iterate:** Repeat steps 2 and 3 for all training examples (one epoch) until the network classifies all items correctly or reaches a maximum iteration limit.

ALGORITHM

Steps:

1. Write the XOR truth table: inputs [[0,0],[0,1],[1,0],[1,1]], targets [0, 1, 1, 0].
2. Show that a single neuron cannot learn XOR (the classes are not linearly separable): train the perceptron and watch the error count never reach 0.
3. Build a feed-forward network 2-4-1: input layer (A, B), hidden layer of 4 sigmoid neurons, output layer of 1 sigmoid neuron.
4. Initialise the weights with small random numbers and the biases with 0; choose the learning rate (0.5).
5. Forward pass: a1 = sigmoid(X W1 + b1) and a2 = sigmoid(a1 W2 + b2).
6. Compute the loss = mean (target - a2)^2.
7. Backward pass (backpropagation): d2 = (a2 - t) a2 (1 - a2); d1 = (d2 W2^T) a1 (1 - a1).
8. Update: W2 = W2 - L a1^T d2, W1 = W1 - L X^T d1 (and the biases); repeat for many epochs until the loss is small.
9. Verify: output > 0.5 means 1; compare with the truth table and with scikit-learn's MLPClassifier.

ERRORS and TEST CASES, USE CASES

Errors and how to avoid them

Error / symptom	Cause and remedy
Accuracy stays 0.5 with a single neuron	XOR is not linearly separable - a hidden layer is required
MLP accuracy 0.5 with only 2 hidden neurons (random_state = 1)	training is stuck in a poor local minimum - use more hidden neurons or another seed
Loss does not fall (or becomes nan)	learning rate too small / too large - tune it
ConvergenceWarning: maximum iterations reached	increase max_iter

Test cases

No.	Input / condition	Expected output
TC1	Single neuron (perceptron rule) trained on XOR, 100 epochs	errors never reach 0 (3, 3, 4, 4, ...); accuracy 0.5
TC2	scikit-learn Perceptron on XOR	accuracy 0.5
TC3	Hand-designed network: XOR = AND(OR, NAND)	outputs 0, 1, 1, 0 - matches the truth table
TC4	Backpropagation network 2-4-1, 10000 epochs	outputs 0.007, 0.981, 0.981, 0.024 - all four correct, loss 0.00034
TC5	MLPClassifier with 4 hidden neurons / with 2 hidden neurons (random_state = 1)	accuracy 1.0 / accuracy 0.5
TC6	MLP predict() with a 3-value input	ValueError: X has 3 features, but MLPClassifier is expecting 2 features

Use cases

- Parity and error detection: XOR is the parity / checksum operation used in communication and storage.
- The classic example of why deep networks need hidden layers (non-linear classification).
- Cryptography and digital circuits (XOR cipher, adders).

REFERENCES: Refer lab manual for the same

Important Questions: -

1. What is a perceptron, and how does it relate to a feed-forward neural network?
2. Why are AND and NAND gates considered linearly separable problems?
3. What is the role of the activation function in a neural network?
4. What is backpropagation, and how does it help in training the model?
5. What is the difference between MLPClassifier and a simple single-layer Perceptron in scikit-learn?
6. Why might increasing max_iter be necessary for some logic gate models to converge?

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:13	LAB Ref: BE/2026-27/	Starting date:
	Roll No:	
PROBLEM STATEMENT	Implementation of EX-OR gate using feed forward Neural Network	

Libraries: numpy, matplotlib, scikit-learn. XOR is first attempted with a single neuron (which fails), then solved with feed-forward networks that have a hidden layer: a hand-designed network, a 2-4-1 network trained with backpropagation written from scratch, and scikit-learn's MLPClassifier.

PROGRAM AND OUTPUT

Import libraries and create the XOR truth table

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import Perceptron

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_xor = np.array([0, 1, 1, 0])

print("A B | XOR")
for (a, b), t in zip(X, y_xor):
    print(a, b, "|", t)
```

Output:

```
A B | XOR
0 0 | 0
0 1 | 1
1 0 | 1
1 1 | 0
```

Single neuron (perceptron) class and verification function

Code:

```
class SimplePerceptron:
    """Single neuron with a step activation, trained with the perceptron learning rule."""
    def __init__(self, lr=0.1, epochs=100):
        self.lr = lr
        self.epochs = epochs

    def step(self, z):
        return np.where(z >= 0, 1, 0)

    def predict(self, X):
        return self.step(X @ self.w + self.b)

    def fit(self, X, y):
        self.w = np.zeros(X.shape[1])
        self.b = 0.0
        self.history = []
        for epoch in range(self.epochs):
            errors = 0
            for xi, target in zip(X, y):
                out = self.step(xi @ self.w + self.b)
                update = self.lr * (target - out)
                self.w = self.w + update * xi
                self.b = self.b + update
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:13	LAB Ref: BE/2026-27/	Starting date:
	Roll No:	
PROBLEM STATEMENT	Implementation of EX-OR gate using feed forward Neural Network	

```
                errors += int(update != 0)
            self.history.append(errors)
            if errors == 0:
                break
        return self

def verify(model, X, y, name):
    pred = model.predict(X)
    print(name, "- truth table verification")
    print("A B | Expected | Predicted | Result")
    for (a, b), t, p in zip(X, y, pred):
        print(a, b, "|    ", t, "   |    ", p, "    |", "Correct" if t == p else "WRONG")
    print("Accuracy:", (pred == y).mean())
```

Output:

A single neuron cannot learn XOR

Code:

```
single = SimplePerceptron(lr=0.1, epochs=100).fit(X, y_xor)
print("Errors in each of the first 12 epochs:", single.history[:12])
print("Epochs run:", len(single.history), " errors in the last epoch:", single.history[-1])
print("Weights:", single.w.round(2), " bias:", round(single.b, 2), "\n")
verify(single, X, y_xor, "XOR (single neuron)")

p = Perceptron(max_iter=1000, tol=None, random_state=0).fit(X, y_xor)
print("\nscikit-learn Perceptron accuracy on XOR:", p.score(X, y_xor))
```

Output:

```
Errors in each of the first 12 epochs: [3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
Epochs run: 100  errors in the last epoch: 4
Weights: [-0.1  0. ]  bias: 0.0

XOR (single neuron) - truth table verification
A B | Expected | Predicted | Result
0 0 |     0    |     1     | WRONG
0 1 |     1    |     1     | Correct
1 0 |     1    |     0     | WRONG
1 1 |     0    |     0     | Correct
Accuracy: 0.5

scikit-learn Perceptron accuracy on XOR: 0.5
```

Observation: the error count never reaches 0 (3, 3, 4, 4, ...) and the accuracy is only 0.5 for both the from-scratch neuron and scikit-learn's Perceptron, because the XOR classes cannot be separated by one straight line.

XOR with a hand-designed two-layer network: XOR = AND(OR, NAND)

Code:

```
def step(z):
    return (z >= 0).astype(int)

W1 = np.array([[1, -1],
               [1, -1]])          # column 1 -> OR neuron, column 2 -> NAND neuron
b1 = np.array([-0.5, 1.5])
W2 = np.array([1, 1])             # output neuron acts as AND
b2 = -1.5
```

```
hidden = step(X @ W1 + b1)
output = step(hidden @ W2 + b2)
print("A B | OR NAND | XOR (output)")
for (a, b), h, o in zip(X, hidden, output):
    print(a, b, "|", h[0], "  ", h[1], "  |", o)
print("Matches XOR truth table:", (output == y_xor).all())
```

Output:

```
A B | OR NAND | XOR (output)
0 0 | 0    1   | 0
0 1 | 1    1   | 1
1 0 | 1    1   | 1
1 1 | 1    0   | 0
Matches XOR truth table: True
```

Observation: the OR and NAND neurons of the hidden layer feed an AND neuron in the output layer, which reproduces XOR exactly. This shows why a hidden layer is needed.

Feed-forward network (2-4-1) trained with backpropagation

Code:

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

rng = np.random.default_rng(0)
hidden_units, lr = 4, 0.5
W1 = rng.normal(0, 1, (2, hidden_units)); b1 = np.zeros(hidden_units)
W2 = rng.normal(0, 1, (hidden_units, 1));  b2 = np.zeros(1)
t = y_xor.reshape(-1, 1)
loss_history = []

for epoch in range(10001):
    a1 = sigmoid(X @ W1 + b1)                # forward pass: hidden layer
    a2 = sigmoid(a1 @ W2 + b2)               # forward pass: output layer
    loss = np.mean((t - a2) ** 2)
    loss_history.append(loss)
    d2 = (a2 - t) * a2 * (1 - a2)            # backward pass: output error
    d1 = (d2 @ W2.T) * a1 * (1 - a1)         # backward pass: hidden error
    W2 -= lr * a1.T @ d2; b2 -= lr * d2.sum(axis=0)
    W1 -= lr * X.T @ d1;  b1 -= lr * d1.sum(axis=0)
    if epoch % 2000 == 0:
        print("Epoch", epoch, " loss =", round(loss, 5))

print("\nOutputs after training:", a2.ravel().round(3))
pred = (a2 > 0.5).astype(int).ravel()
print("A B | Expected | Network output | Result")
for (a, b), e, o, p_ in zip(X, y_xor, a2.ravel(), pred):
    print(a, b, "|    ", e, "   |     ", round(o, 3), "    |", "Correct" if e == p_ else "WRONG")
print("Accuracy:", (pred == y_xor).mean())
```

Output:

```
Epoch 0  loss = 0.343
Epoch 2000  loss = 0.00327
Epoch 4000  loss = 0.00107
Epoch 6000  loss = 0.00063
Epoch 8000  loss = 0.00044
Epoch 10000  loss = 0.00034
```

```
Outputs after training: [0.007 0.981 0.981 0.024]
A B | Expected | Network output | Result
0 0 |     0    |      0.007     | Correct
0 1 |     1    |      0.981     | Correct
1 0 |     1    |      0.981     | Correct
1 1 |     0    |      0.024     | Correct
Accuracy: 1.0
```

Observation: the loss falls from 0.343 to 0.00034 and the four outputs (0.007, 0.981, 0.981, 0.024) are on the correct side of 0.5.

Feed-forward network with scikit-learn (MLPClassifier)

Code:

```
for hidden in [(2,), (4,)]:
    mlp = MLPClassifier(hidden_layer_sizes=hidden, activation="logistic", solver="lbfgs",
                        max_iter=2000, random_state=1)
    mlp.fit(X, y_xor)
    print("Hidden neurons:", hidden[0], " iterations:", mlp.n_iter_, " loss:", round(mlp.loss_, 4),
          " accuracy:", mlp.score(X, y_xor))

best = MLPClassifier(hidden_layer_sizes=(4,), activation="logistic", solver="lbfgs",
                     max_iter=2000, random_state=1).fit(X, y_xor)
print("\nProbability of output 1:", best.predict_proba(X)[:, 1].round(3))
verify(best, X, y_xor, "XOR (MLP, 4 hidden neurons)")
```

Output:

```
Hidden neurons: 2  iterations: 40  loss: 0.3525  accuracy: 0.5
Hidden neurons: 4  iterations: 28  loss: 0.009  accuracy: 1.0

Probability of output 1: [0.002 0.998 0.999 0.001]
XOR (MLP, 4 hidden neurons) - truth table verification
A B | Expected | Predicted | Result
0 0 |     0    |     0     | Correct
0 1 |     1    |     1     | Correct
1 0 |     1    |     1     | Correct
1 1 |     0    |     0     | Correct
Accuracy: 1.0
```

Observation: with 4 hidden neurons XOR is learned (accuracy 1.0), but with only 2 hidden neurons and random_state = 1 the training gets stuck (loss 0.3525, accuracy 0.5).

Loss curve and decision regions

Code:

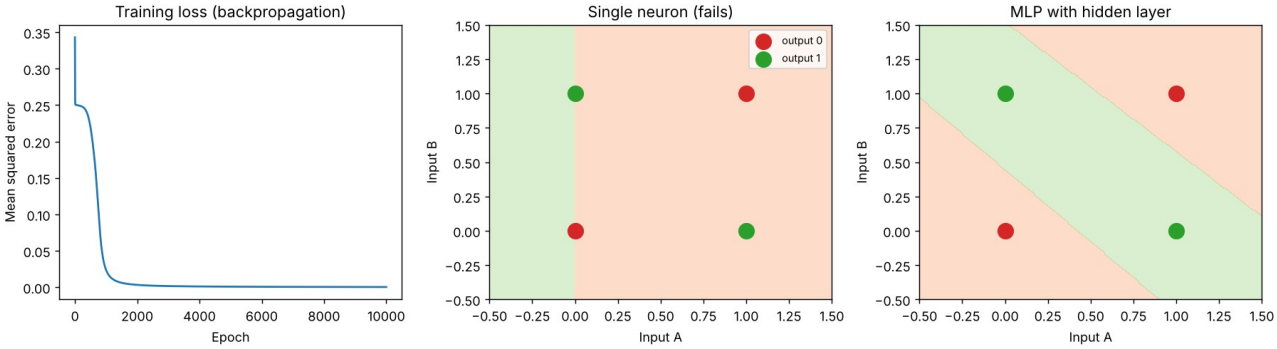
```
fig, ax = plt.subplots(1, 3, figsize=(14, 4))
ax[0].plot(loss_history)
ax[0].set_title("Training loss (backpropagation)")
ax[0].set_xlabel("Epoch")
ax[0].set_ylabel("Mean squared error")

xx, yy = np.meshgrid(np.linspace(-0.5, 1.5, 200), np.linspace(-0.5, 1.5, 200))
grid = np.c_[xx.ravel(), yy.ravel()]
for a, model, title in [(ax[1], single, "Single neuron (fails)"), (ax[2], best, "MLP with hidden layer")]:
    zz = model.predict(grid).reshape(xx.shape)
    a.contourf(xx, yy, zz, alpha=0.3, cmap="RdYlGn", levels=[-0.5, 0.5, 1.5])
```

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	**CLASS: BE**
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:13	**LAB Ref: BE/2026-27/**	**Starting date:**
	Roll No:	
PROBLEM STATEMENT	Implementation of EX-OR gate using feed forward Neural Network	

```
    a.scatter(X[y_xor == 0, 0], X[y_xor == 0, 1], s=140, color="tab:red", label="output 0")
    a.scatter(X[y_xor == 1, 0], X[y_xor == 1, 1], s=140, color="tab:green", label="output 1")
    a.set_title(title)
    a.set_xlabel("Input A")
    a.set_ylabel("Input B")
ax[1].legend(loc="upper right", fontsize=8)
plt.tight_layout()
plt.show()
```

Output:



Observation: the single neuron's decision regions misclassify two of the four points, while the network with a hidden layer forms the two diagonal regions needed for XOR.

IMPORTANT QUESTIONS AND ANSWERS

Q1. What is a perceptron, and how does it relate to a feed-forward neural network?

A perceptron is a single artificial neuron: it computes the weighted sum z = w1*A + w2*B + b of its inputs and passes it through a step activation to give a 0 / 1 output. A feed-forward neural network is made of layers of such neurons (input layer, one or more hidden layers, output layer) in which the signals travel only forward, never in a loop. The perceptron is therefore the simplest feed-forward network (no hidden layer) and the basic building block of larger ones such as MLPClassifier.

Q2. Why are AND and NAND gates considered linearly separable problems?

For AND only the point (1, 1) gives 1, and NAND is its complement, so a single straight line separates the 0-outputs from the 1-outputs in both cases (for example 0.2A + 0.1B - 0.2 = 0), i.e. they are linearly separable and a single neuron can learn them. XOR is different: it gives 1 for (0, 1) and (1, 0) but 0 for (0, 0) and (1, 1), which lie on opposite diagonals, so no single straight line can separate the two classes. That is why the single neuron in this experiment stayed at accuracy 0.5 and a hidden layer was needed.

Q3. What is the role of the activation function in a neural network?

The activation function decides the output of a neuron from its weighted sum and adds non-linearity. Without it, any number of layers would collapse into a single linear function. The step function (0 / 1) is used by the perceptron; sigmoid / logistic, tanh and ReLU are used in multi-layer networks because they are differentiable, which backpropagation needs. In this experiment the from-scratch neuron uses a step function and MLPClassifier uses the logistic (sigmoid) function.

	SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY PUNE - 411043	
	Department of Electronics & Computer Engineering	
	ASSESSMENT YEAR: 2026-2027	CLASS: BE
	SUBJECT: Laboratory Practice I-Data Science & Visualization [410346]	
EXPT No:13	LAB Ref: BE/2026-27/	Starting date:
	Roll No:	
PROBLEM STATEMENT	Implementation of EX-OR gate using feed forward Neural Network	

Q4. What is backpropagation, and how does it help in training the model?

Backpropagation computes the gradient of the loss with respect to every weight by applying the chain rule from the output layer back to the input layer. The weights are then moved against the gradient (w = w - L * dLoss/dw), and the process is repeated until the loss is small. It lets multi-layer networks be trained efficiently. The simple perceptron rule used for the single neuron does not need it, but MLPClassifier trains its weights this way.

Q5. What is the difference between MLPClassifier and a simple single-layer Perceptron in scikit-learn?

	Perceptron	MLPClassifier
Layers	Single layer (no hidden layer)	One or more hidden layers
Decision boundary	Linear (one straight line / plane)	Non-linear
Training	Perceptron update rule	Backpropagation with sgd, adam or lbfgs
Output	Class only	Class and probabilities (predict_proba)
Can solve XOR?	No	Yes (with a hidden layer)

Q6. Why might increasing max_iter be necessary for some logic gate models to converge?

Iterative solvers (sgd, adam, lbfgs) stop either when the loss stops improving or when max_iter is reached. If max_iter is too small the loss is still high when training stops and scikit-learn raises a ConvergenceWarning (in our test, MLPClassifier(max_iter = 5) gave 'Maximum iterations (5) reached and the optimization hasn't converged'). XOR needs more training than the linear gates because it needs a hidden layer: here max_iter = 2000 was used, and lbfgs converged in 28 iterations with 4 hidden neurons. Note that more iterations cannot rescue every run: with only 2 hidden neurons (random_state = 1) training stopped at a poor local minimum with accuracy 0.5, so the number of hidden neurons and the initialisation matter as well.

CONCLUSION:
Signature & Date: